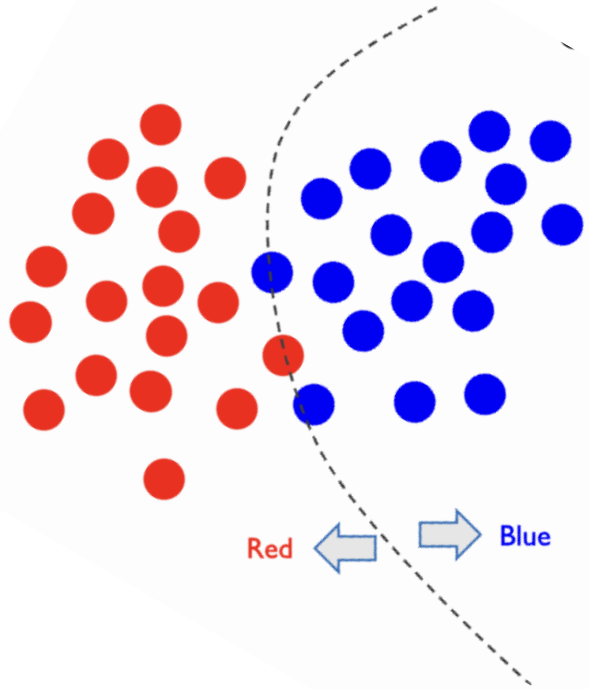
Reconocimiento de Patrones

Versión 2026



Clasificadores Básicos

(CAPÍTULO 4 – PR.04.A)

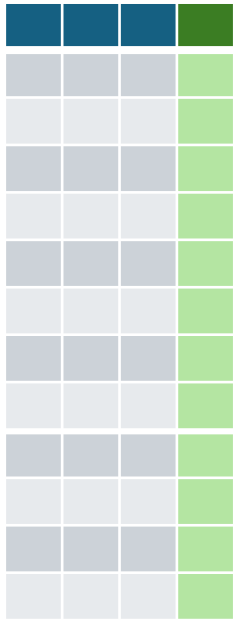


Domingo Mery

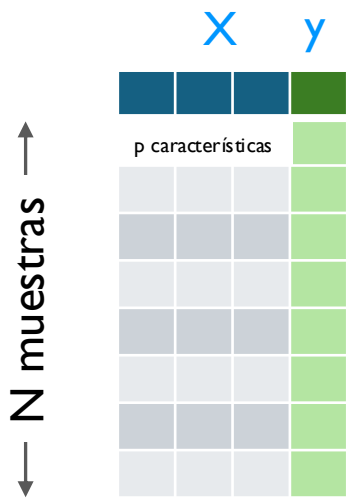
Ciencia de la Computación
Universidad Católica – Chile
http://domingomery.ing.uc.cl

Conjunto de Datos

Características Labels

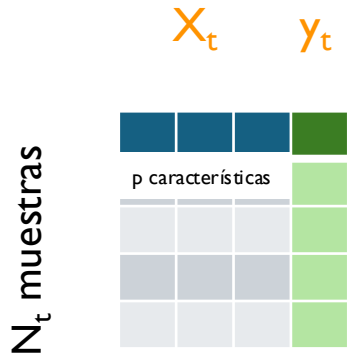


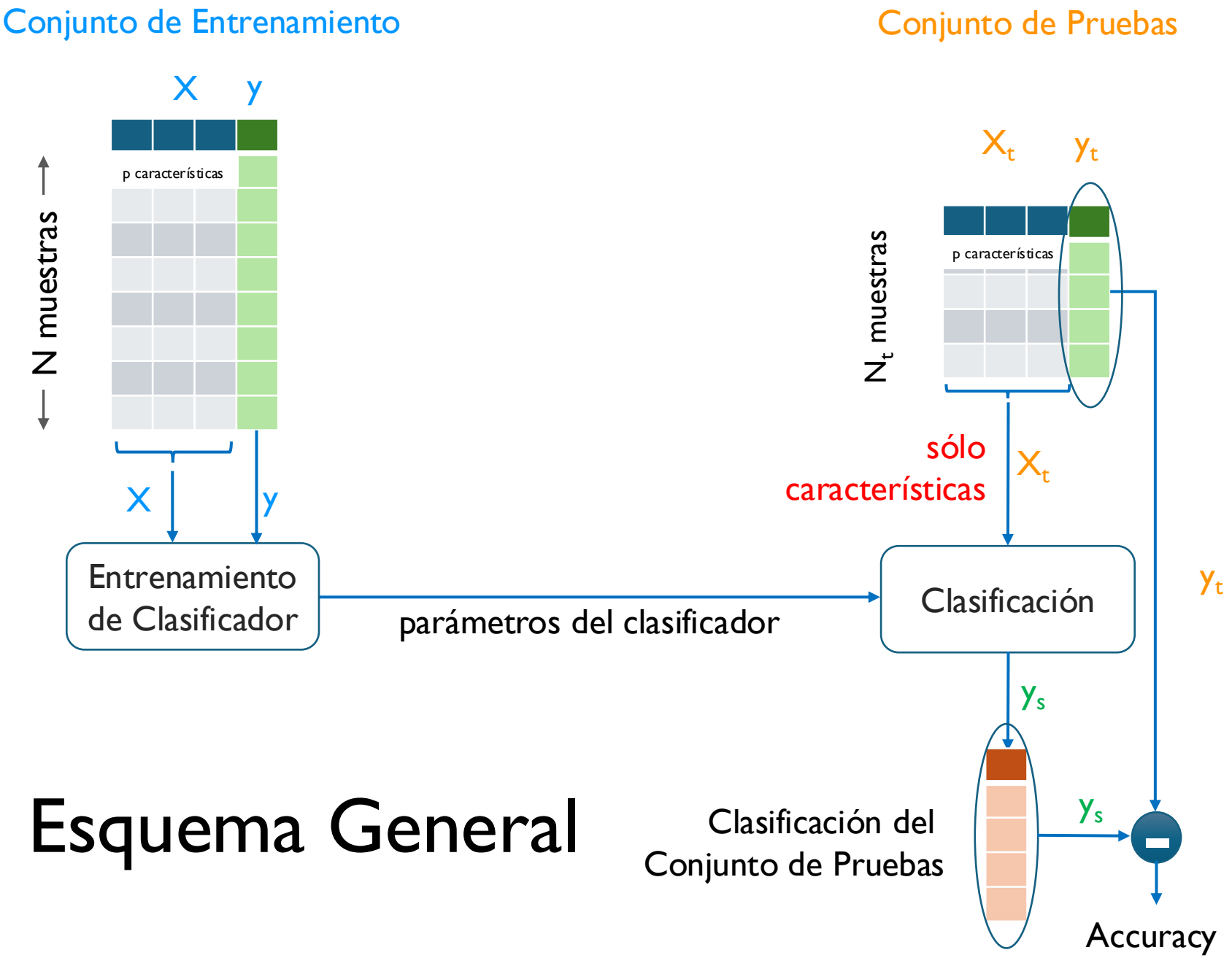
Conjunto de Entrenamiento



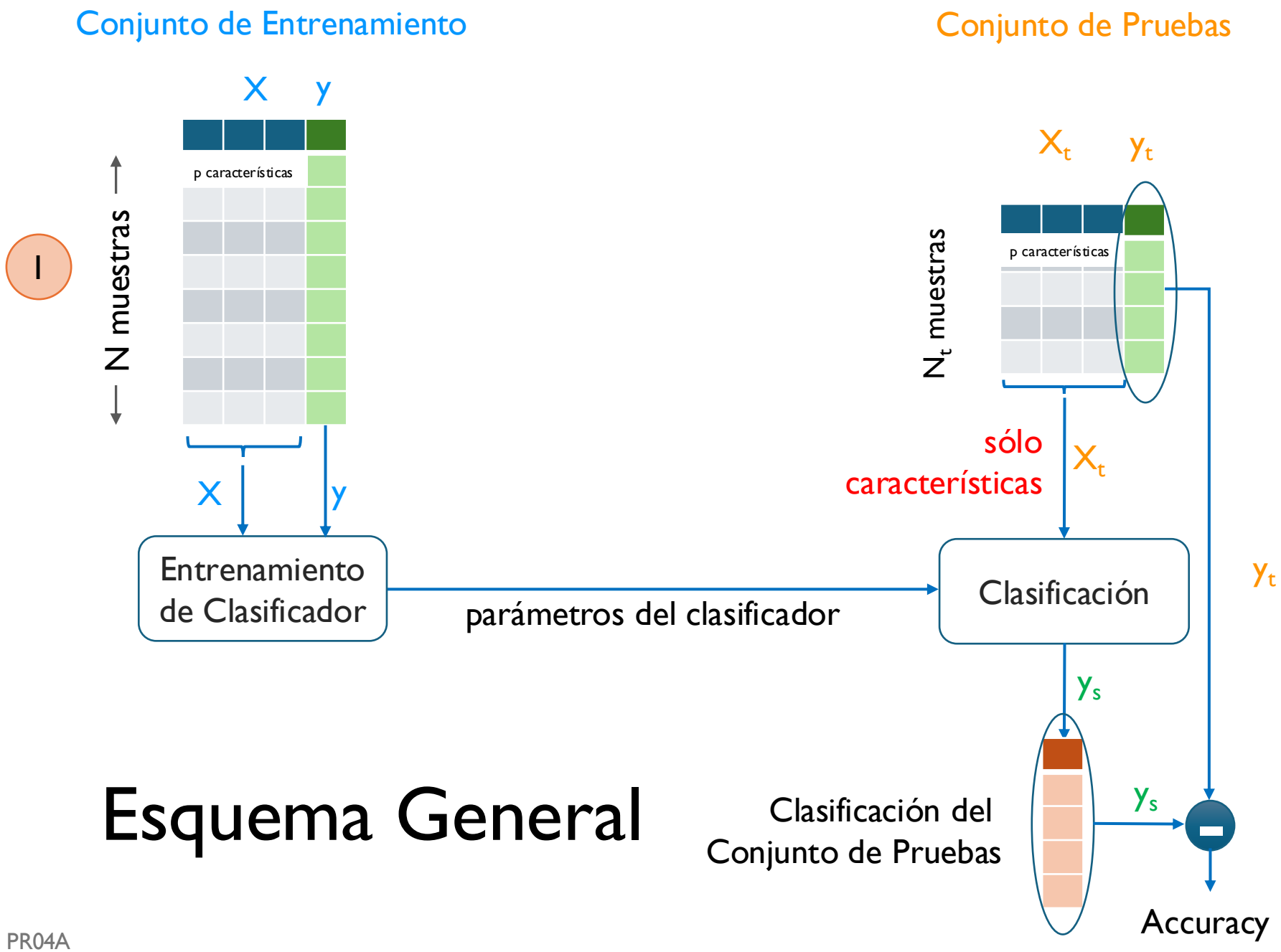
0

Conjunto de Pruebas

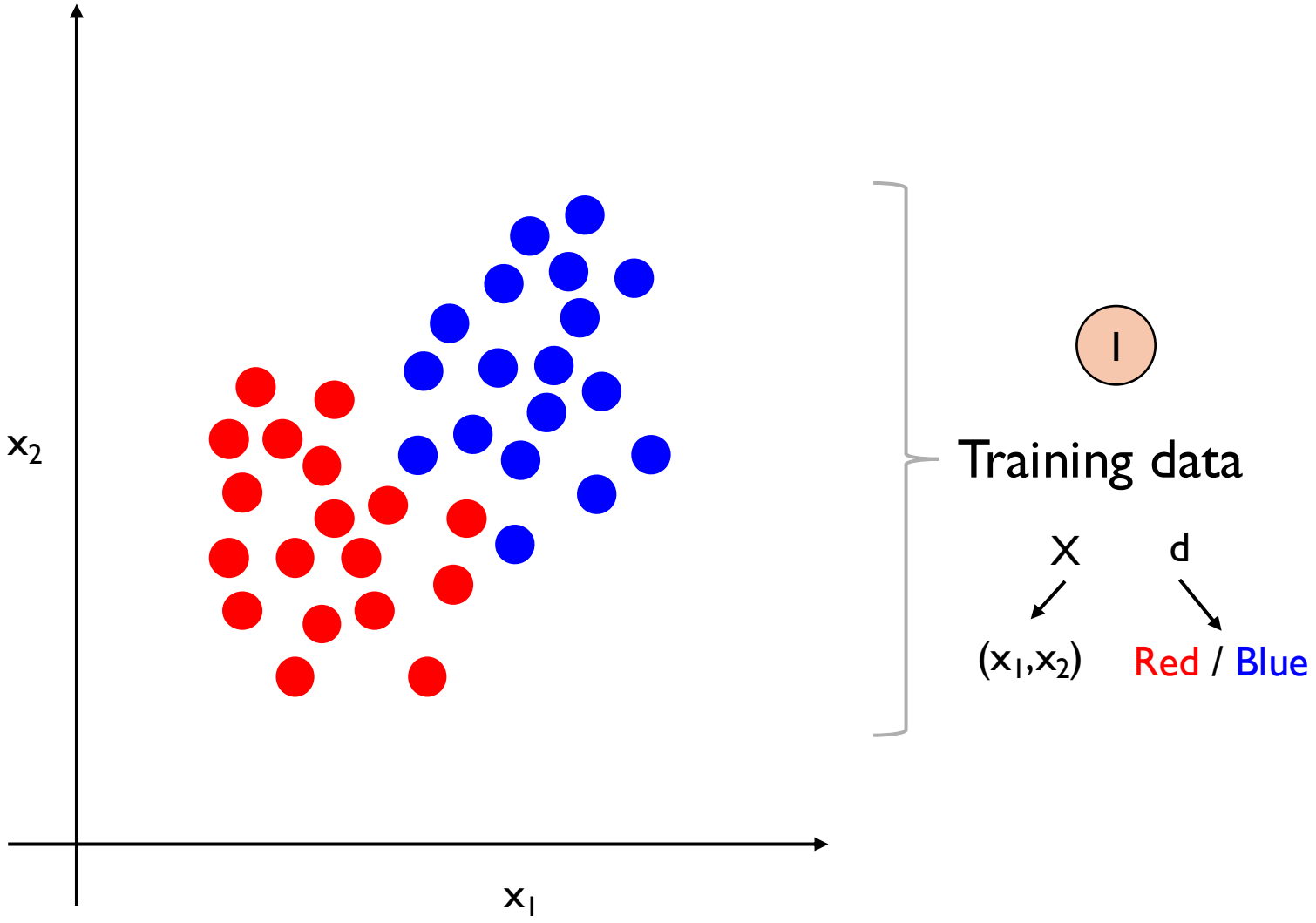


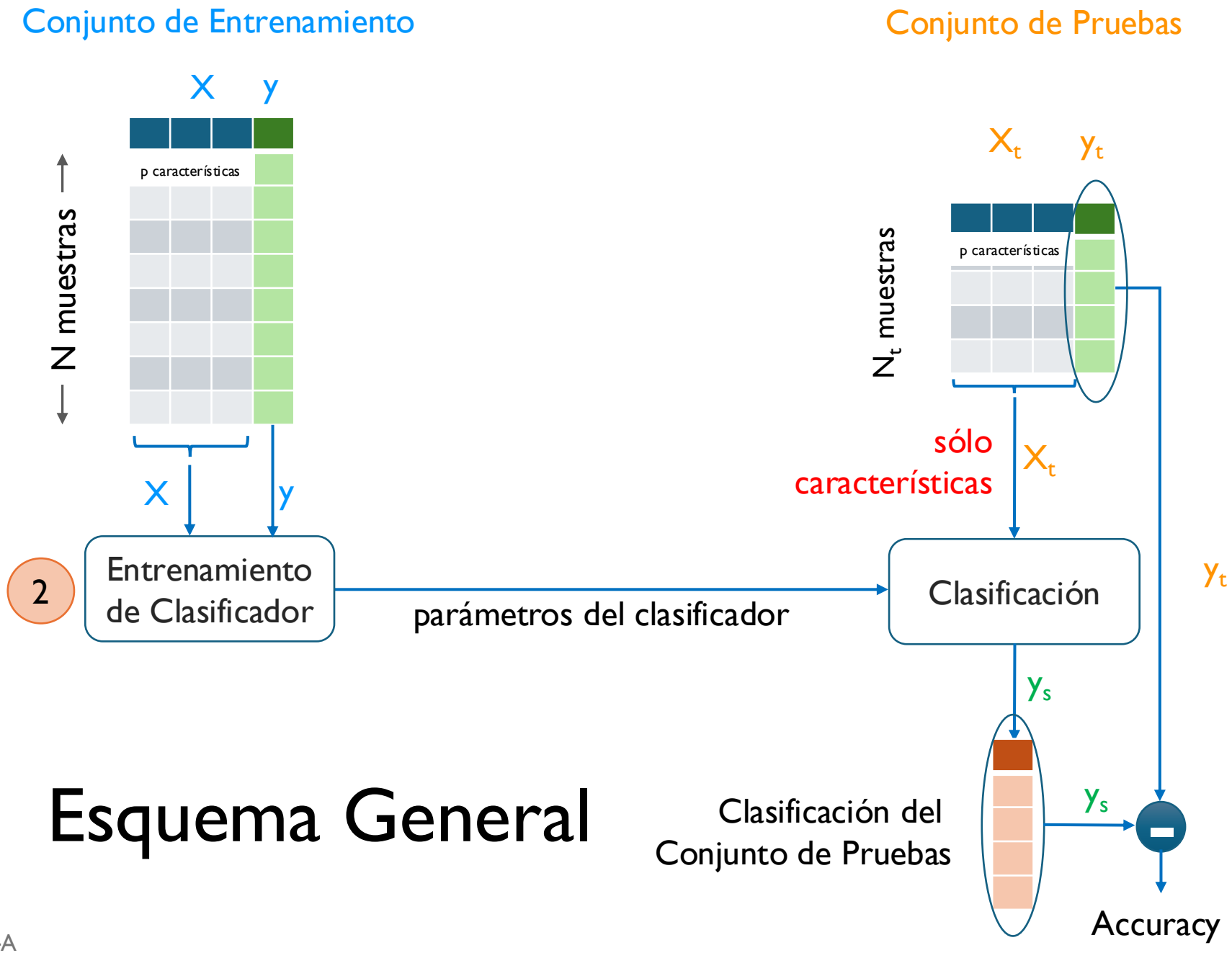


Esquema General

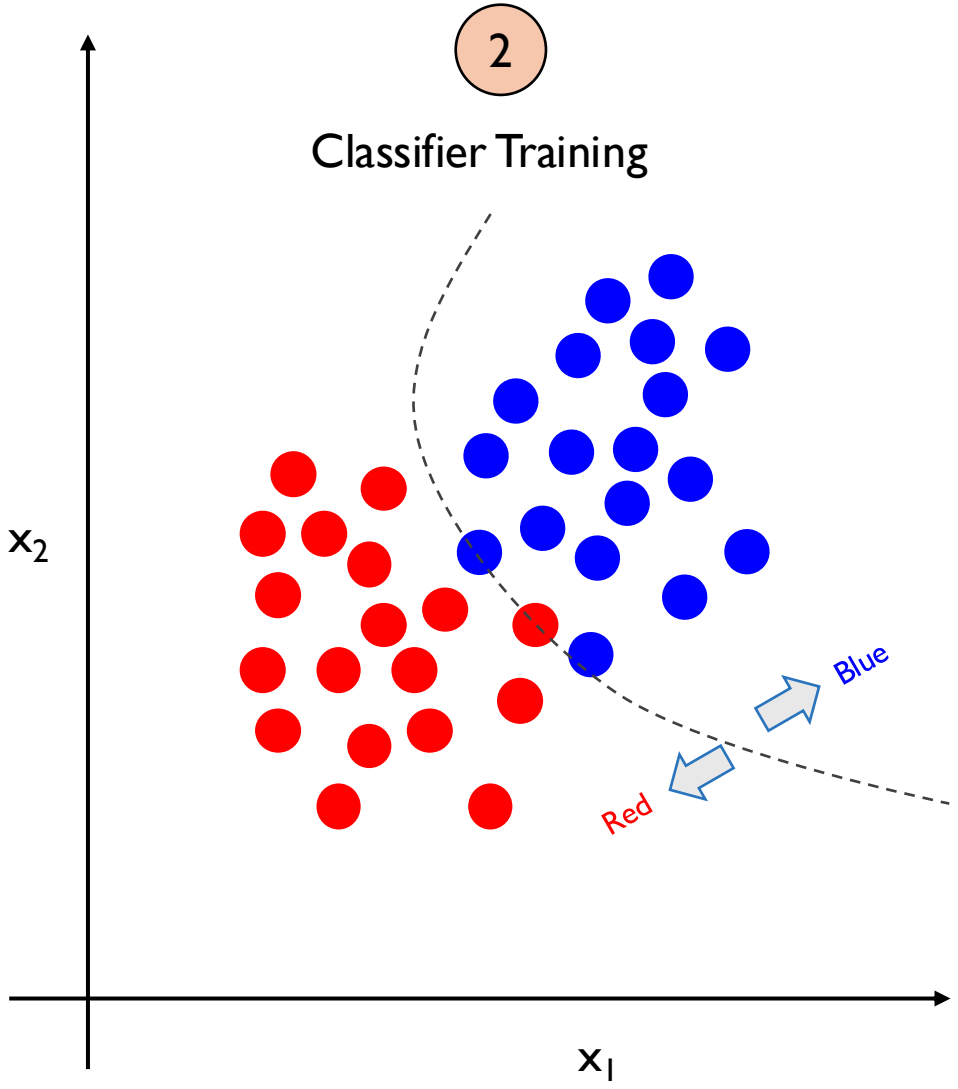


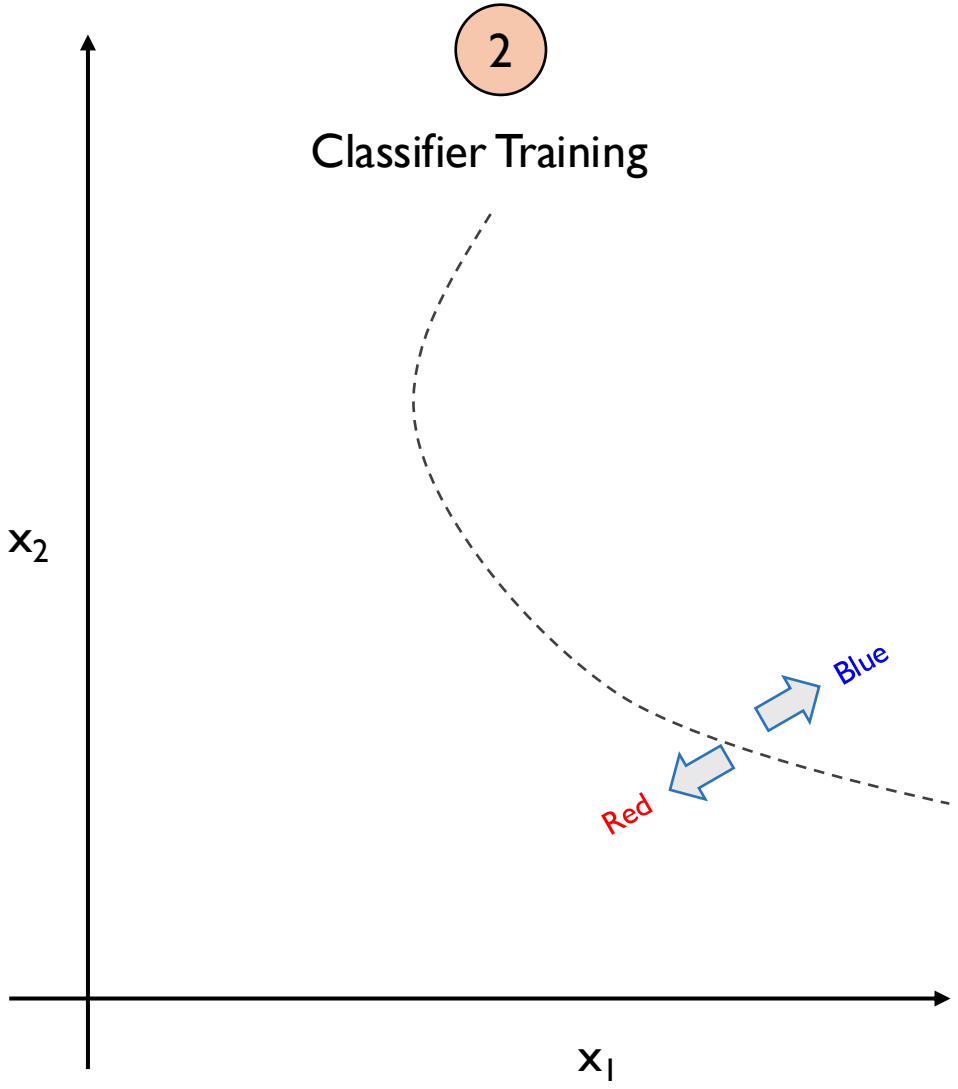
Esquema General

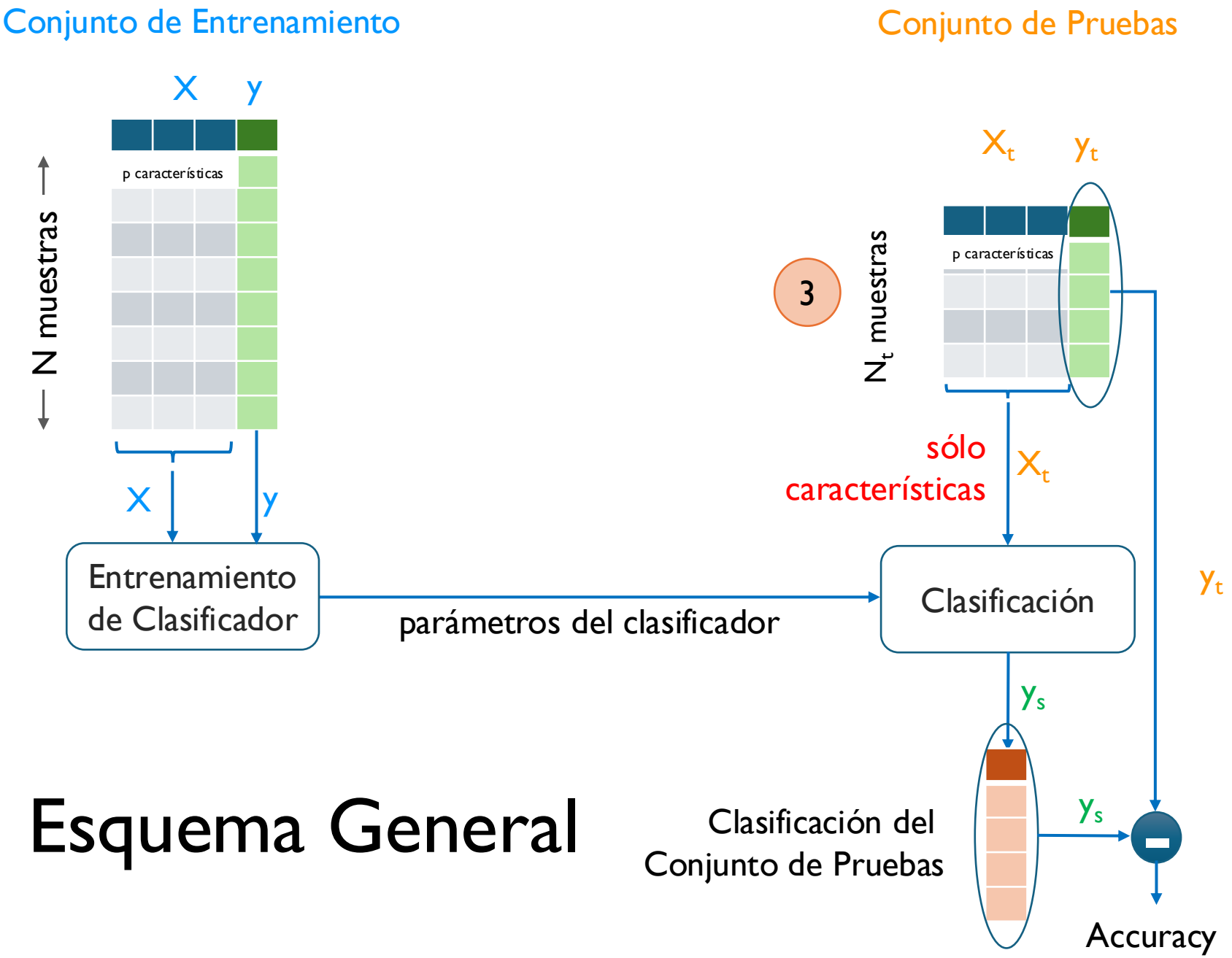

x2
x1
I
Training data
X
d
(x1,x2)
Red / Blue



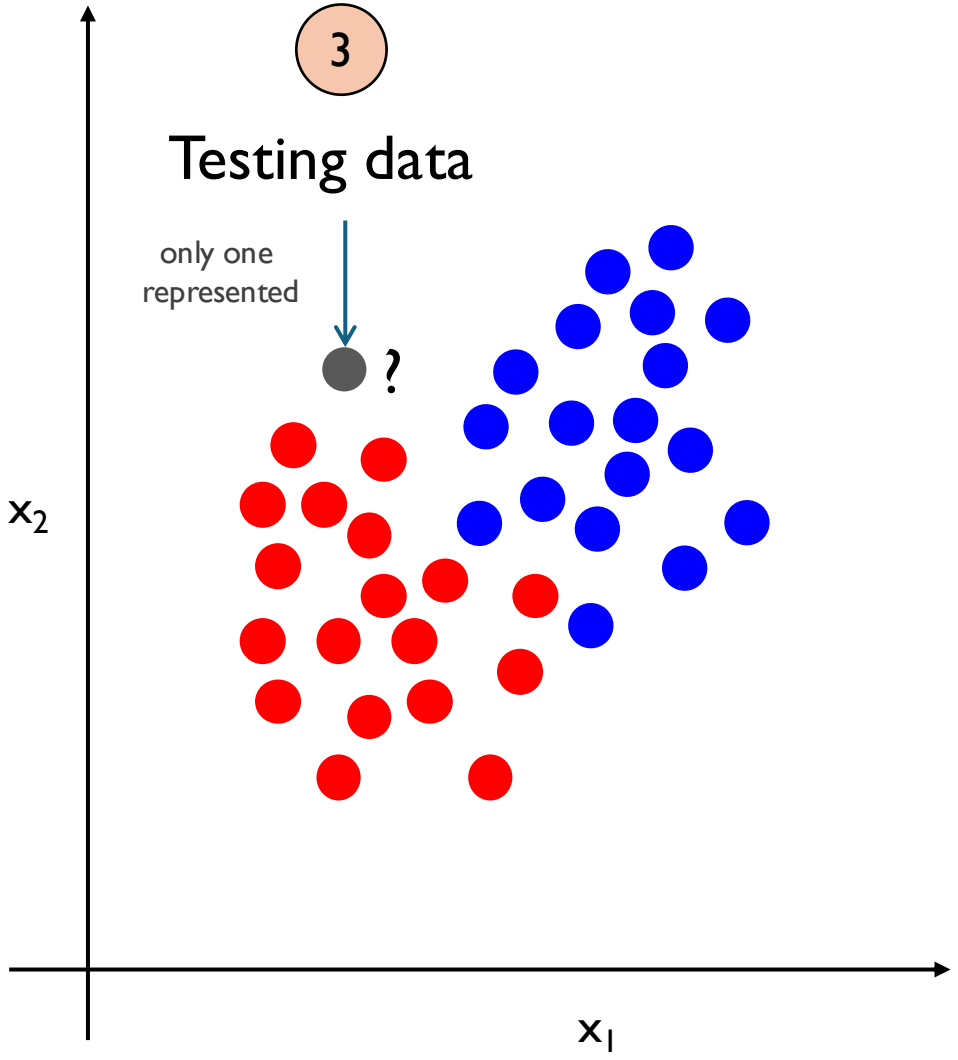
Esquema General

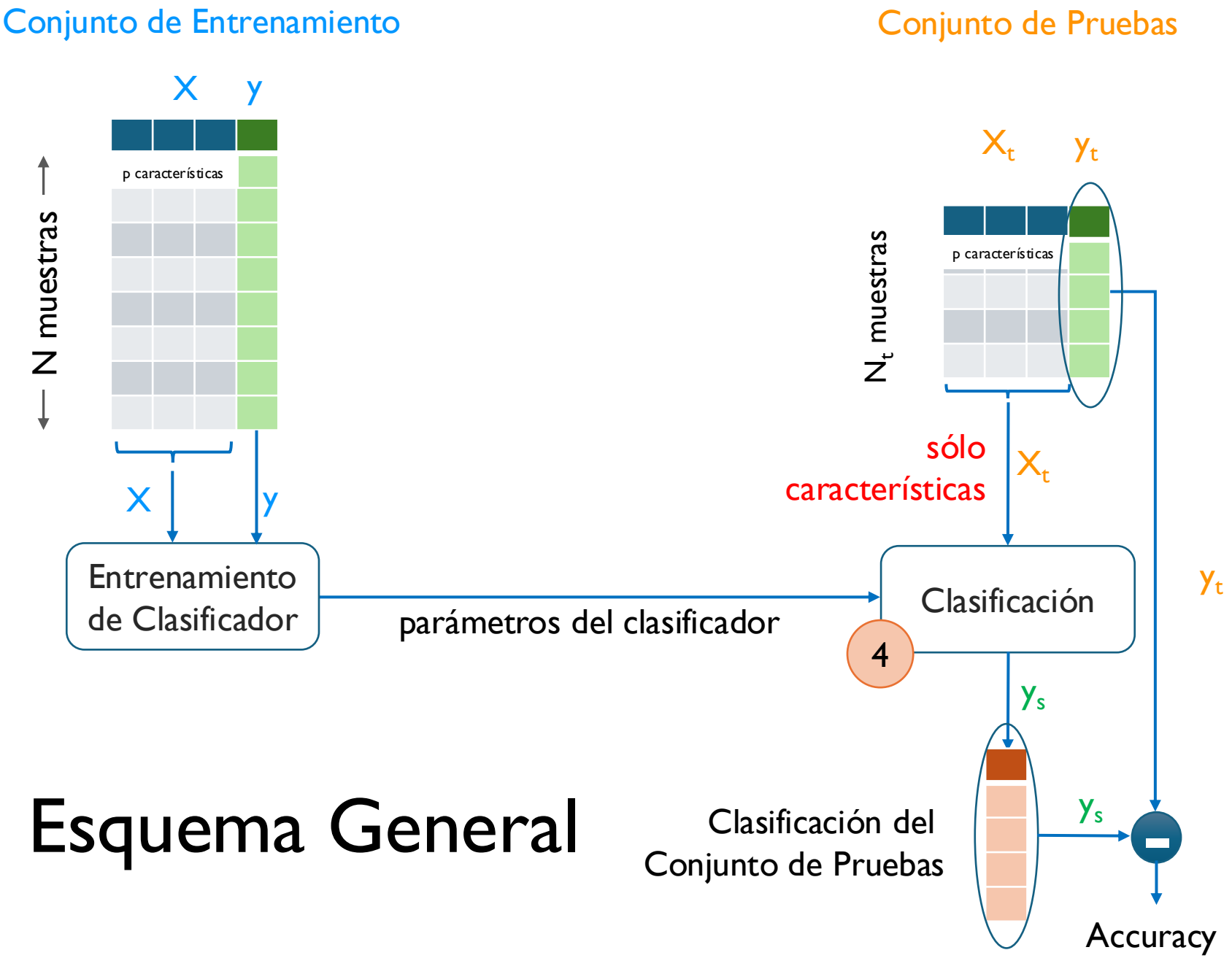

2
Classifier Training
x2
x1
Blue
Red


2
Classifier Training
x2
x1
Blue
Red

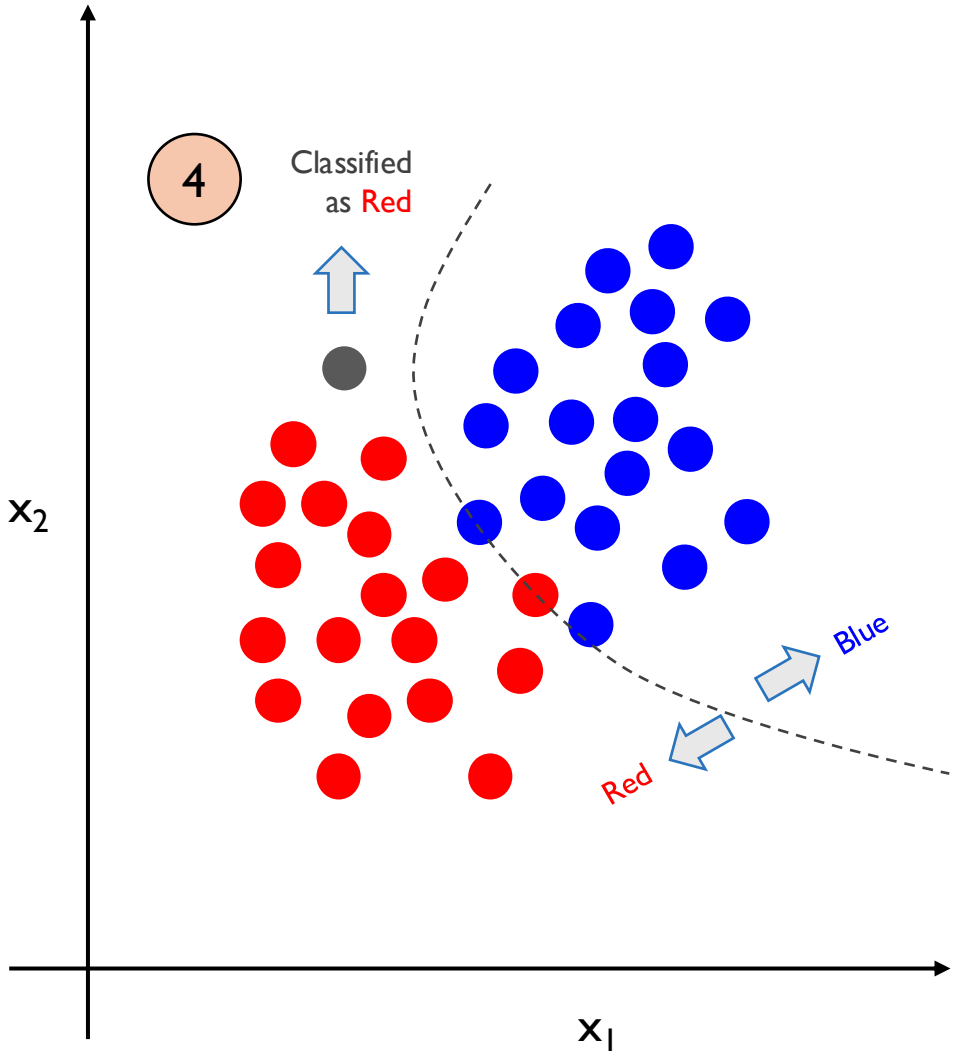


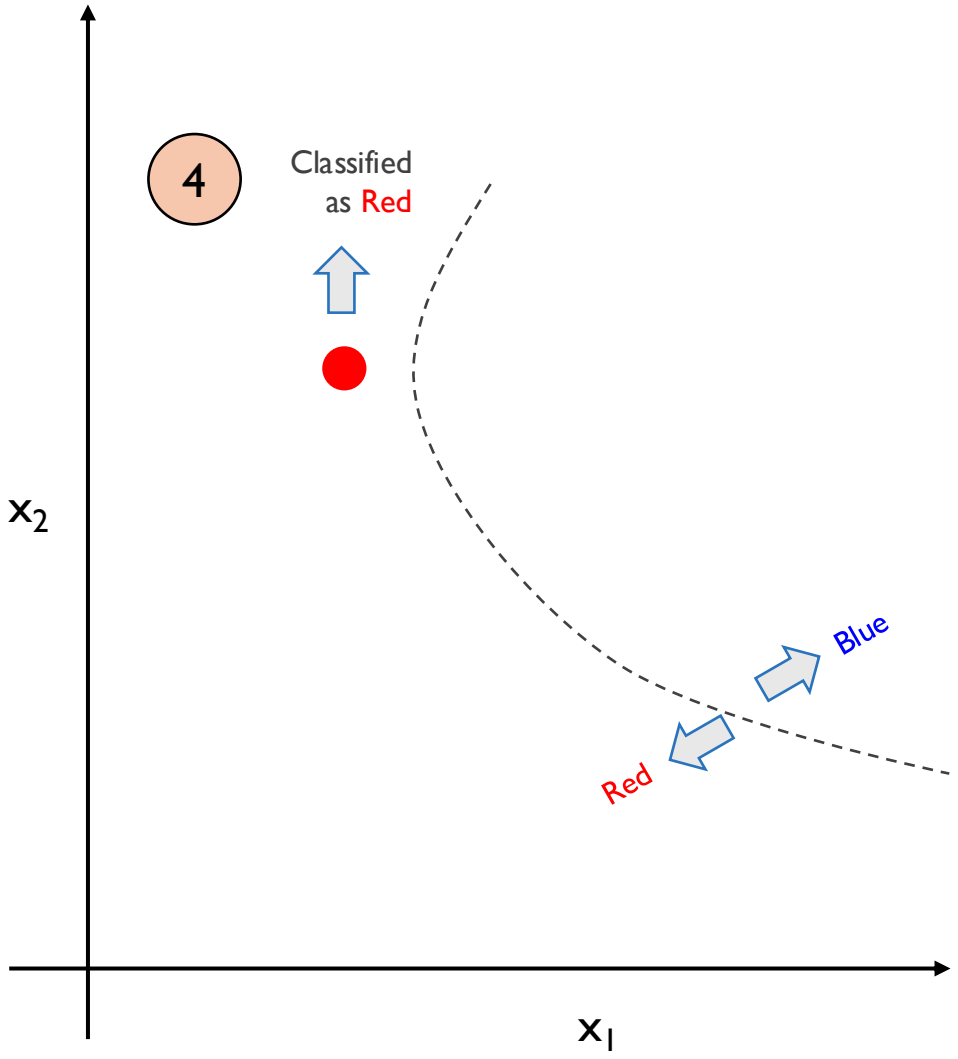
Esquema General

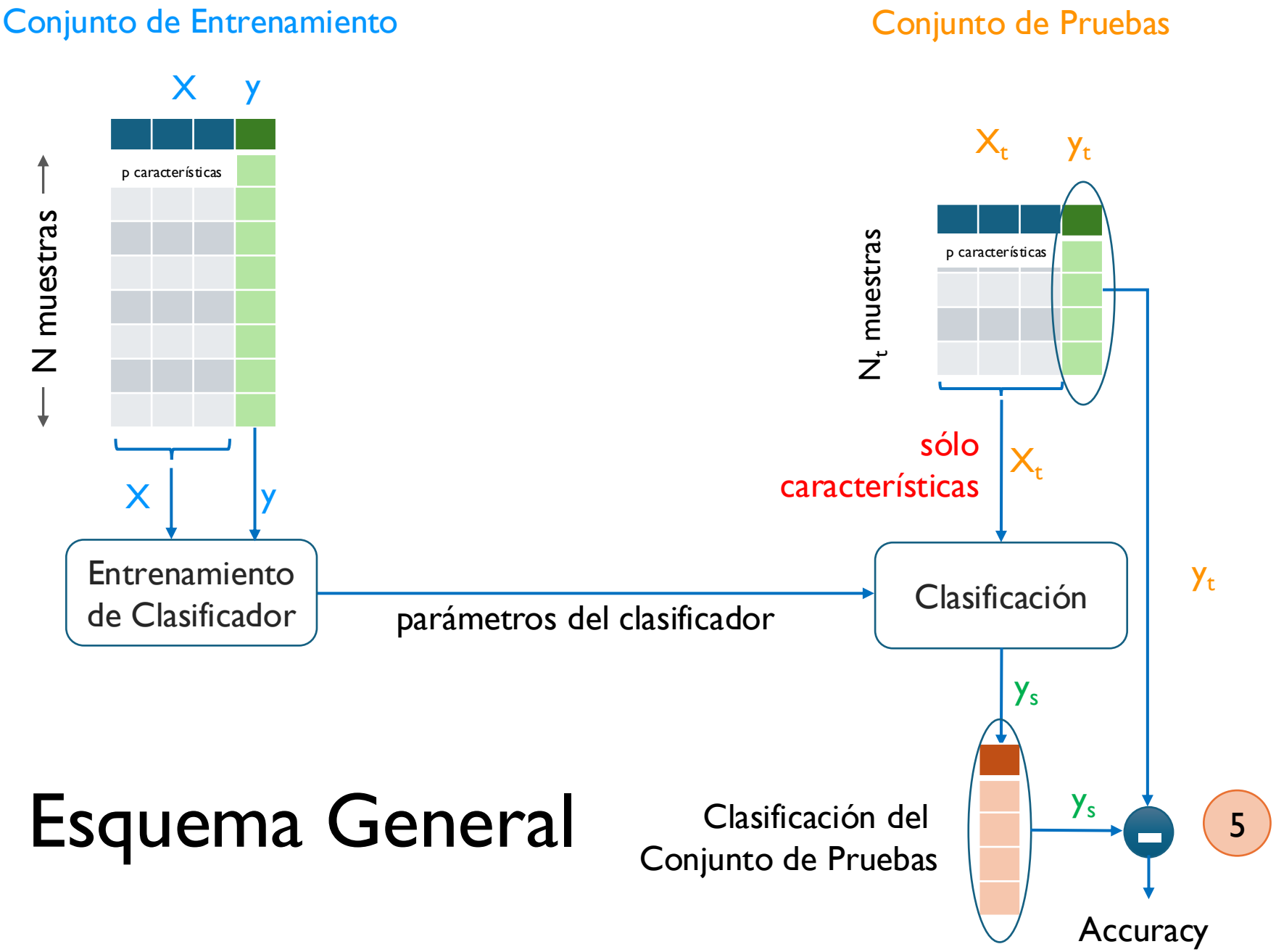

3
Testing data
only one
represented
?
x2
x1



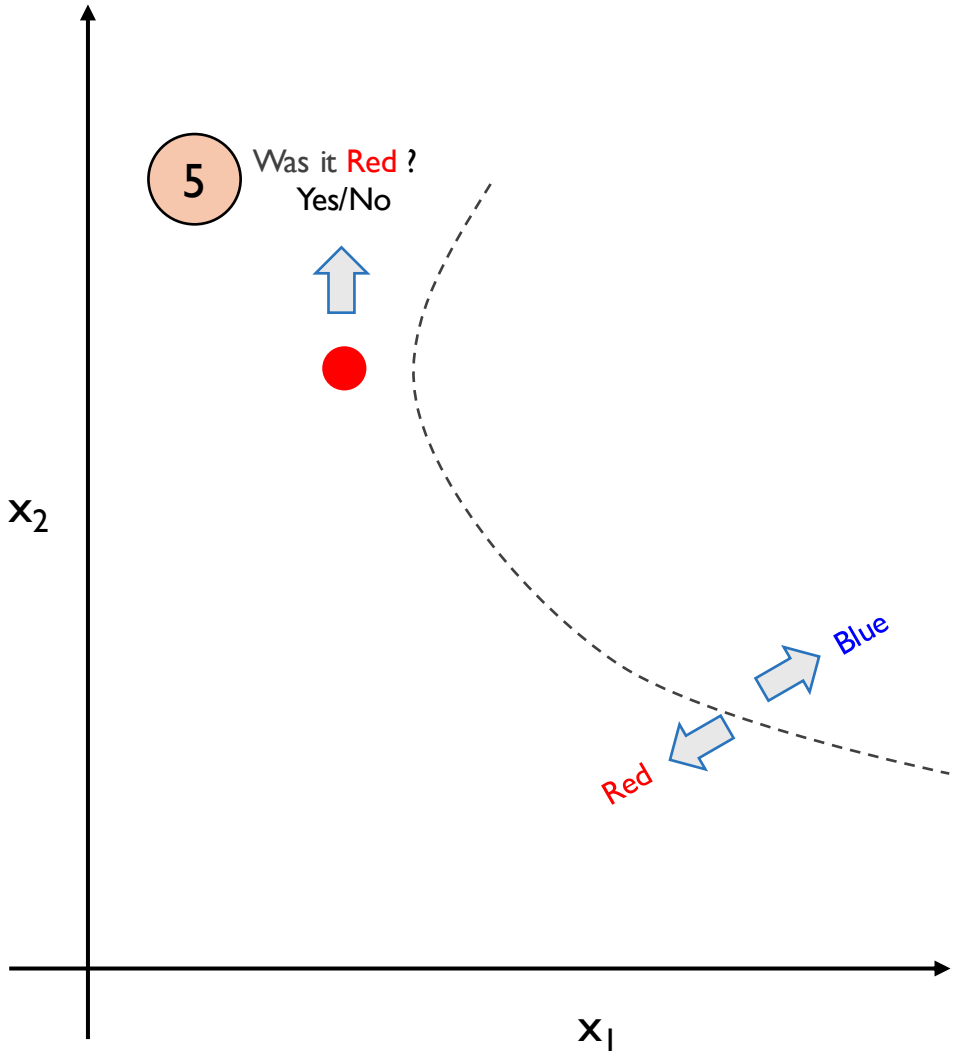
Esquema General

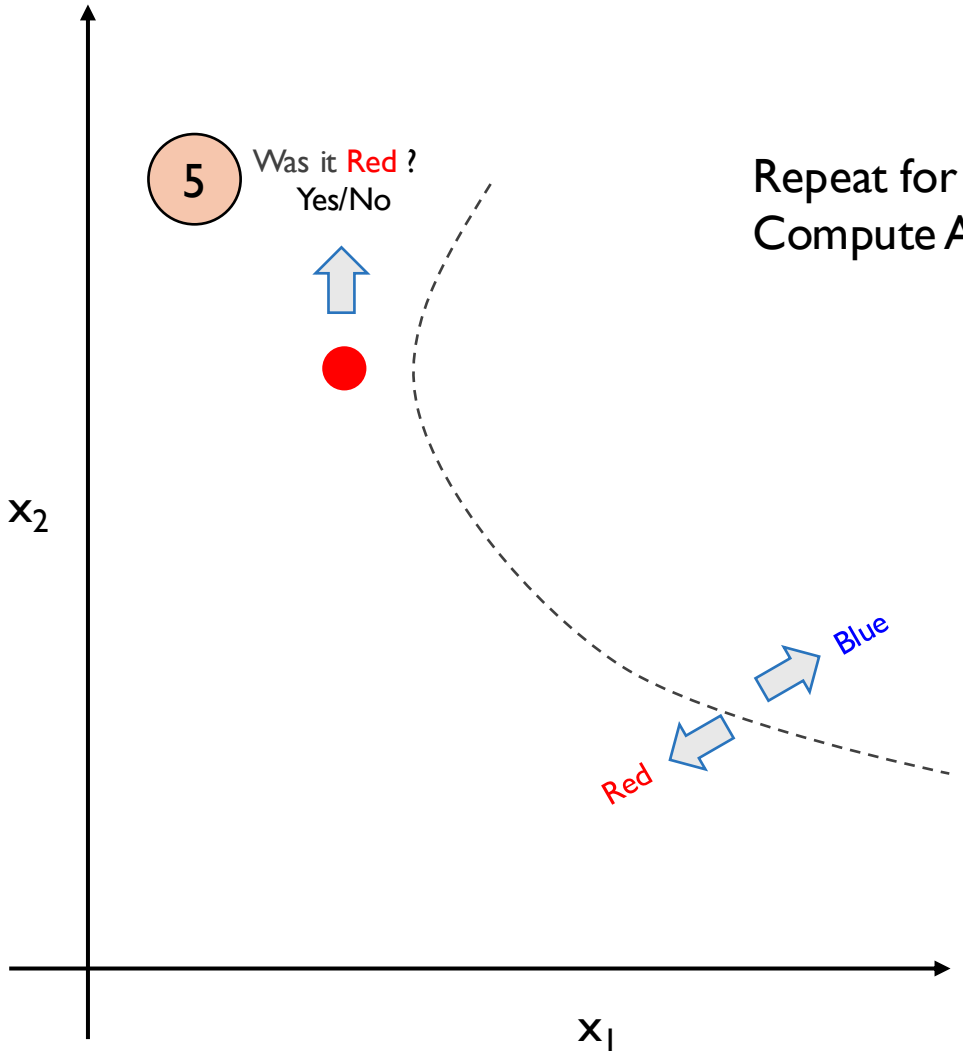

4
Classified
as Red
Blue
Red
x2
x1


4
Classified
as Red
Blue
Red
x_2
x_1

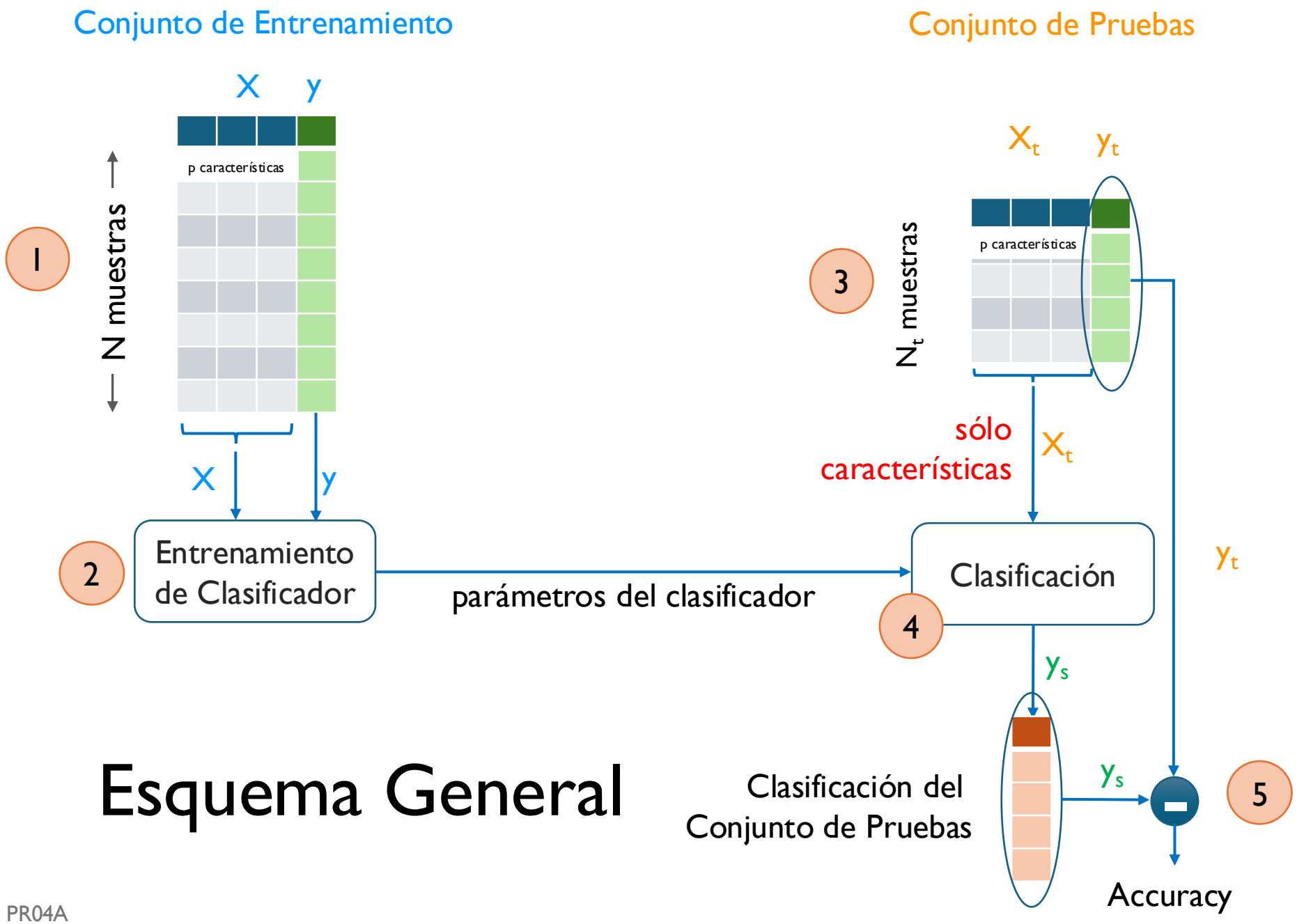


Esquema General

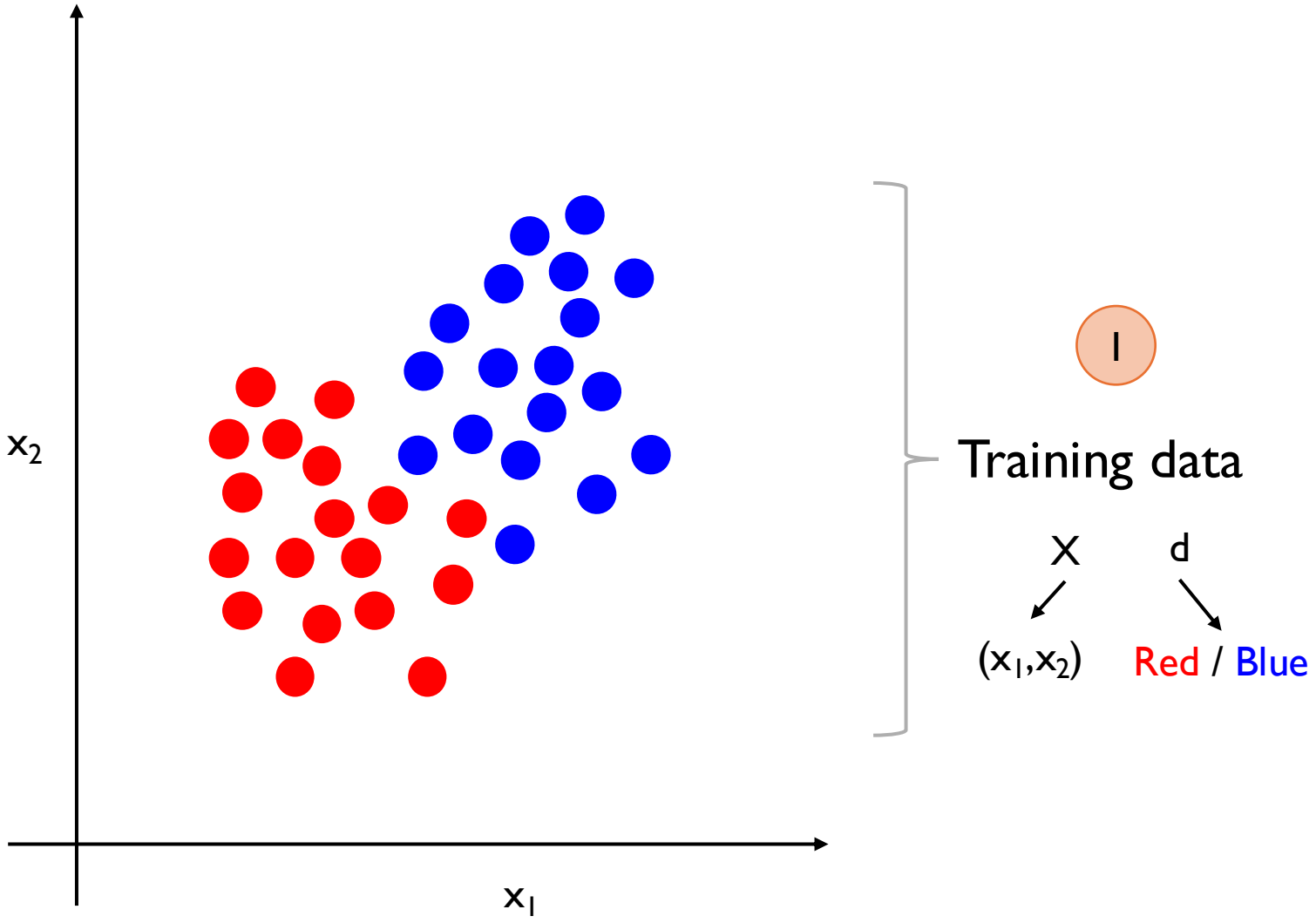

5
Was it Red ?
Yes/No
Blue
Red
x_2
x_1

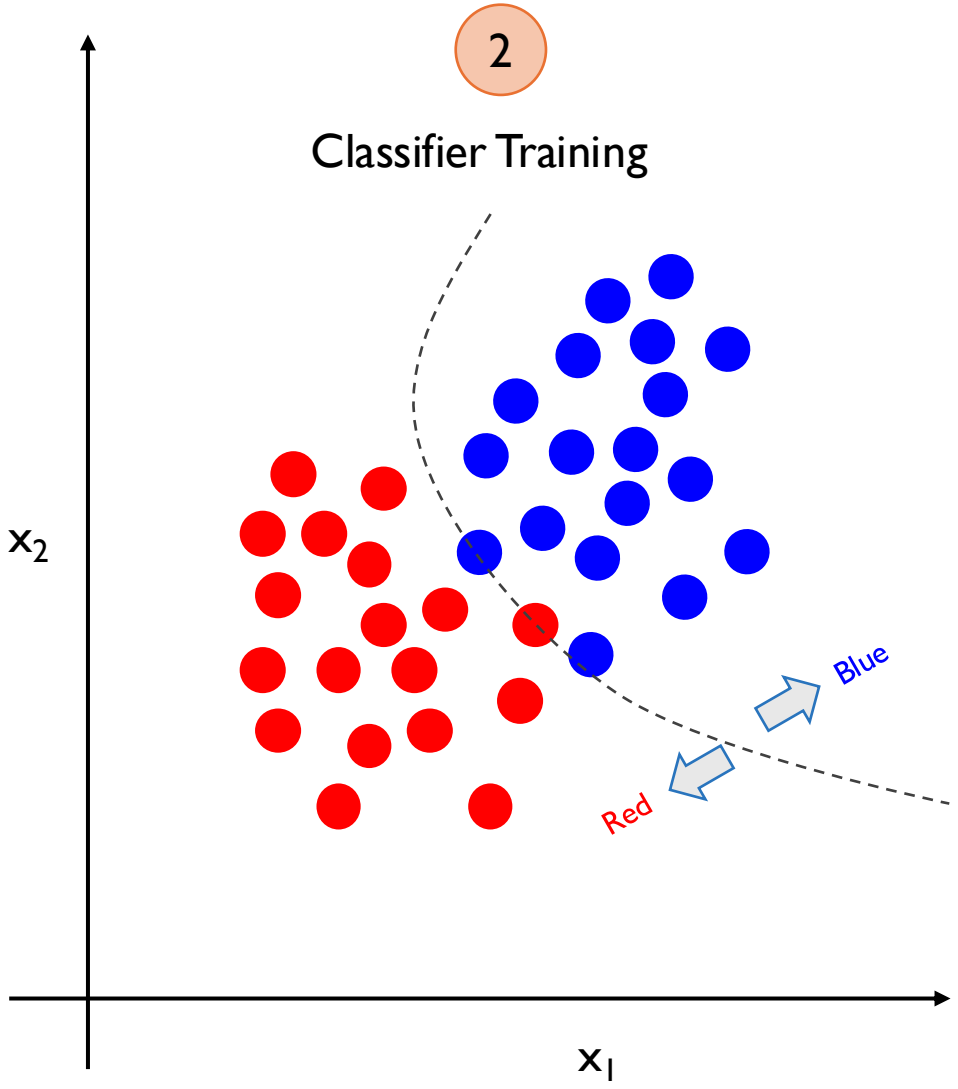


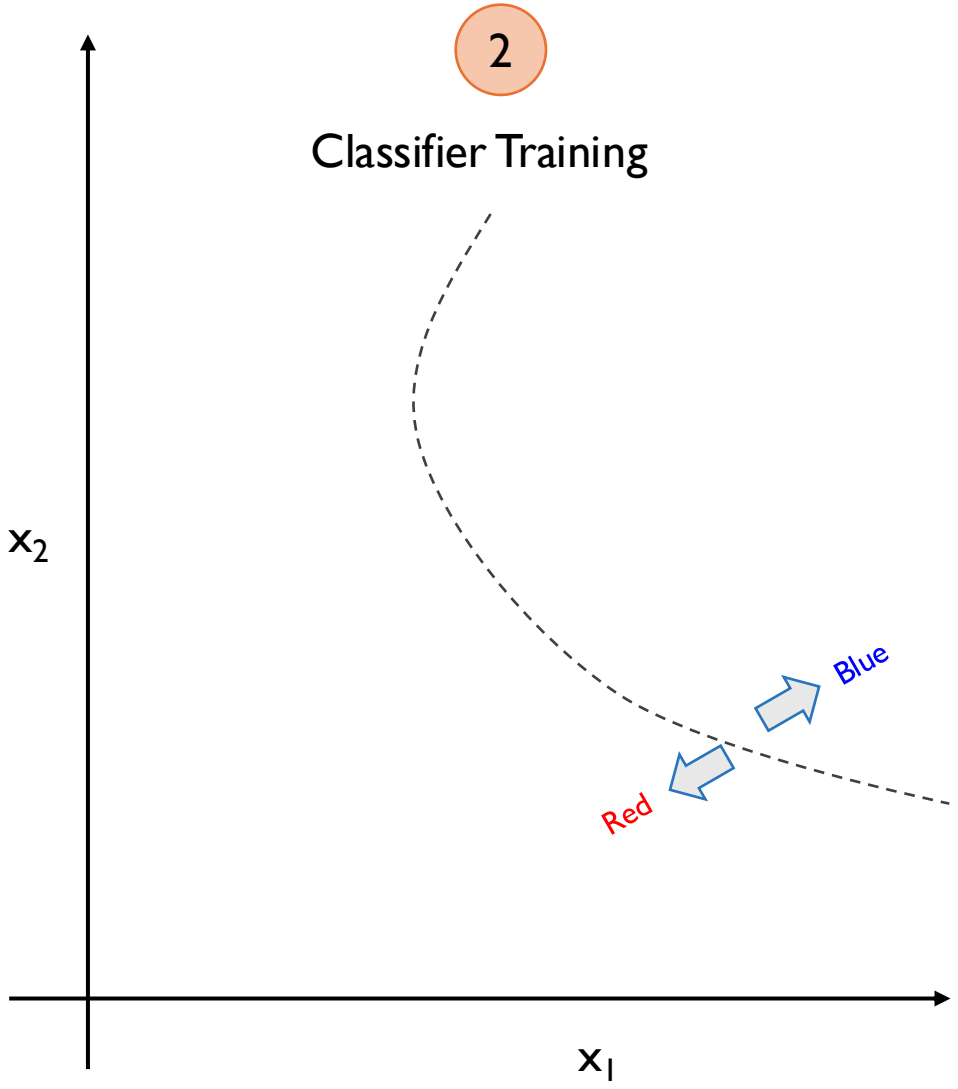
Repeat for all testing samples…
Compute Accuracy

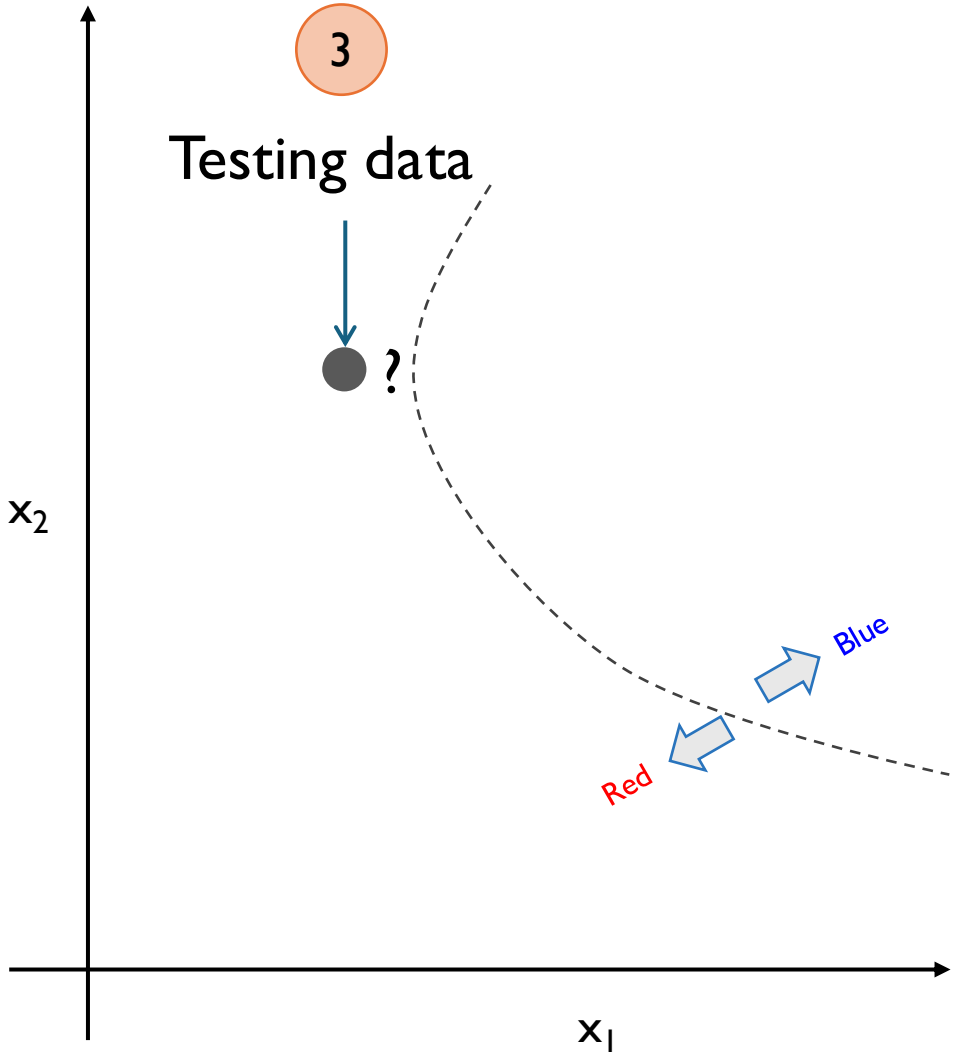


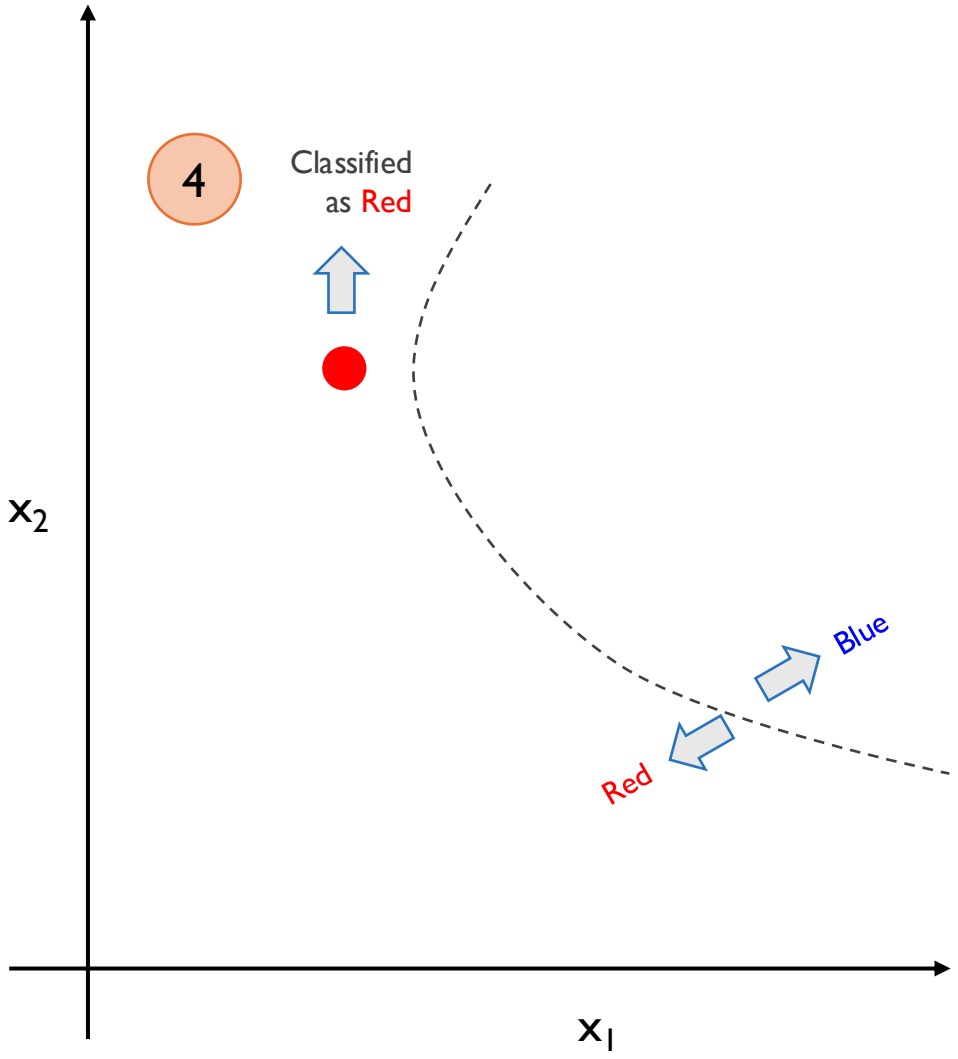
Esquema General

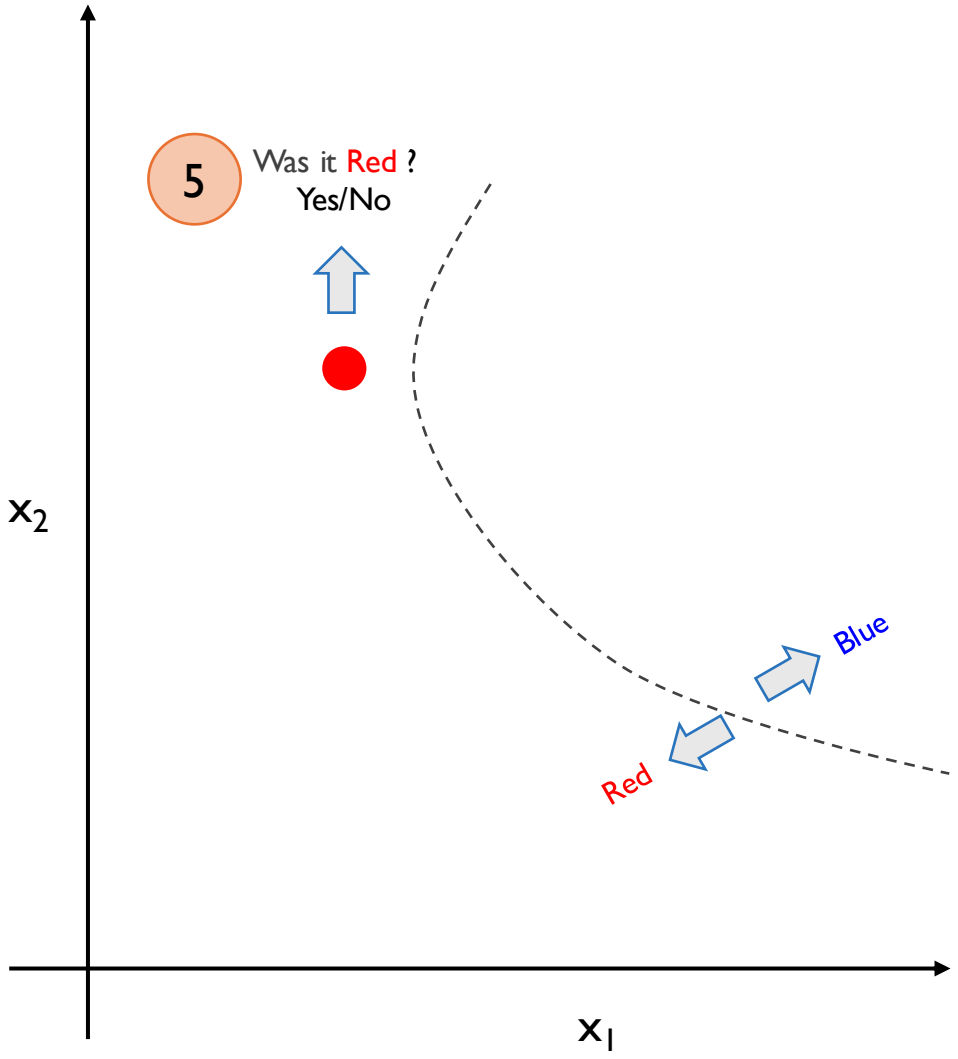

x2
x1
I
Training data
X
d
(x1,x2)
Red / Blue

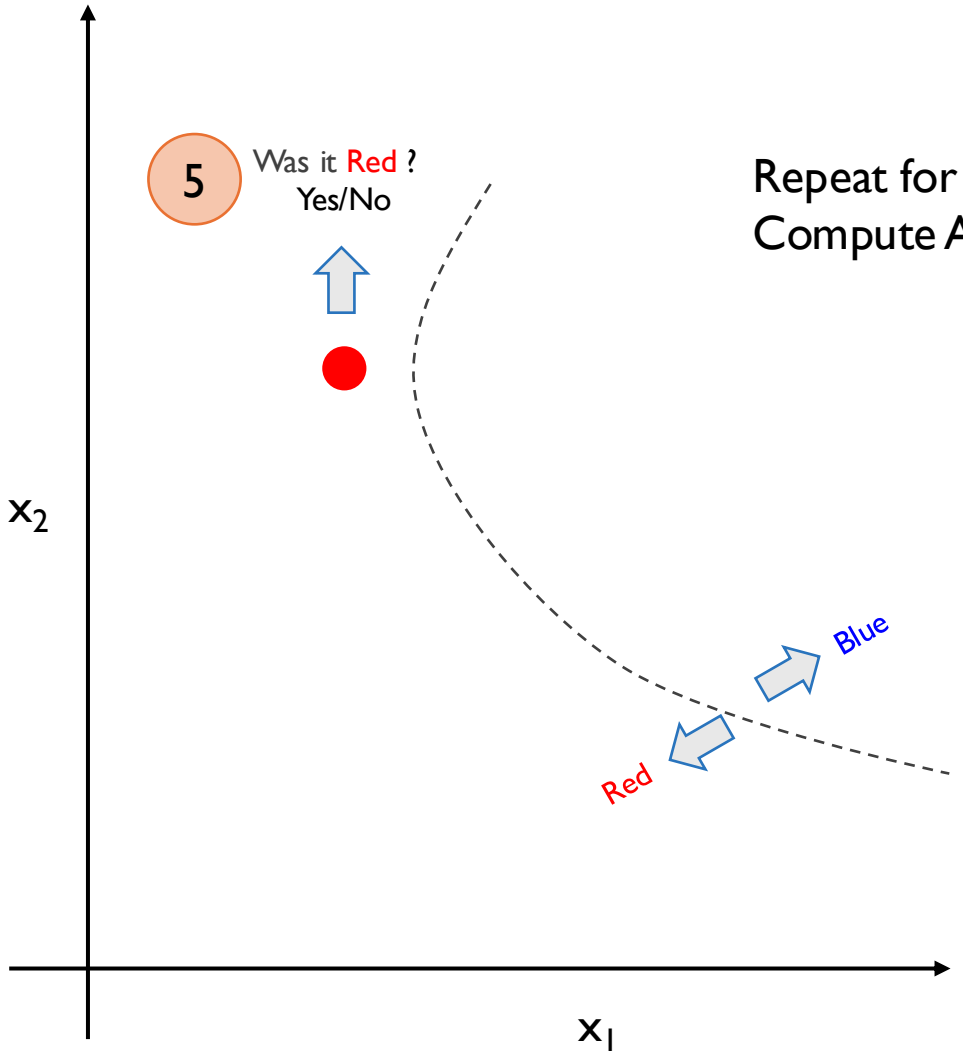

2
Classifier Training
x2
x1
Blue
Red


2
Classifier Training
x2
x1
Blue
Red

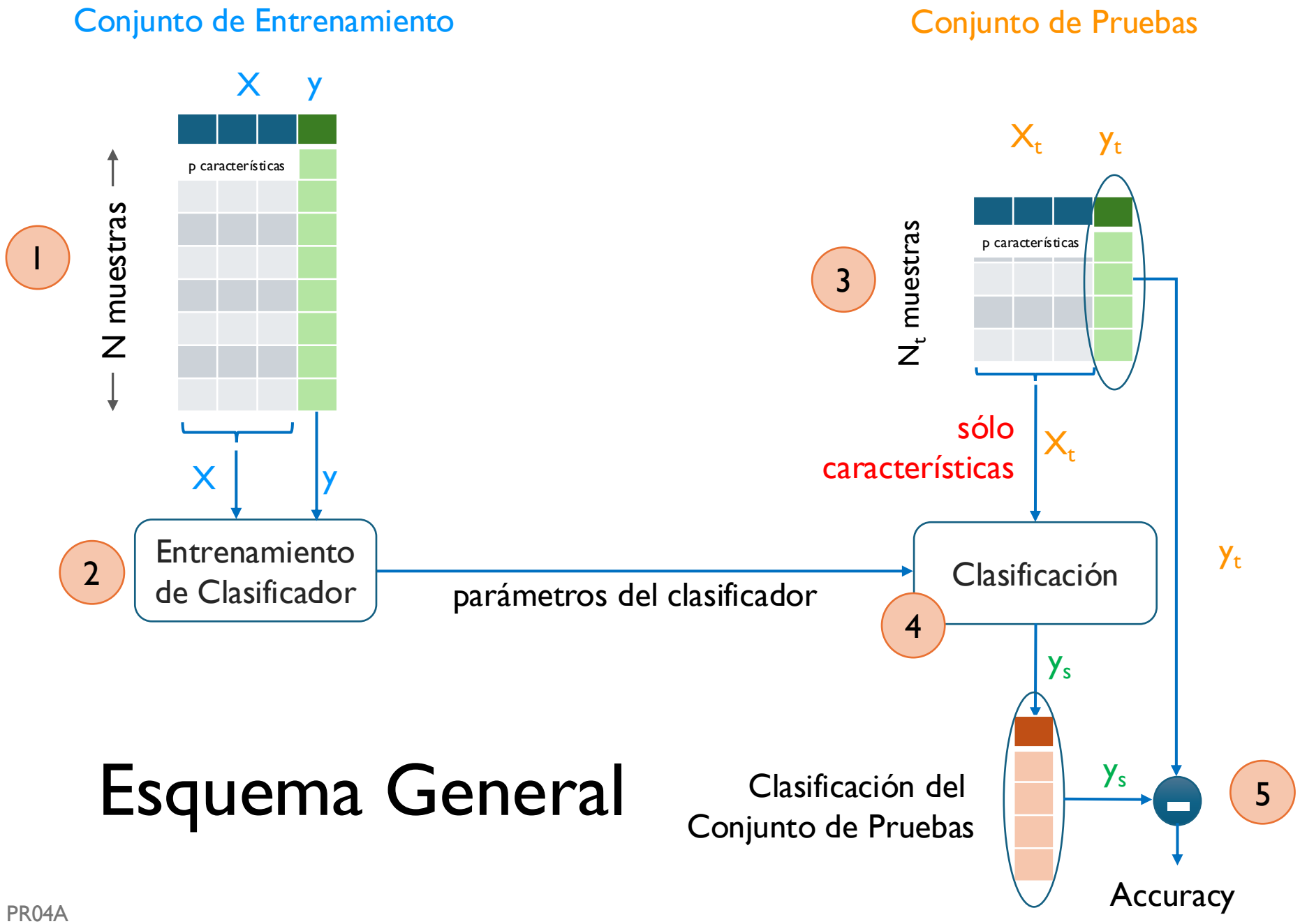

3
Testing data
?
x_2
Blue
Red
x_1


4
Classified
as Red
Blue
Red
x_2
x_1


5
Was it Red ?
Yes/No
Blue
Red
x_2
x_1

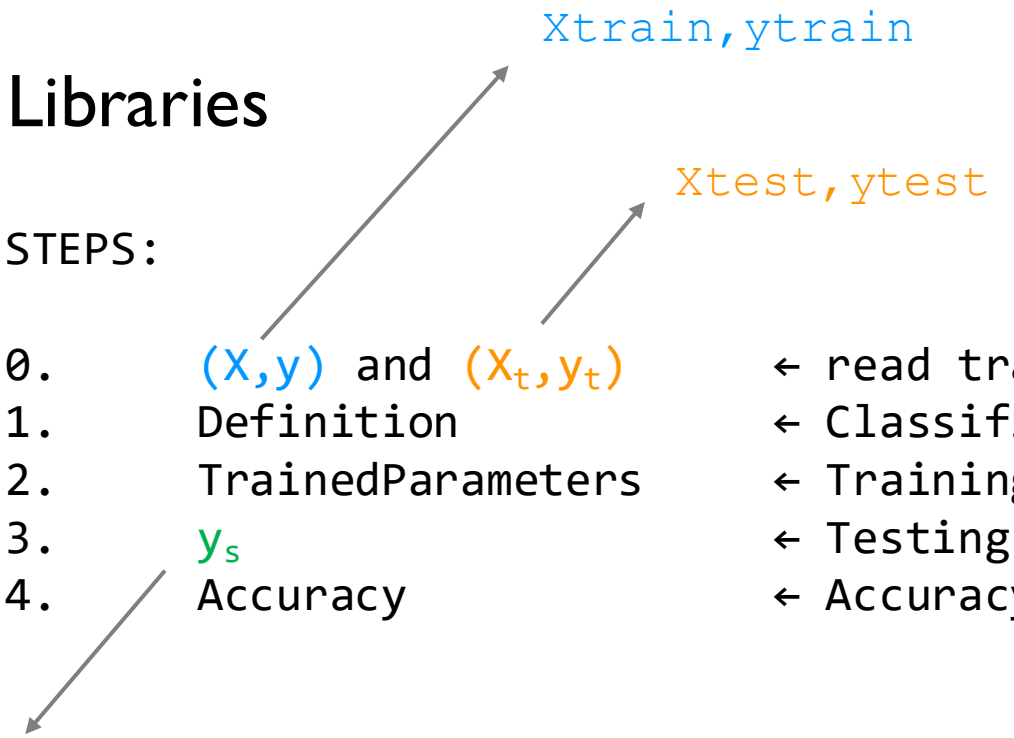


Repeat for all testing samples…
Compute Accuracy



Esquema General

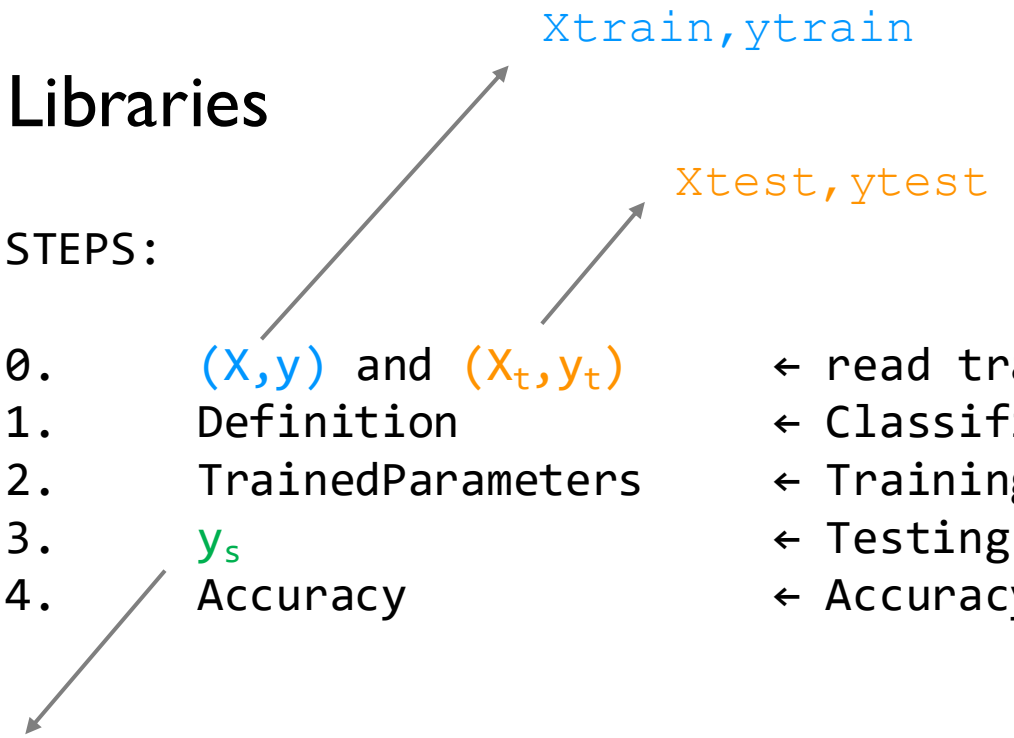
Libraries



Example for KNN-3

```
1. clf  = KNeighborsClassifier(n_neighbors=3)
2. clf.fit(Xtrain, ytrain)
3. ypred = clf.predict(Xtest)
4. acc   = accuracy_score(ypred,ytest)
```

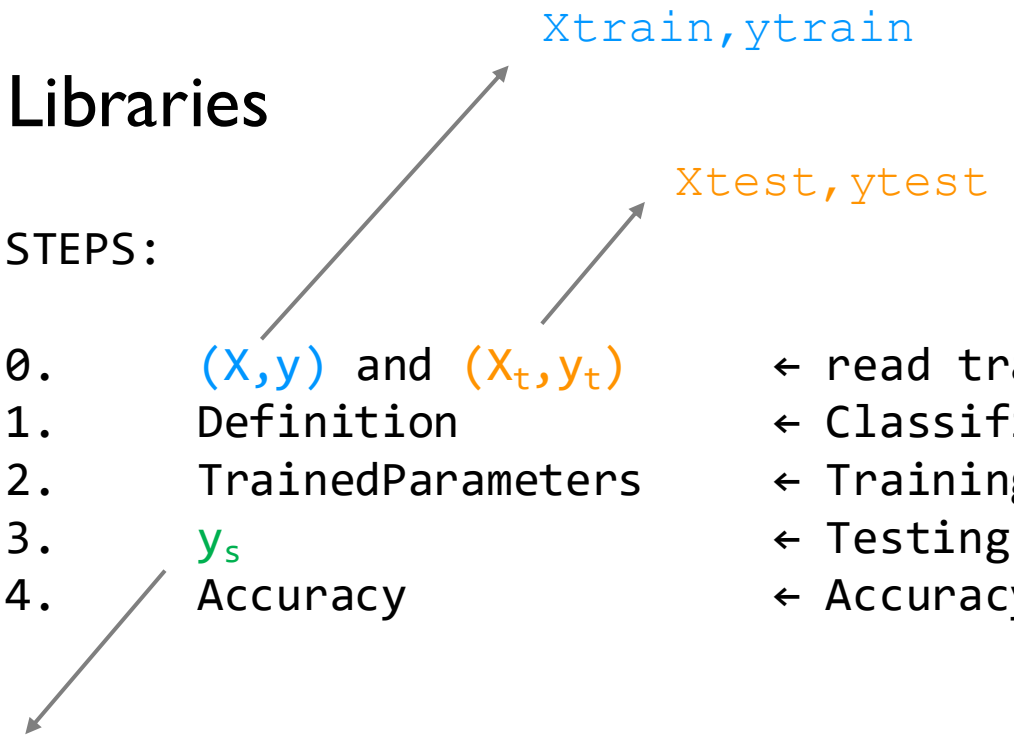
Libraries



Example for SVM

```
1. clf = SVC(parameters here)
2. clf.fit(Xtrain, ytrain)
3. ypred = clf.predict(Xtest)
4. acc   = accuracy_score(ypred,ytest)
```

Libraries



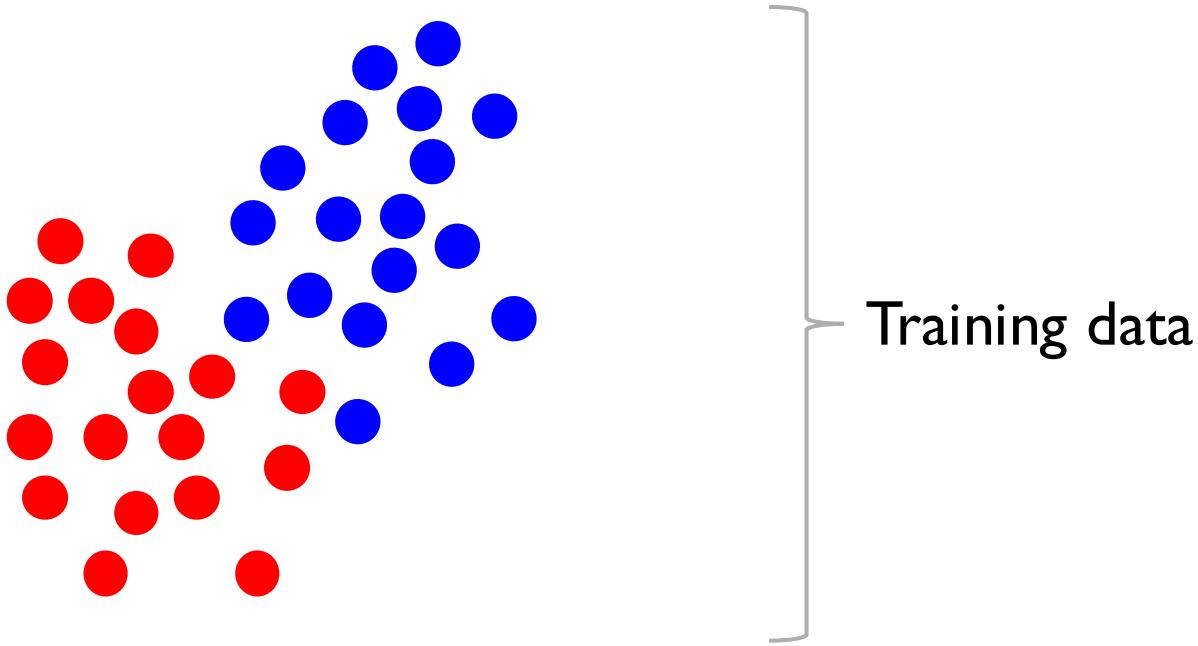
Example for Neural Networks

```
1. clf = MLPClassifier(parameters here)
2. clf.fit(Xtrain, ytrain)
3. ypred = clf.predict(Xtest)
4. acc   = accuracy_score(ypred,ytest)
```

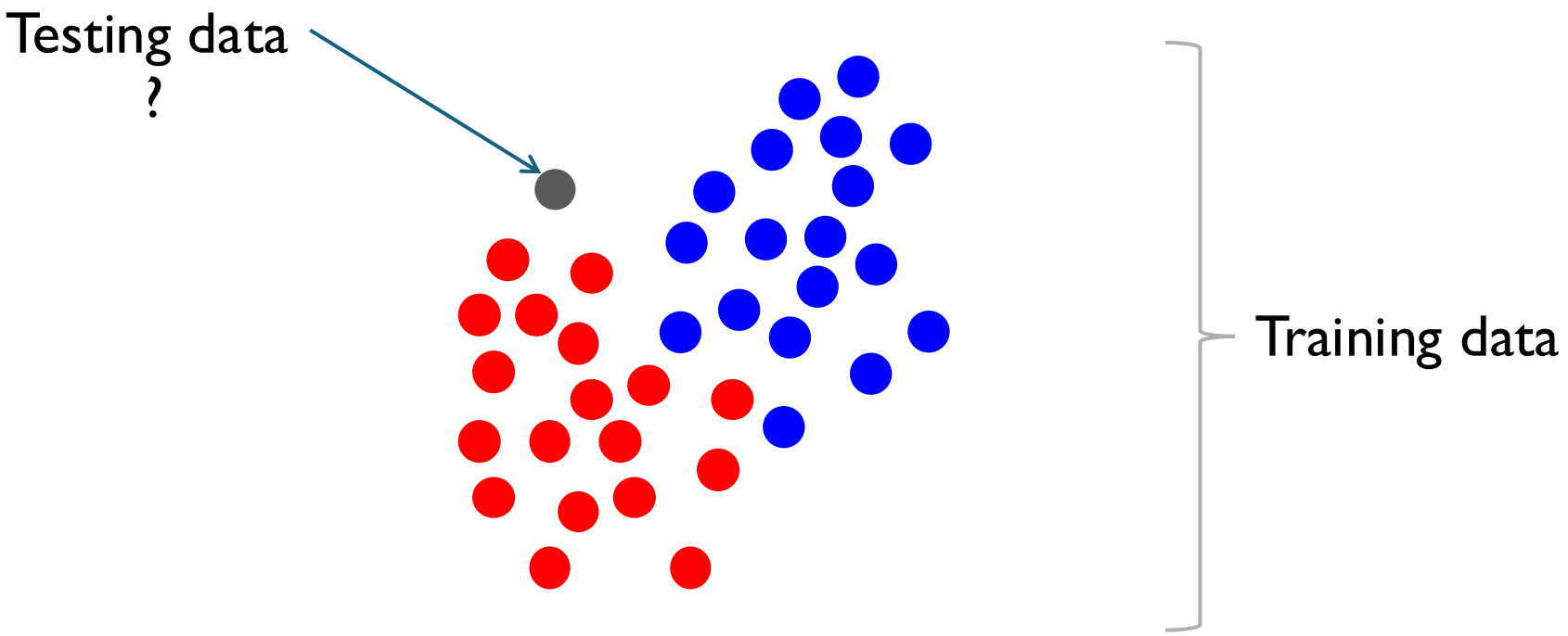
KNN

KNN: k nearest neighbors

KNN: k nearest neighbors (two classes)

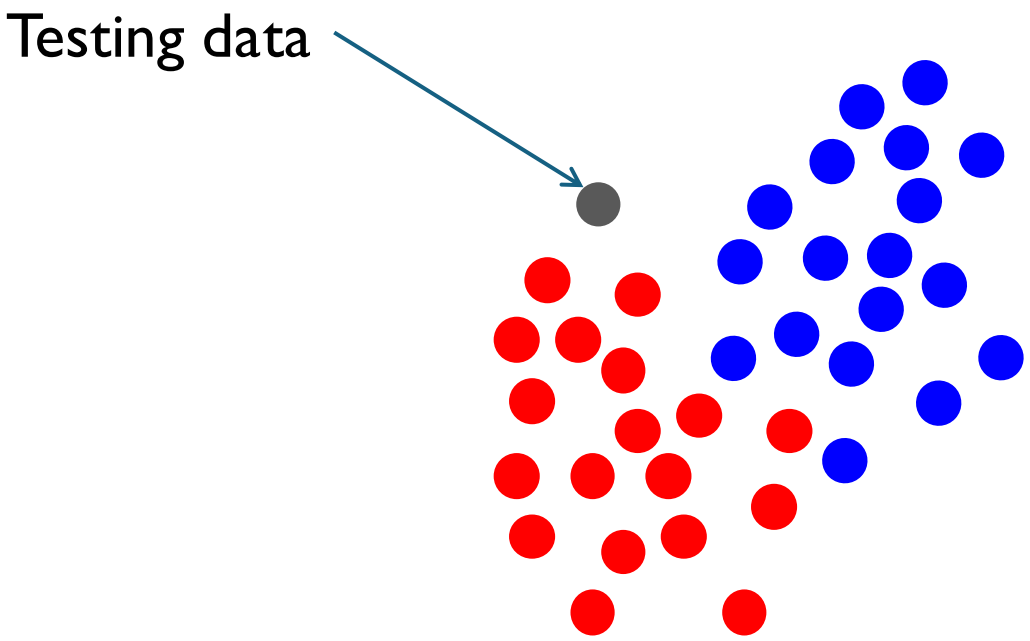


KNN: k nearest neighbors (two classes)

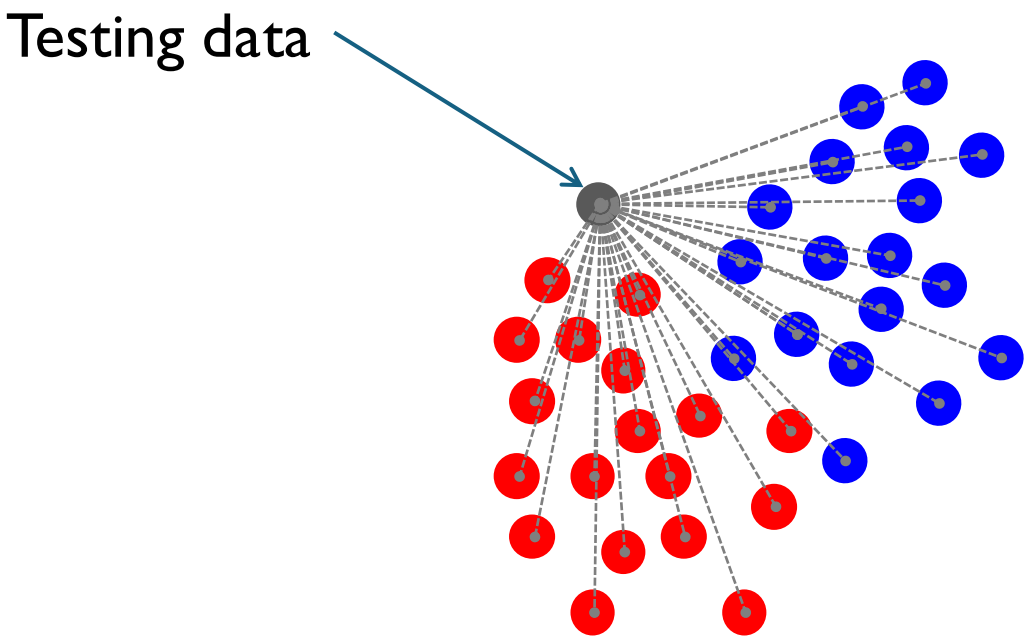


KNN: k nearest neighbors (two classes)

KNN Algorithm



KNN: k nearest neighbors (two classes)

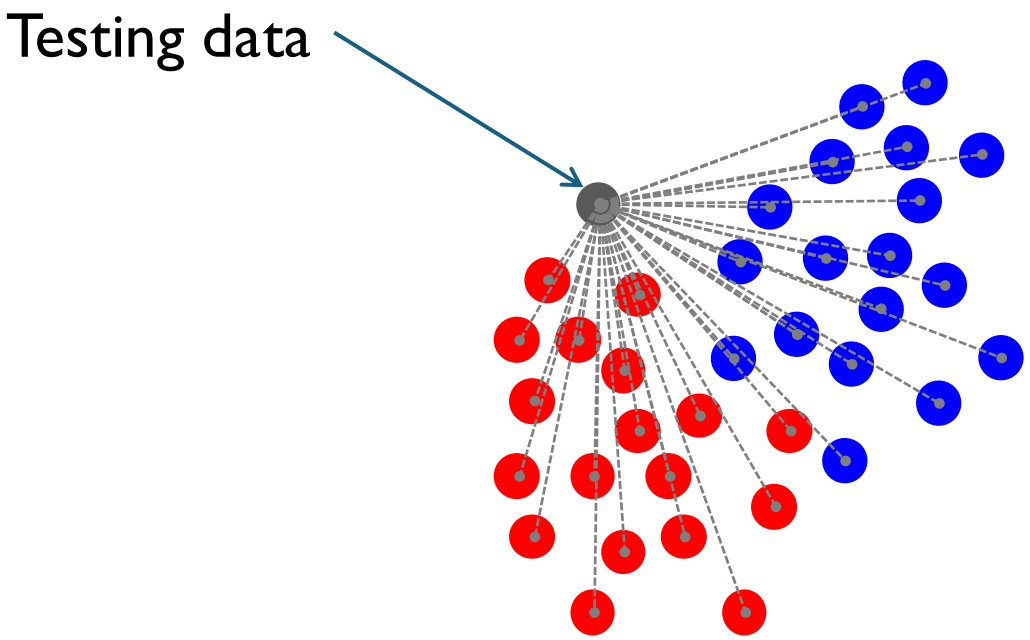


KNN Algorithm

1. Distances

0.3655
0.7015
0.6712
0.7474
0.4313
0.2880
0.0115
0.9202
0.5304
0.9362
0.5989
0.9447
0.0569
0.3264
0.6811
0.1332
0.0226
0.2435
0.0705
0.4839
0.3631
0.1090
0.6296
0.0508
0.7660
0.9544
0.6487
0.1519
0.7936
0.9525

KNN: k nearest neighbors (two classes)

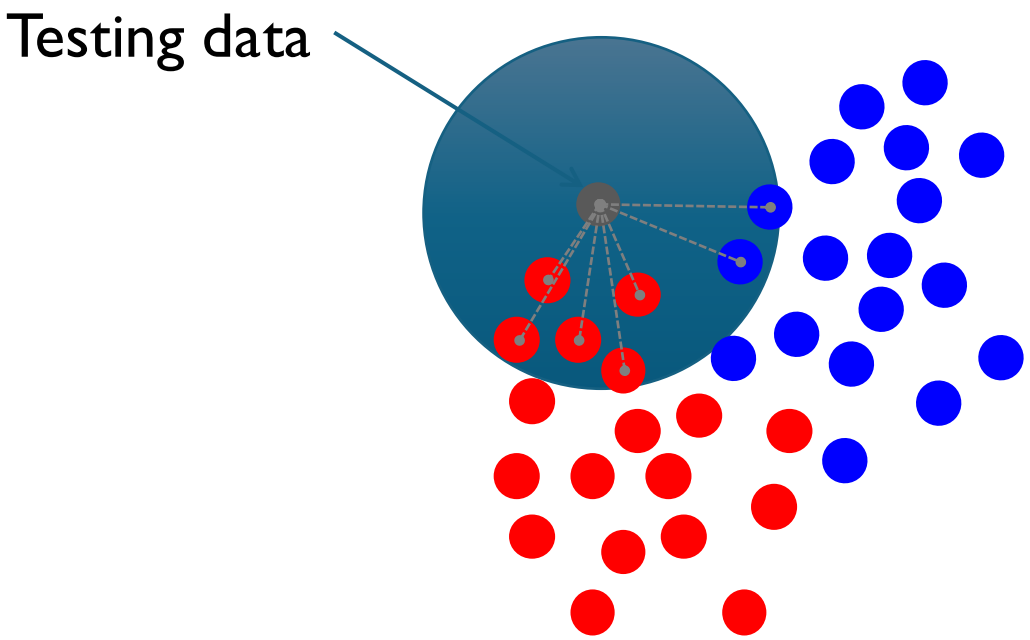


KNN Algorithm

1. Distances
2. Sort

0.3655	0.0115
0.7015	0.0226
0.6712	0.0508
0.7474	0.0569
0.4313	0.0705
0.2880	0.1090
0.0115	0.1332
0.9202	0.1519
0.5304	0.2435
0.9362	0.2880
0.5989	0.3264
0.9447	0.3631
0.0569	0.3655
0.3264	0.4313
0.6811	0.4839
0.1332	0.5304
0.0226	0.5989
0.2435	0.6296
0.0705	0.6487
0.4839	0.6712
0.3631	0.6811
0.1090	0.7015
0.6296	0.7474
0.0508	0.7660
0.7660	0.7936
0.9544	0.9202
0.6487	0.9362
0.1519	0.9447
0.7936	0.9525
0.9525	0.9544

KNN: k nearest neighbors (two classes)

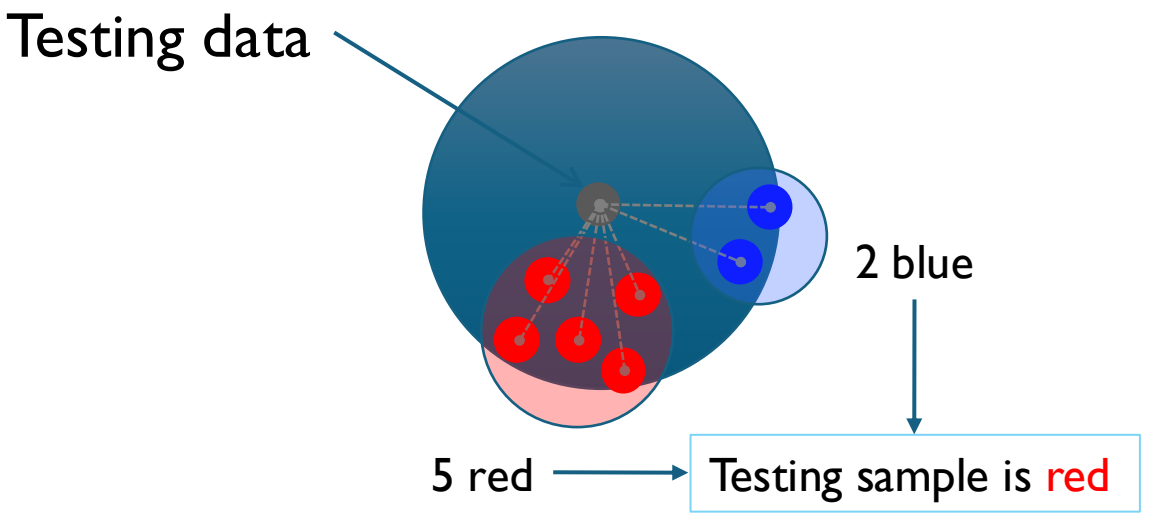


KNN Algorithm

1. Distances
2. Sort
3. Take the k nearest
   (example k=7)

0.0115
0.0226
0.0508
0.0569
0.0705
0.1090
0.1332
0.1519
0.2435
0.2880
0.3264
0.3631
0.3655
0.4313
0.4839
0.5304
0.5989
0.6296
0.6487
0.6712
0.6811
0.7015
0.7474
0.7660
0.7936
0.9202
0.9362
0.9447
0.9525
0.9544

KNN: k nearest neighbors (two classes)

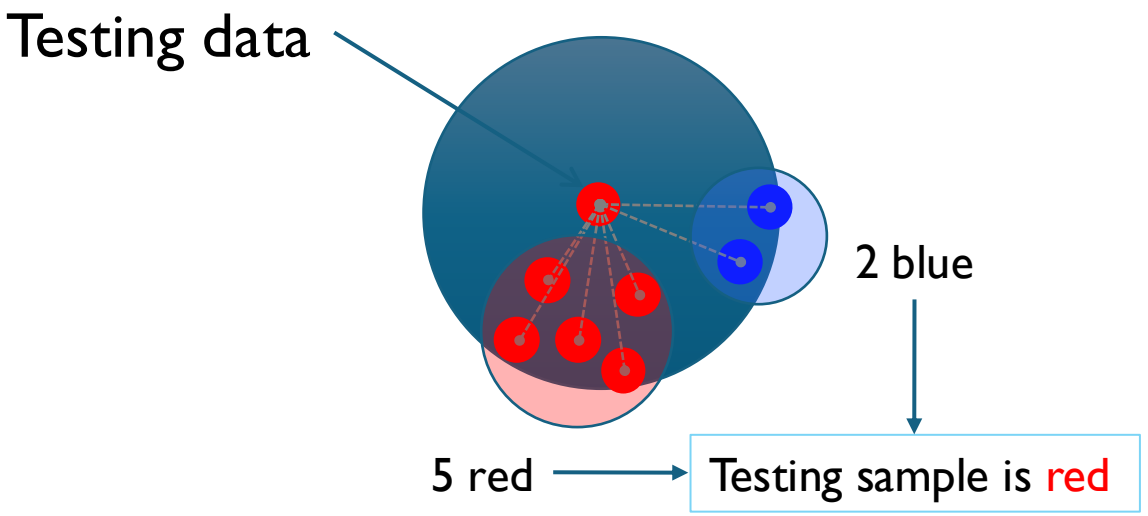


KNN Algorithm

1. Distances
2. Sort
3. Take the k nearest
   (example k=7)
4. Majority vote

0.0115
0.0226
0.0508
0.0569
0.0705
0.1090
0.1332
0.1519
0.2435
0.2880
0.3264
0.3631
0.3655
0.4313
0.4839
0.5304
0.5989
0.6296
0.6487
0.6712
0.6811
0.7015
0.7474
0.7660
0.7936
0.9202
0.9362
0.9447
0.9525
0.9544

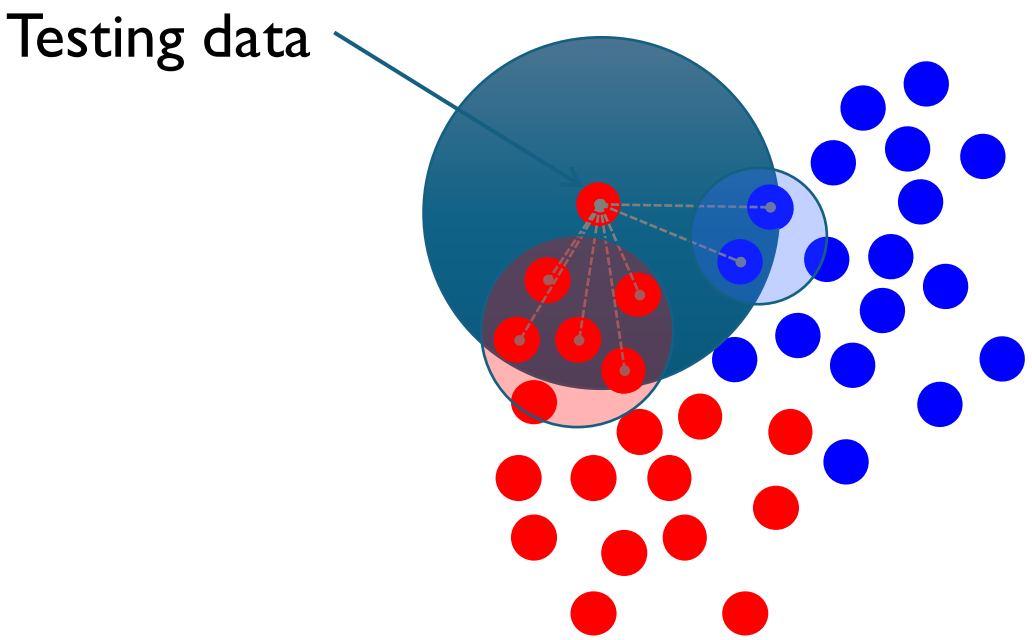
KNN: k nearest neighbors (two classes)



KNN Algorithm

1. Distances
2. Sort
3. Take the k nearest
   (example k=7)
4. Majority vote

KNN: k nearest neighbors (two classes)



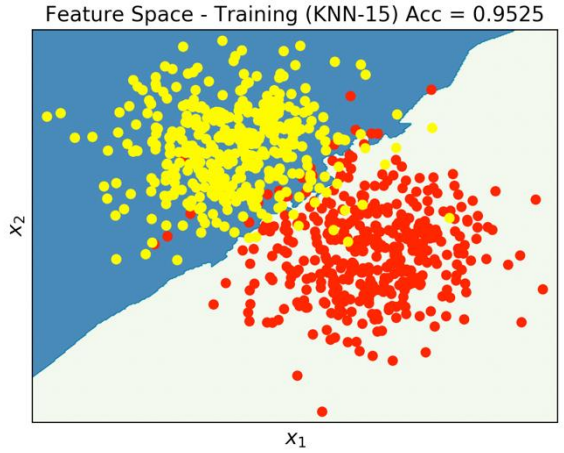
KNN Algorithm

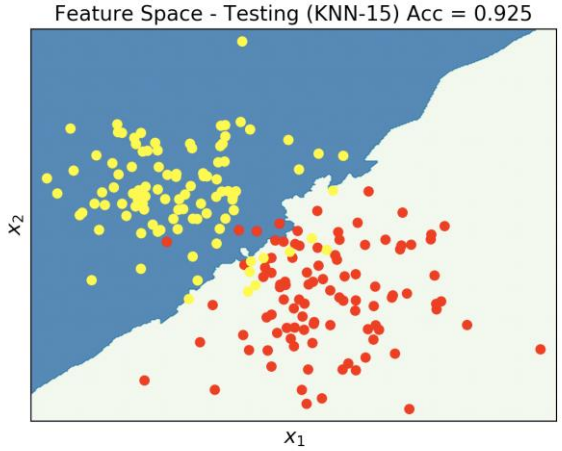
1. Distances
2. Sort
3. Take the k nearest
   (example k=7)
4. Majority vote

TRAINING

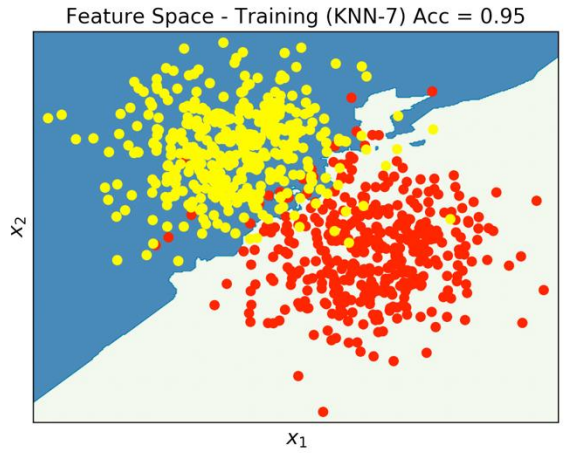
TESTING

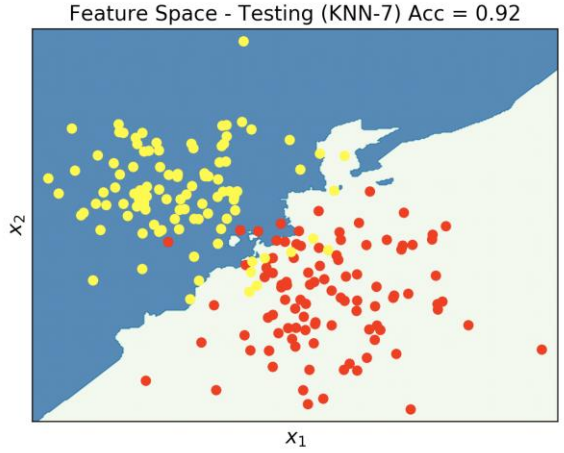
k = 15





k = 7

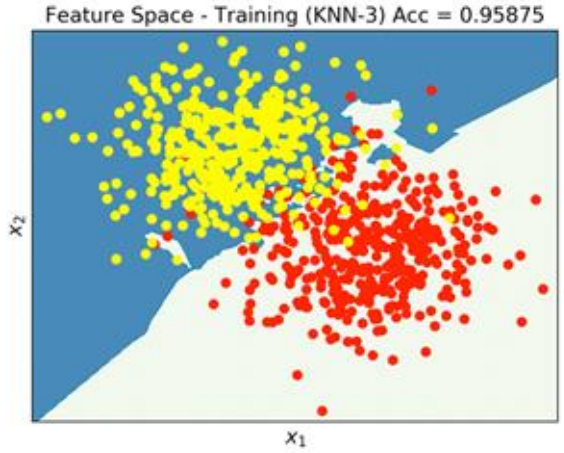


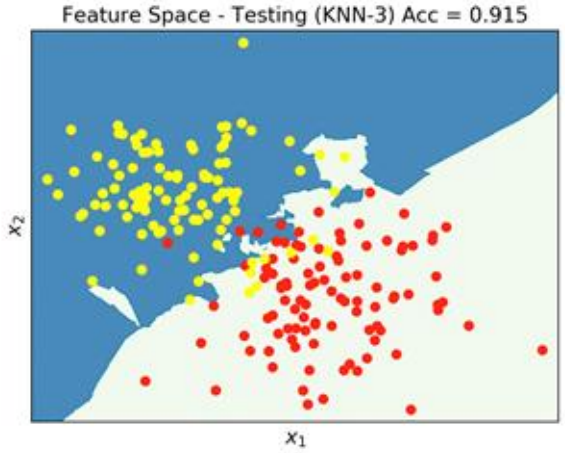


TRAINING

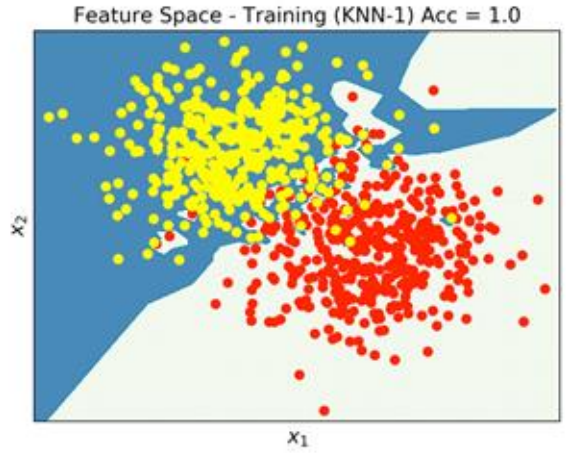
TESTING

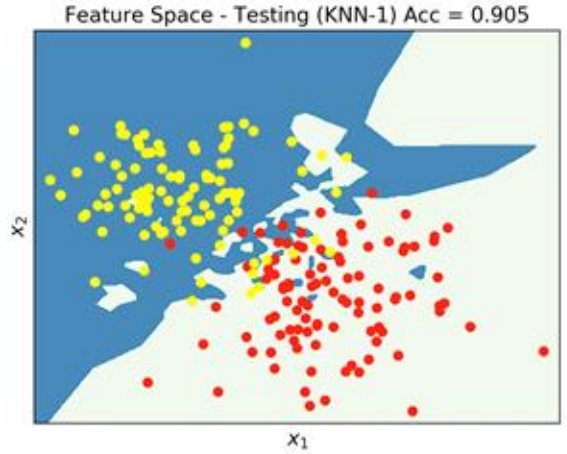
k = 3





k = 1

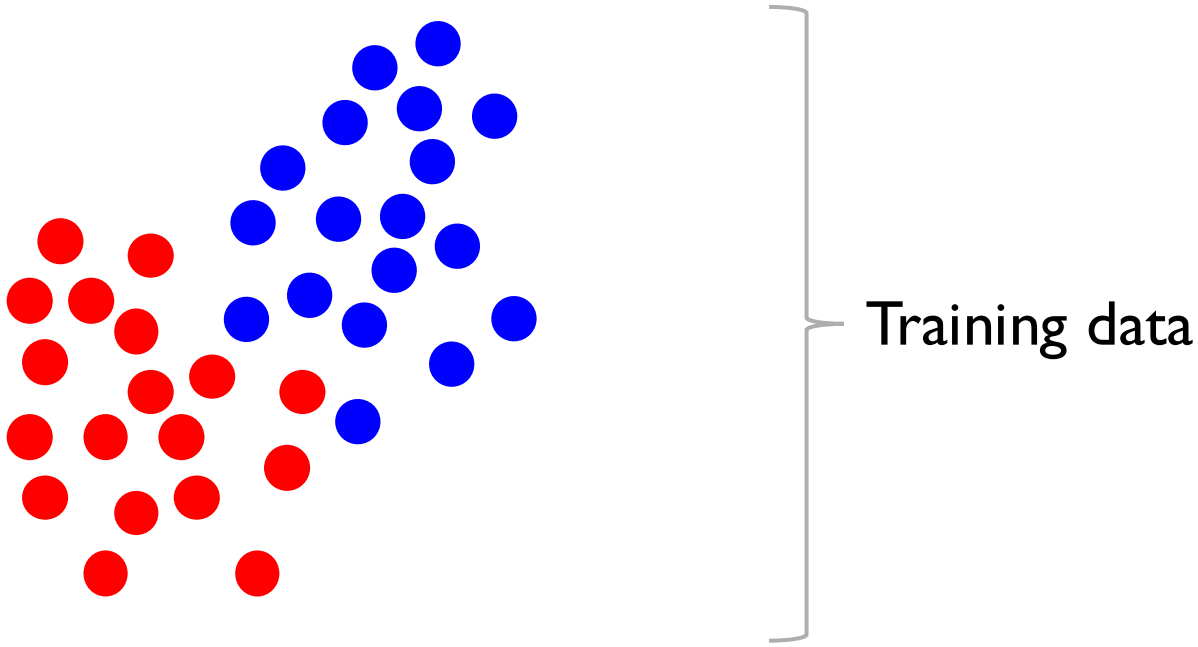




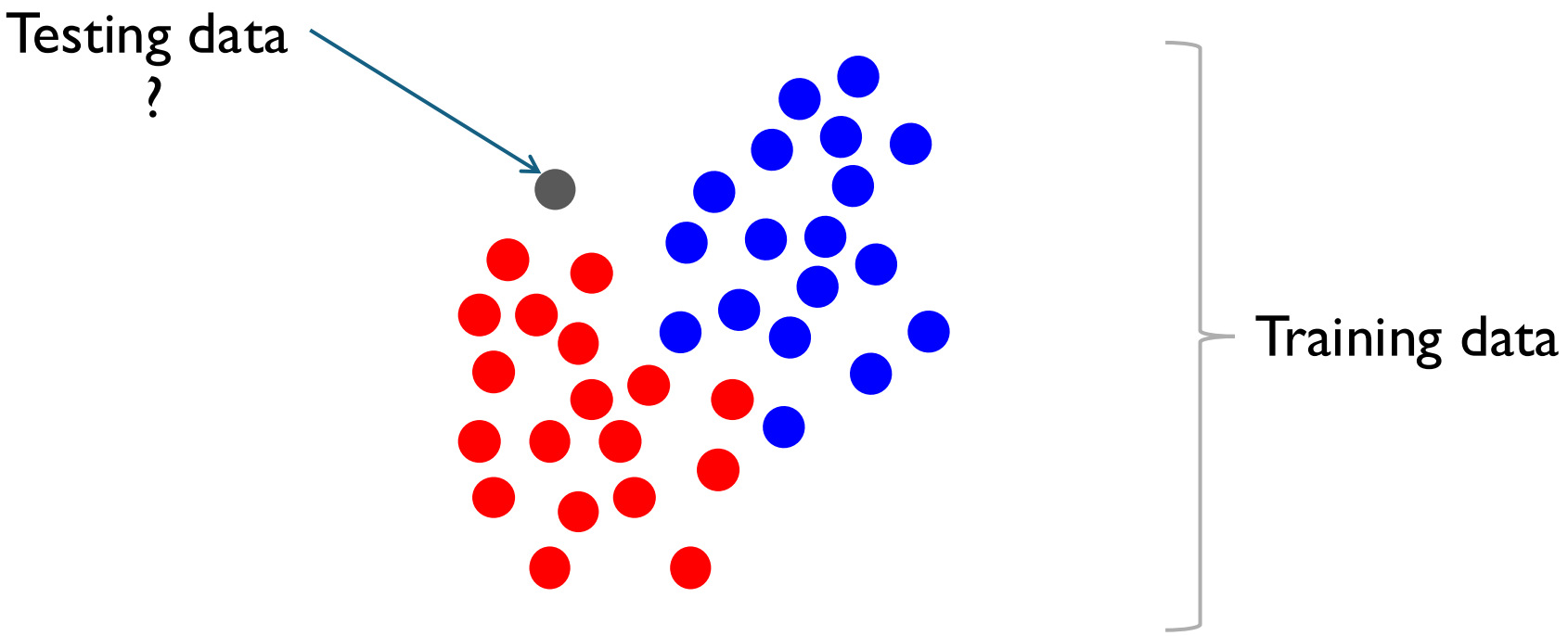
Minimal Distance

DMIN: Minimal Distance

DMIN: Minimal Distance

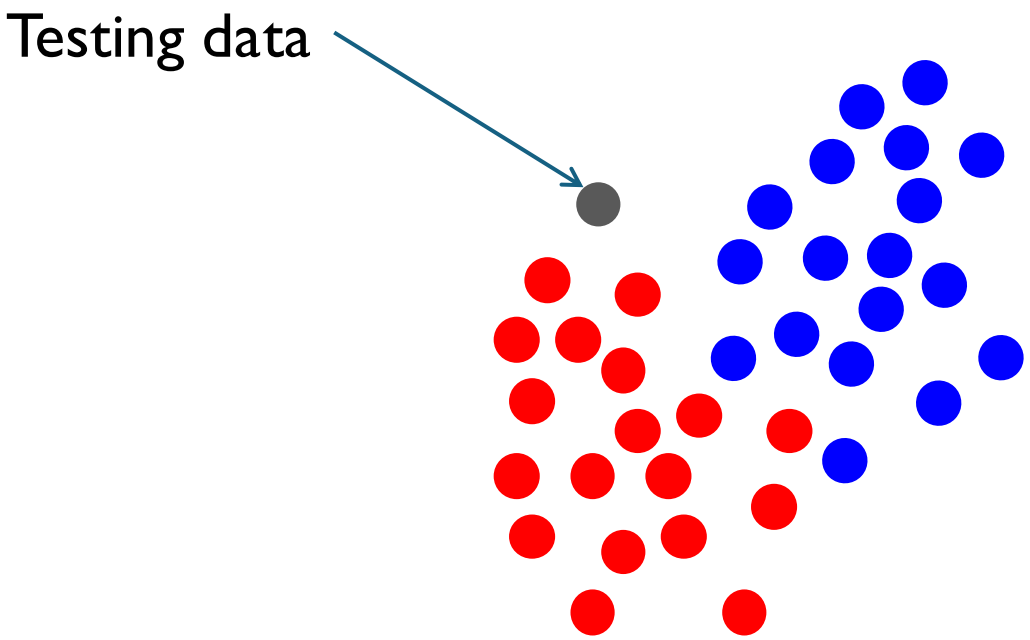


DMIN: Minimal Distance (two classes)

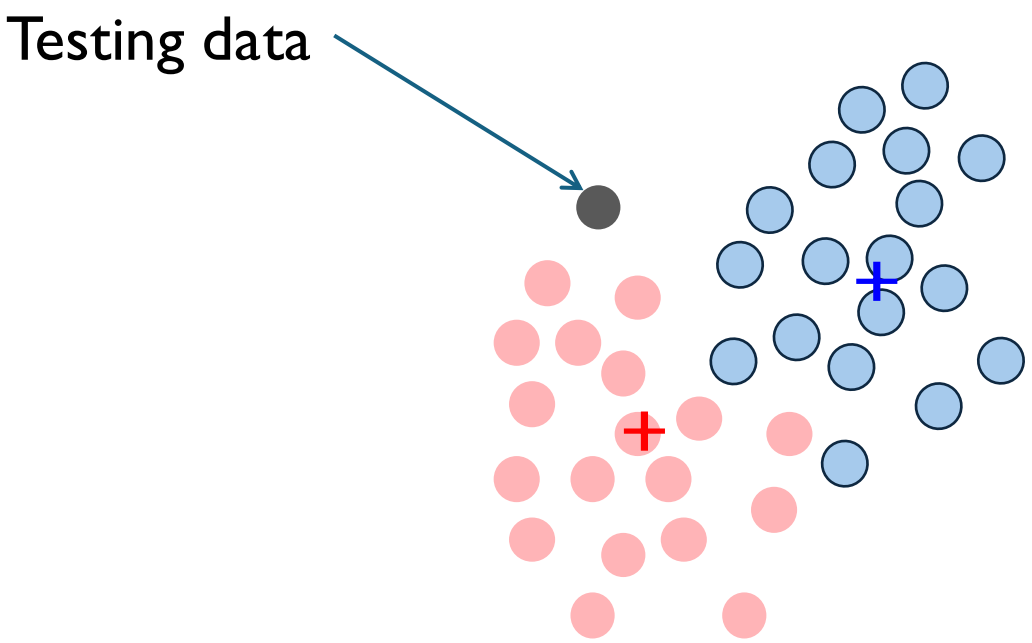


DMIN: Minimal Distance (two classes)

DMIN Algorithm



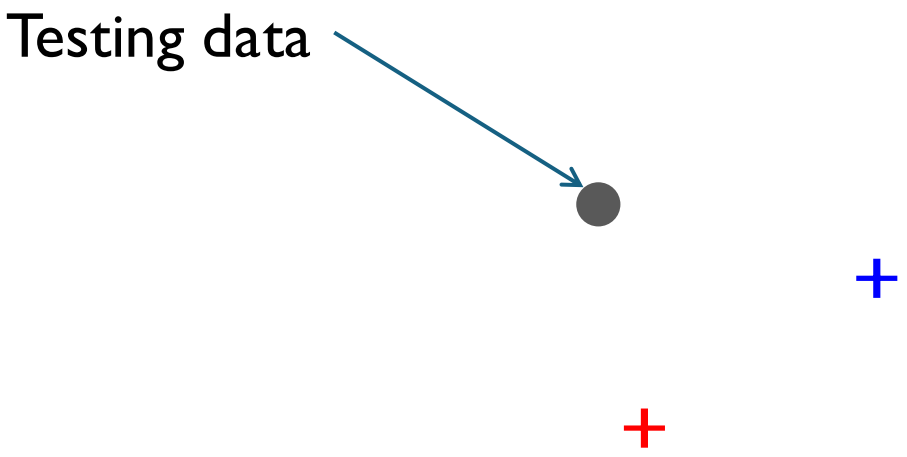
DMIN: Minimal Distance (two classes)



DMIN Algorithm

1. Centers of mass of each class

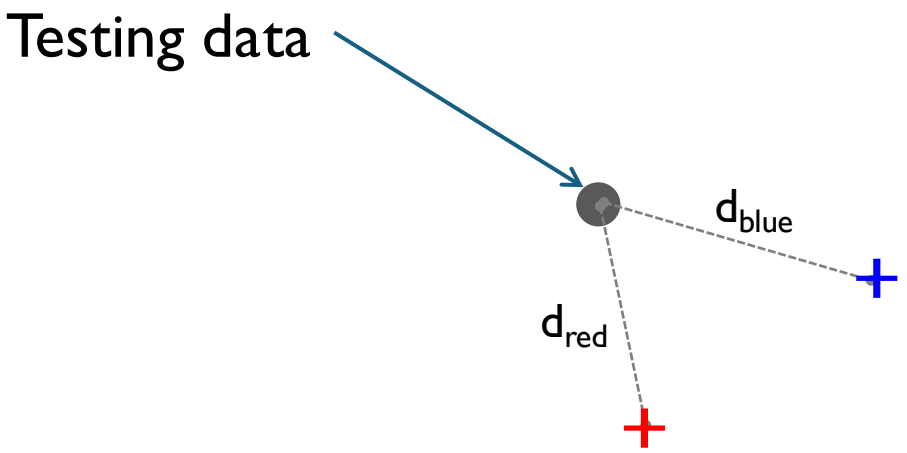
DMIN: Minimal Distance (two classes)



DMIN Algorithm

1. Centers of mass of each class

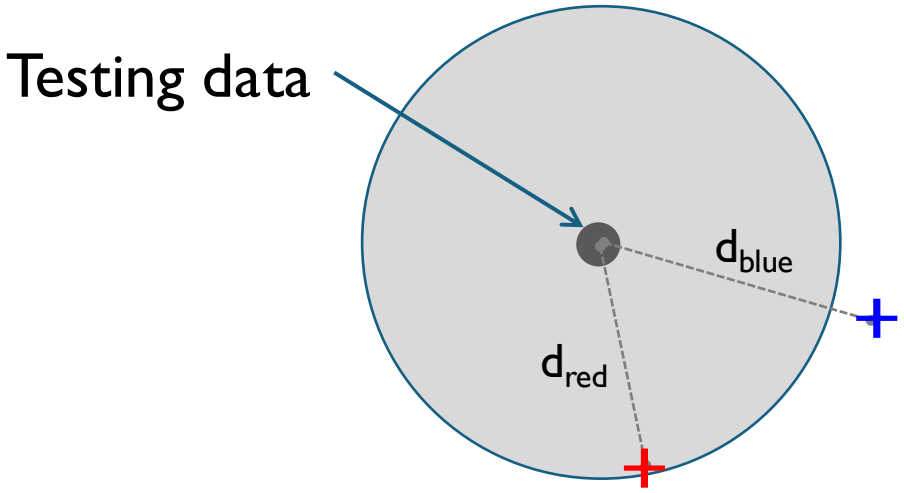
DMIN: Minimal Distance (two classes)



DMIN Algorithm

1. Centers of mass of each class
2. Distances

DMIN: Minimal Distance (two classes)

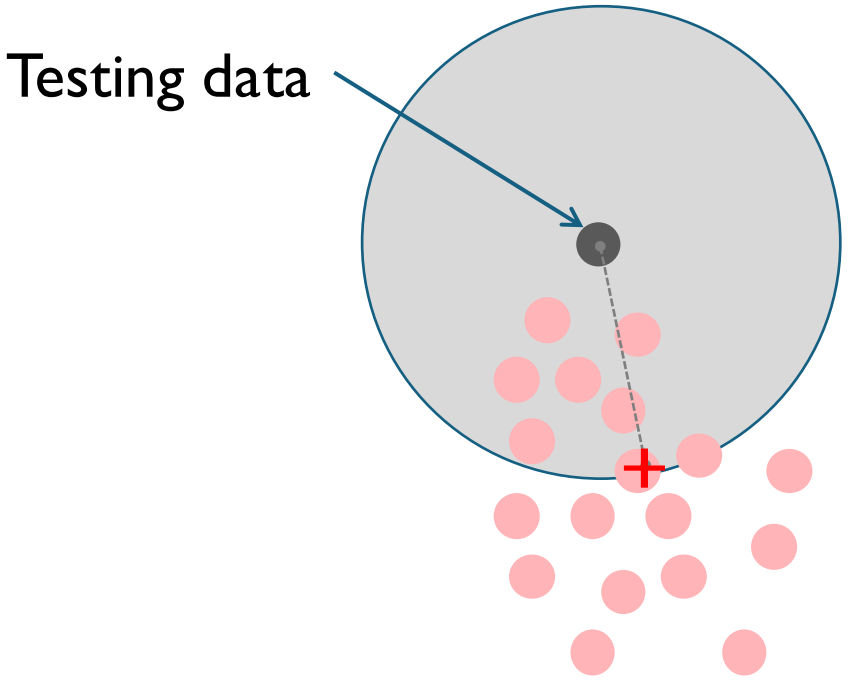


DMIN Algorithm

1. Centers of mass of each class
2. Distances
3. Take the class of the minimal distance

$d_{red} < d_{blue}$ ➔ Testing sample is red

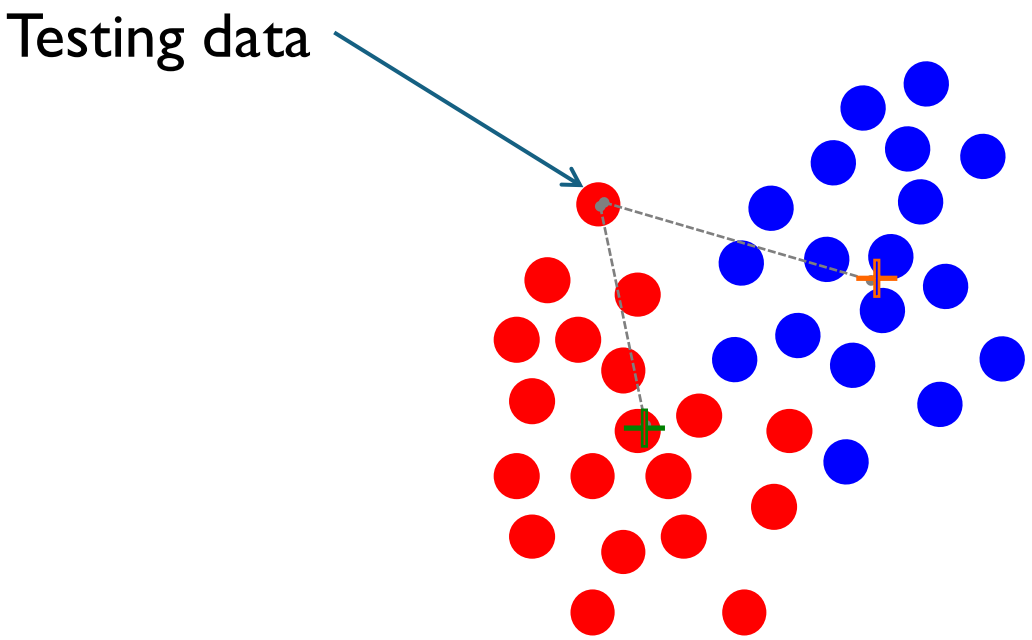
DMIN: Minimal Distance (two classes)



DMIN Algorithm

1. Centers of mass of each class
2. Distances
3. Take the class of the minimal distance

DMIN: Minimal Distance (two classes)

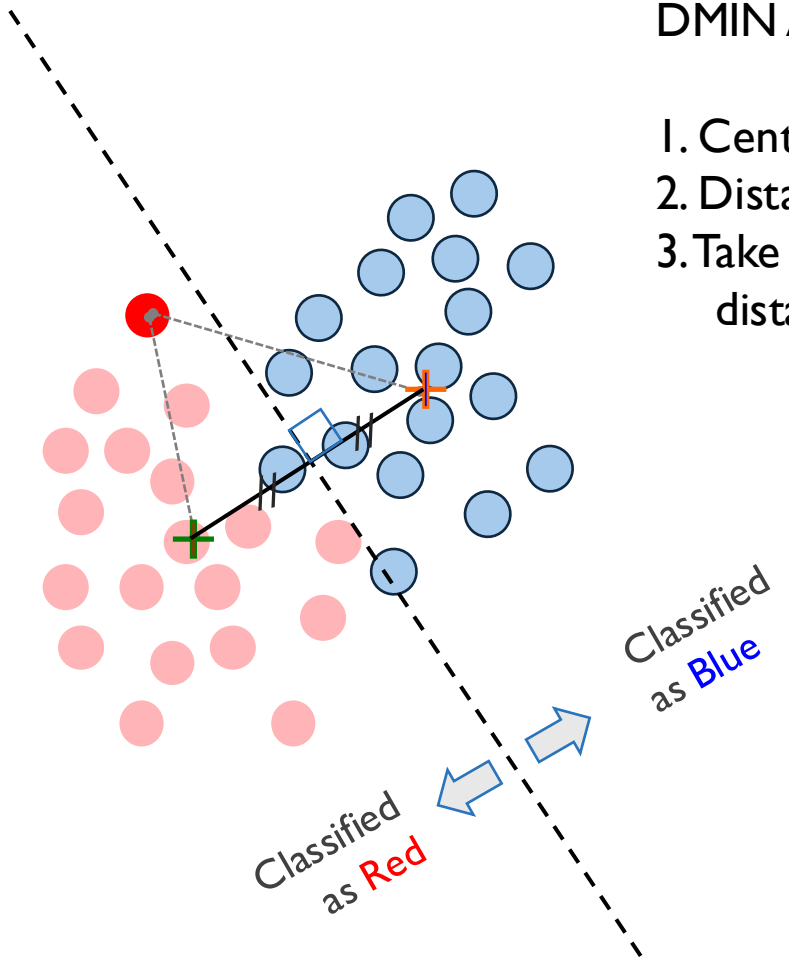


DMIN Algorithm

1. Centers of mass of each class
2. Distances
3. Take the class of the minimal distance

DMIN: Minimal Distance (two classes)

Decision Line

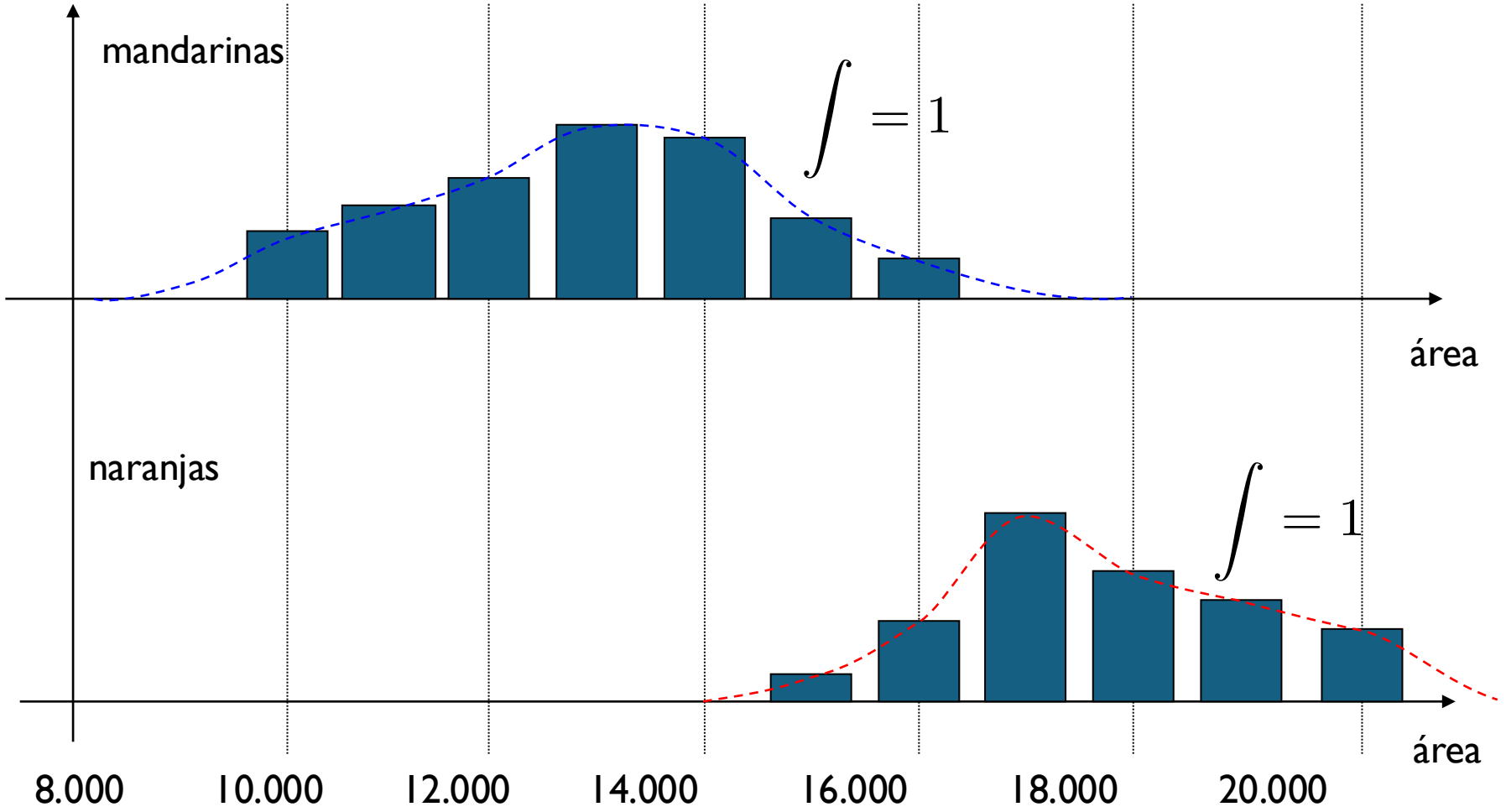


DMIN Algorithm

1. Centers of mass of each class
2. Distances
3. Take the class of the minimal distance

Bayes

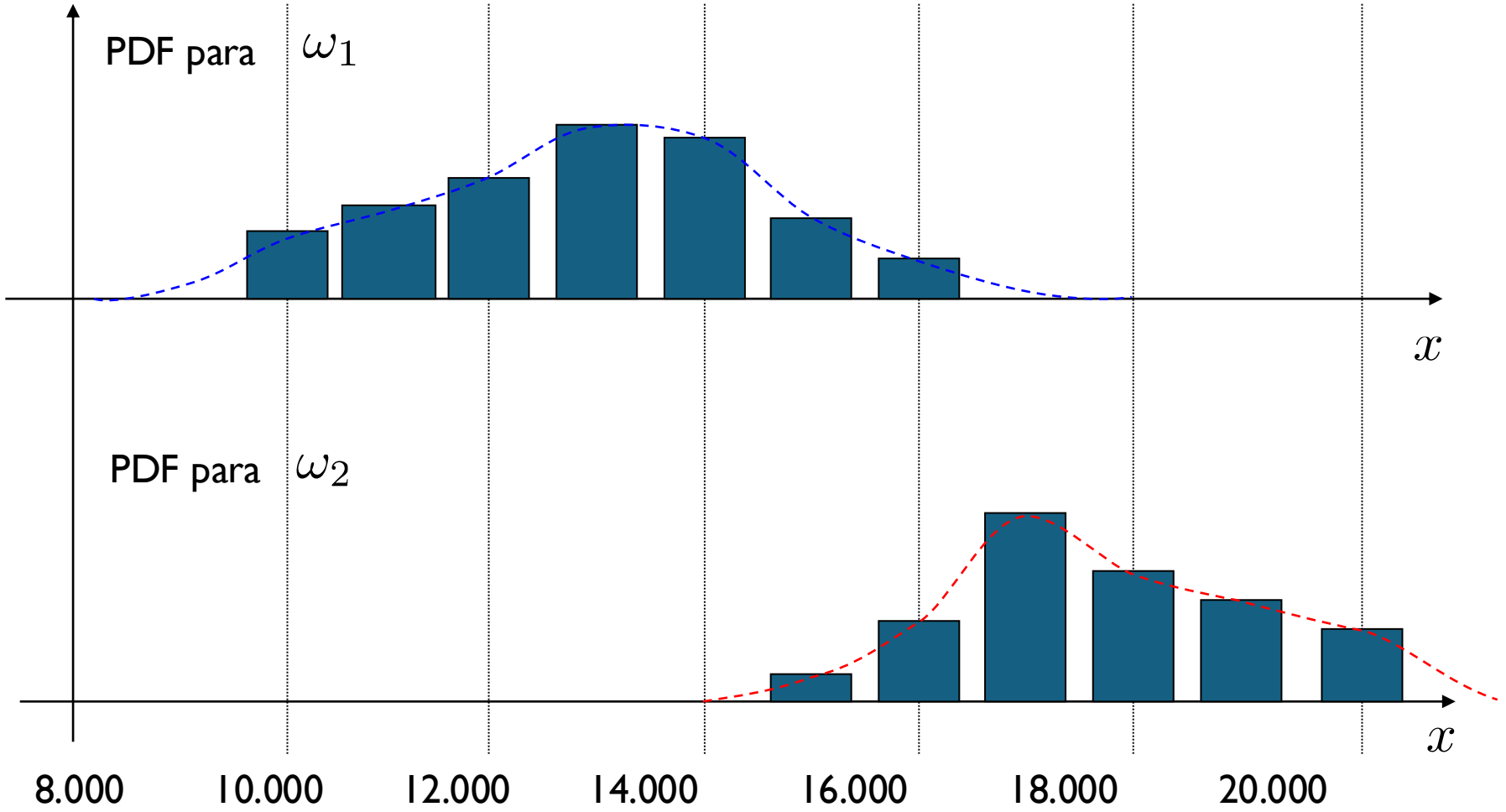
PDF: Probability Density Function



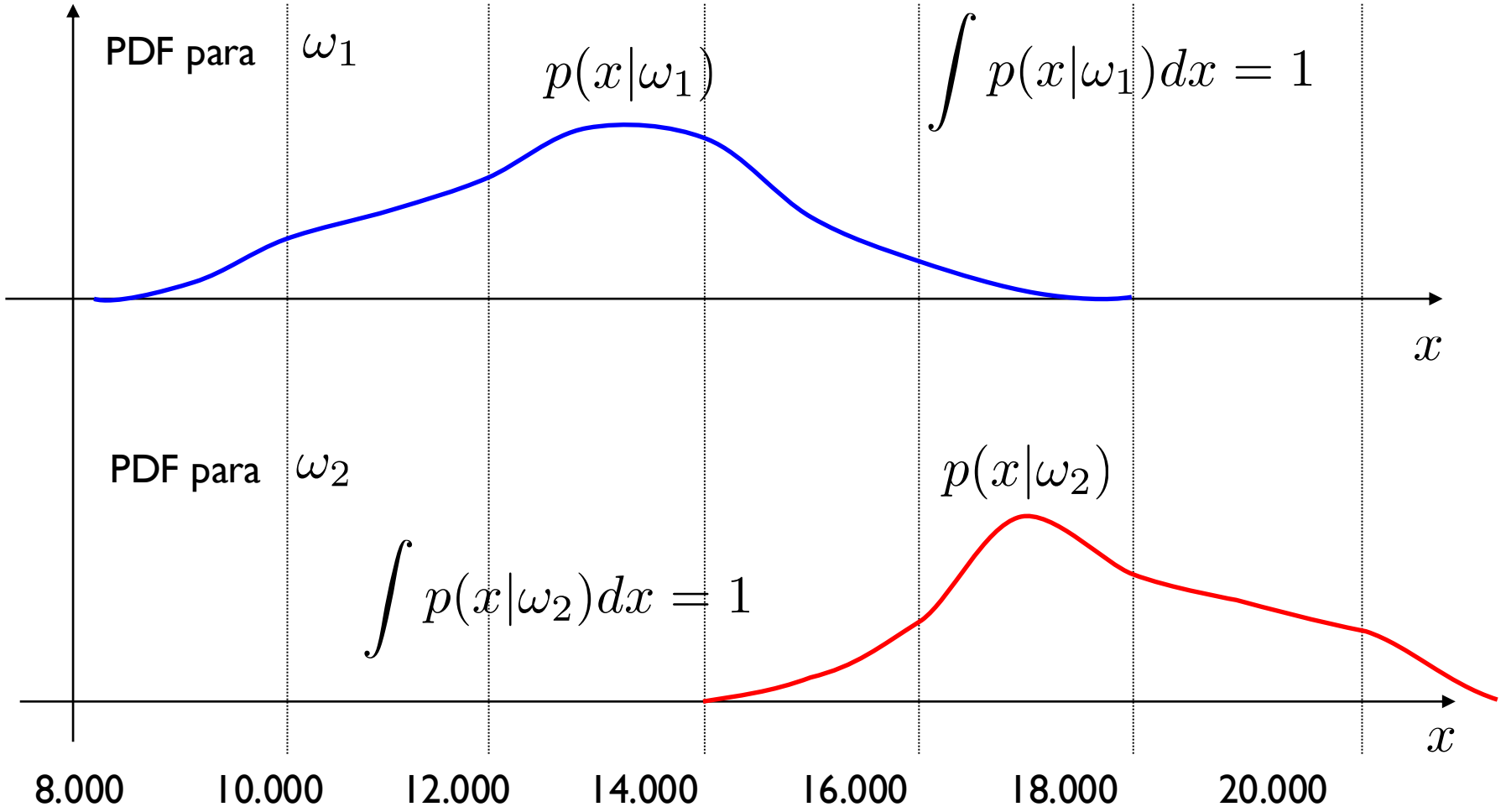
Definiciones

ω_1	Clase Mandarina
ω_2	Clase Naranja
x	Característica (área)

PDF: Probability Density Function



PDF: Probability Density Function



Definiciones

ω_1	Clase Mandarina
ω_2	Clase Naranja
x	Característica (área)
$p(x\|\omega_1)$	Distribución de x para Mandarinas
$p(x\|\omega_2)$	Distribución de x para Naranjas

Clasificación según Bayes

Dado el valor x (área), calculamos

- p_1, la probabilidad de que sea ω_1 (mandarina)
- p_2, la probabilidad de que sea ω_2 (naranja)

Clasificamos x como ω_1 si $p_1 > p_2$,

en caso contrario clasificamos x como ω_2

Clasificación según Bayes

Dado el valor x (área), calculamos

- p_1, la probabilidad de que sea ω_1 (mandarina)
- p_2, la probabilidad de que sea ω_2 (naranja)

Clasificamos x como ω_1 si $p_1 > p_2$,

en caso contrario clasificamos x como ω_2

ALGORITMO:

$$p(\omega_1|x) > p(\omega_2|x) \rightarrow \omega_1 \text{ (mandarina)}$$

$$\text{else} \rightarrow \omega_2 \text{ (naranja)}$$

$p_1 = p(\omega_1|x)$

$p_2 = p(\omega_2|x)$

Clasificación según Bayes

ALGORITMO:

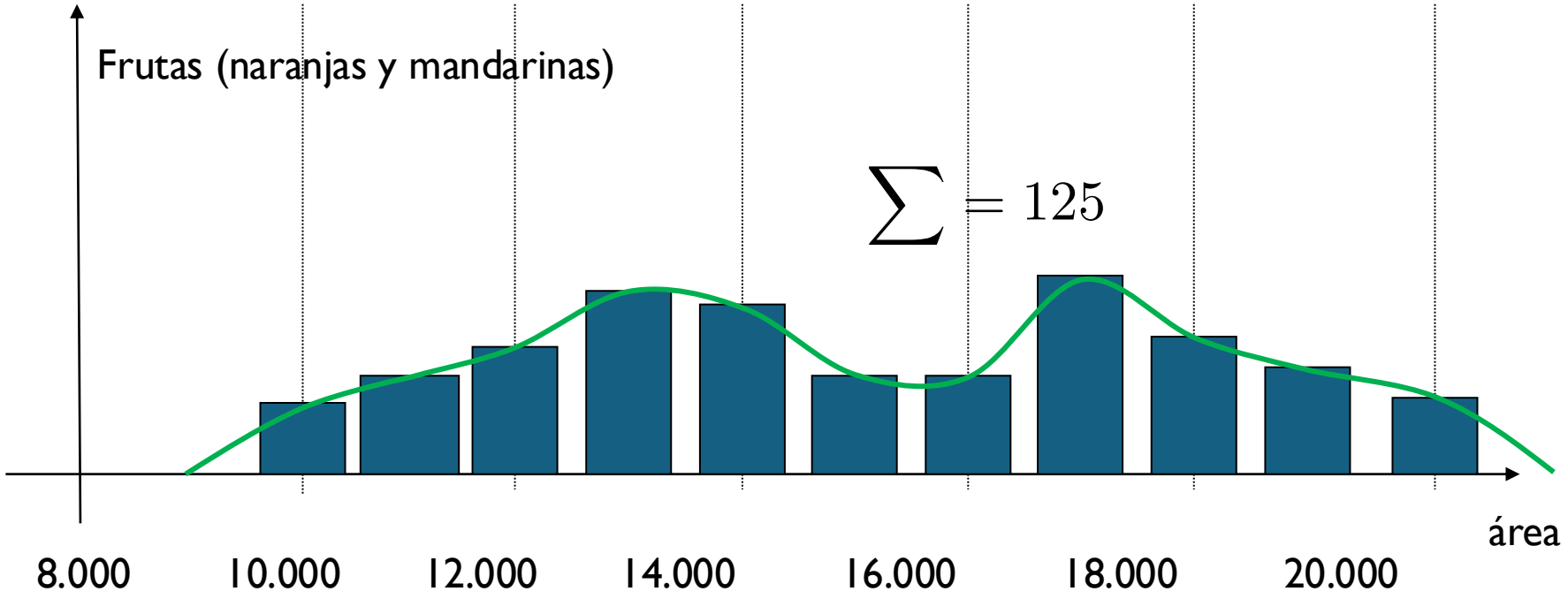
$p(\omega_1|x) > p(\omega_2|x) \rightarrow \omega_1$ (mandarina)

else $\rightarrow \omega_2$ (naranja)

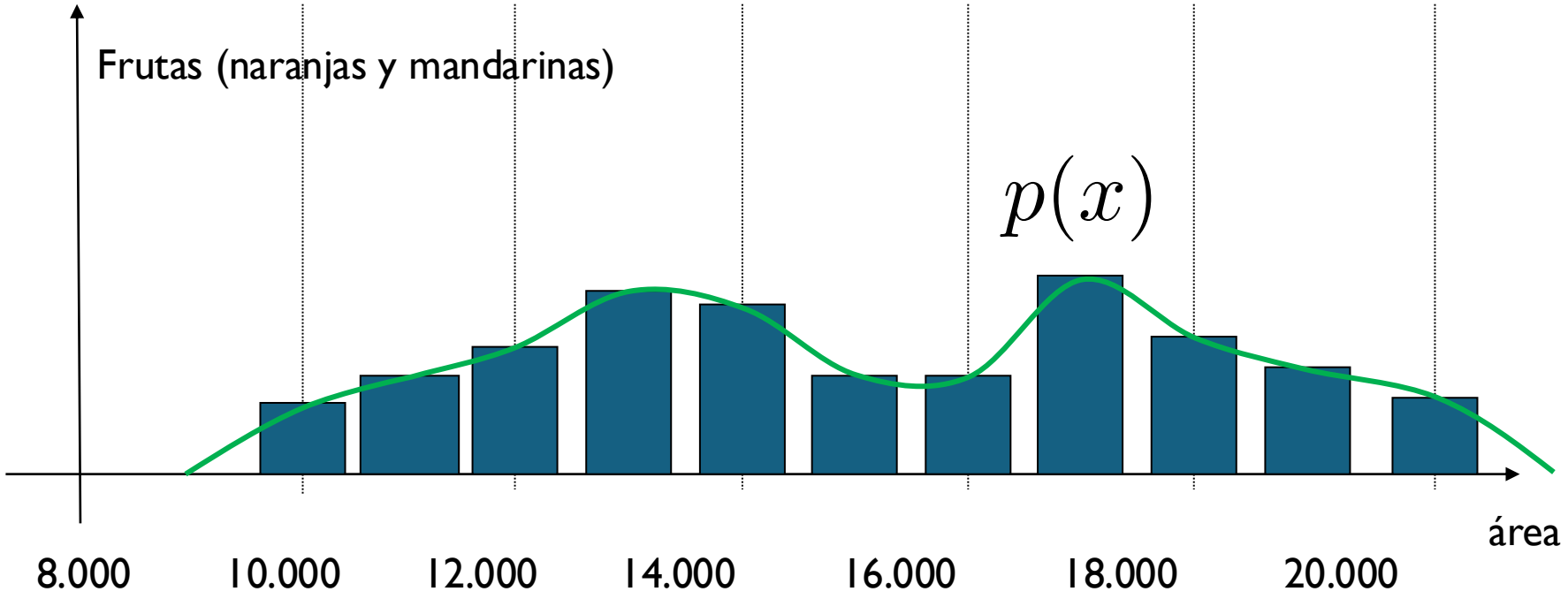
$$p(\omega_1|x) = \frac{p(\omega_1)p(x|\omega_1)}{p(x)}$$

$$p(\omega_2|x) = \frac{p(\omega_2)p(x|\omega_2)}{p(x)}$$

Histogramas:



PDF:


Frutas (naranjas y mandarinas)
$p(x)$
área
8.000
10.000
12.000
14.000
16.000
18.000
20.000

Clasificación según Bayes

ALGORITMO:

$p(\omega_1|x) > p(\omega_2|x) \rightarrow \omega_1$ (mandarina)

else $\rightarrow \omega_2$ (naranja)

$$p(\omega_1|x) = \frac{p(\omega_1)p(x|\omega_1)}{p(x)}$$

$$p(\omega_2|x) = \frac{p(\omega_2)p(x|\omega_2)}{p(x)}$$

Clasificación según Bayes

ALGORITMO:

$$p(\omega_1|x) > p(\omega_2|x) \rightarrow \omega_1 \text{ (mandarina)}$$

$$\text{else} \rightarrow \omega_2 \text{ (naranja)}$$



$$\frac{p(\omega_1)p(x|\omega_1)}{\cancel{p(x)}} > \frac{p(\omega_2)p(x|\omega_2)}{\cancel{p(x)}}$$

Clasificación según Bayes

ALGORITMO:

$p(\omega_1|x) > p(\omega_2|x) \rightarrow \omega_1$ (mandarina)

else $\rightarrow \omega_2$ (naranja)

$$p(\omega_1)p(x|\omega_1) > p(\omega_2)p(x|\omega_2)$$

Clasificación según Bayes

ALGORITMO:

$$p(\omega_1|x) > p(\omega_2|x) \rightarrow \omega_1 \text{ (mandarina)}$$

$$\text{else} \rightarrow \omega_2 \text{ (naranja)}$$

ALGORITMO:

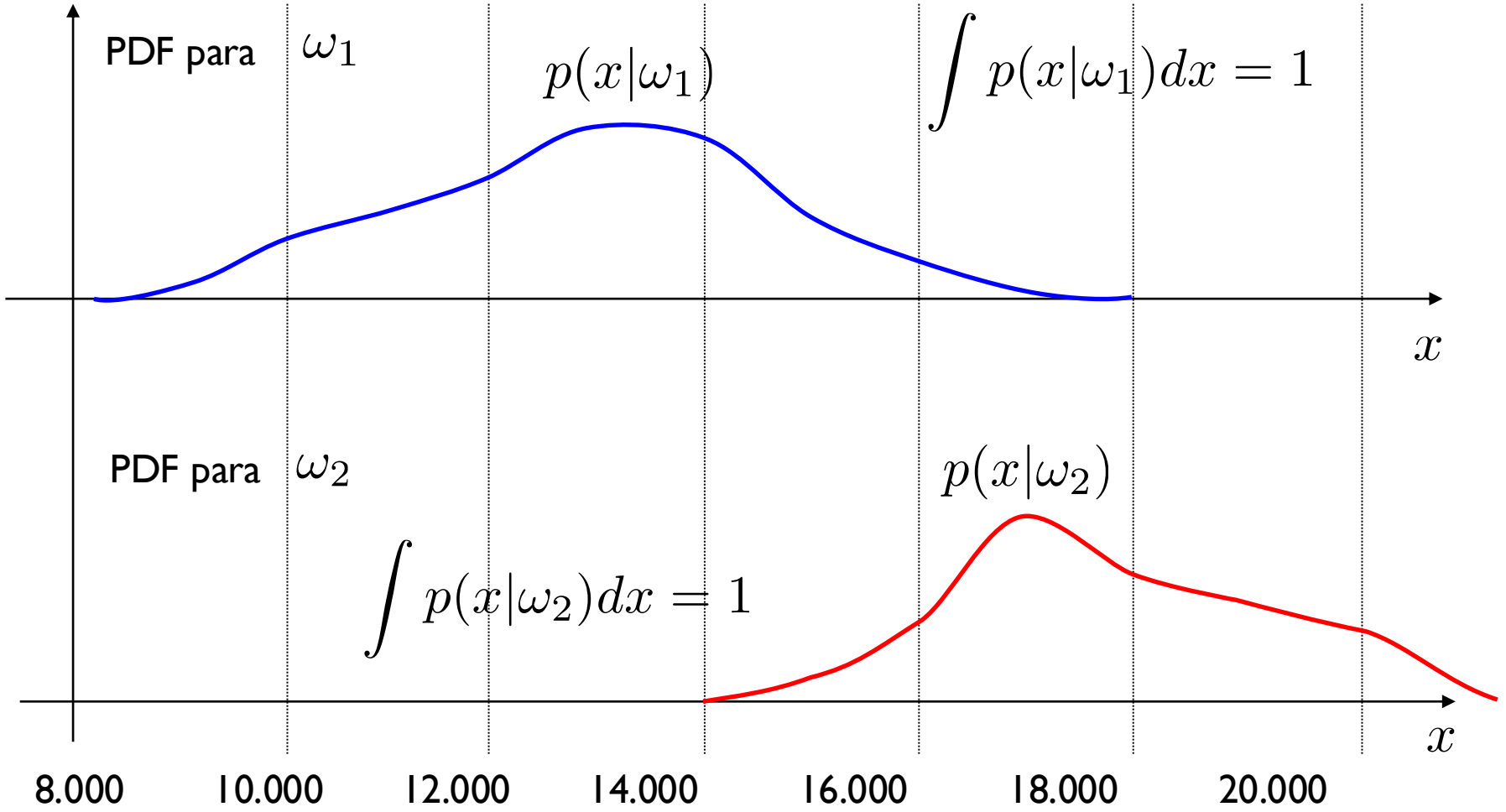
$$p(\omega_1)p(x|\omega_1) > p(\omega_2)p(x|\omega_2) \rightarrow \omega_1 \text{ (mandarina)}$$

$$\text{else} \rightarrow \omega_2 \text{ (naranja)}$$

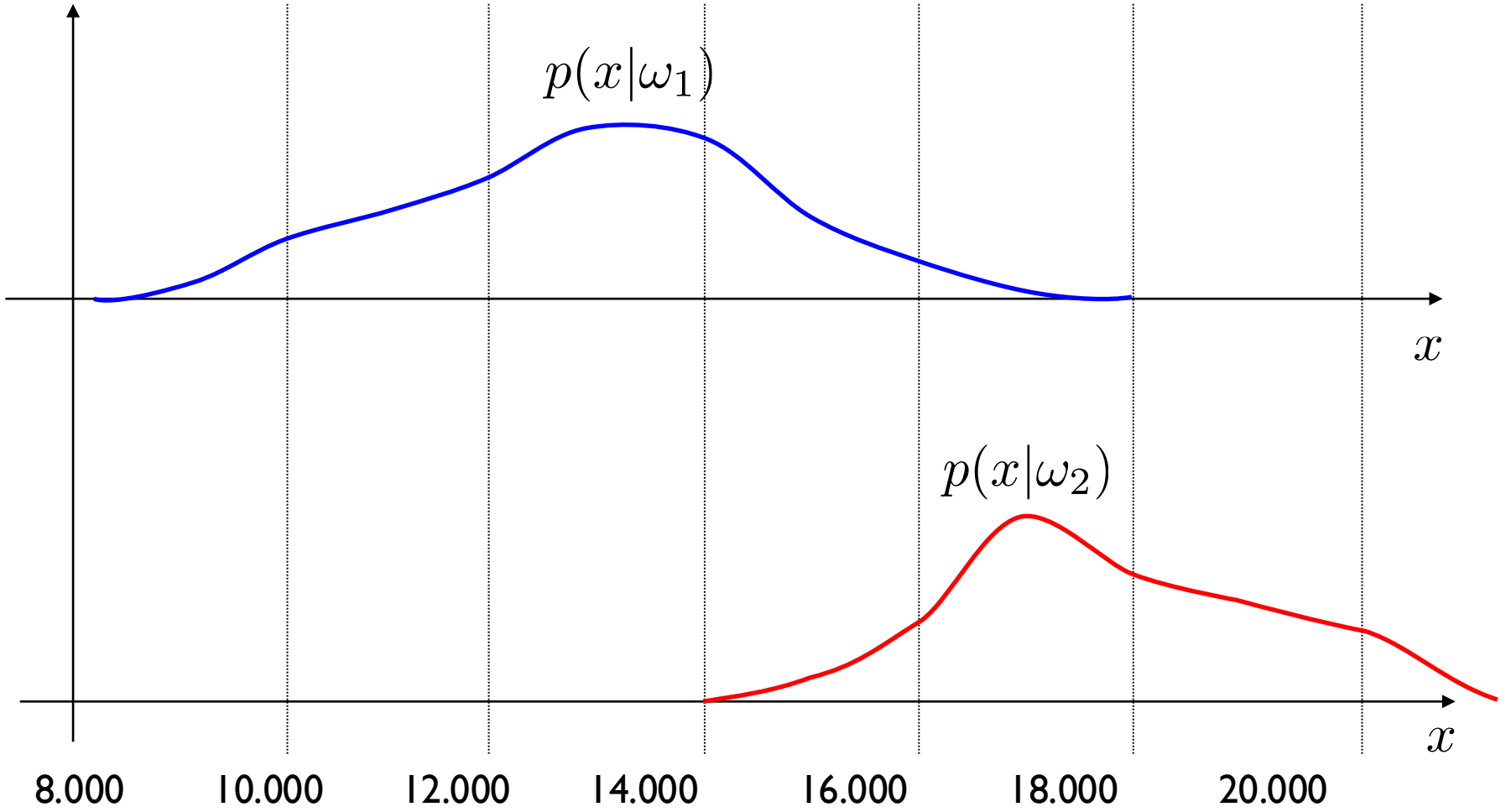
Definiciones

ω_1	Clase Mandarina
ω_2	Clase Naranja
x	Característica (área)
$p(x\|\omega_1)$	Distribución de x para Mandarinas
$p(x\|\omega_2)$	Distribución de x para Naranjas
$p(\omega_1)$	Probabilidad a priori de clase ω_1
$p(\omega_2)$	Probabilidad a priori de clase ω_2

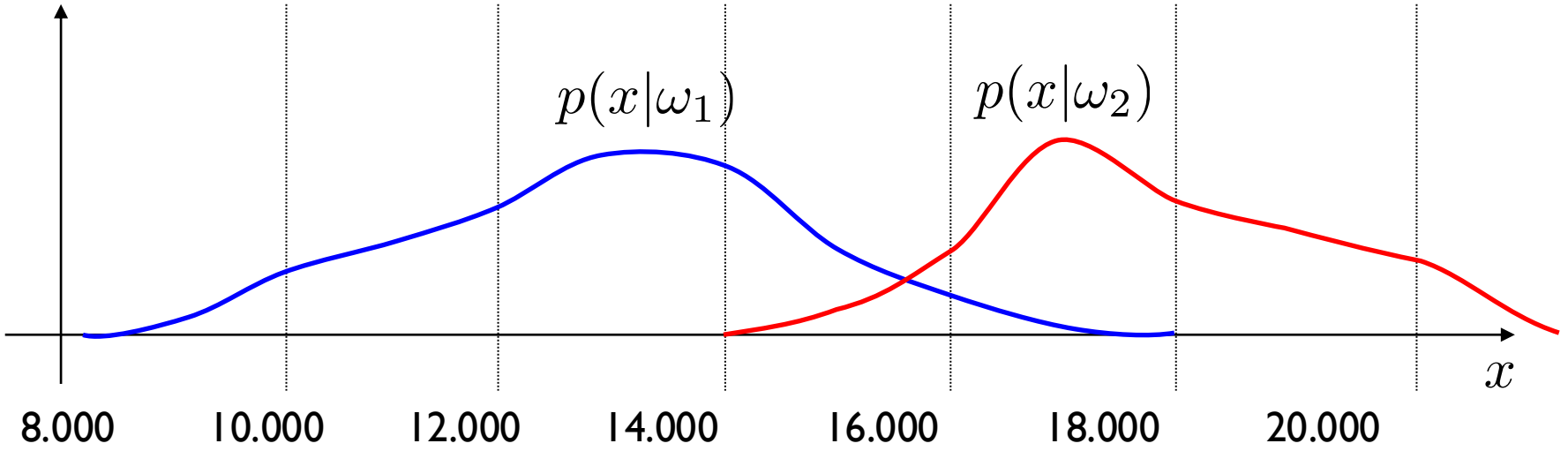
PDF: Probability Density Function



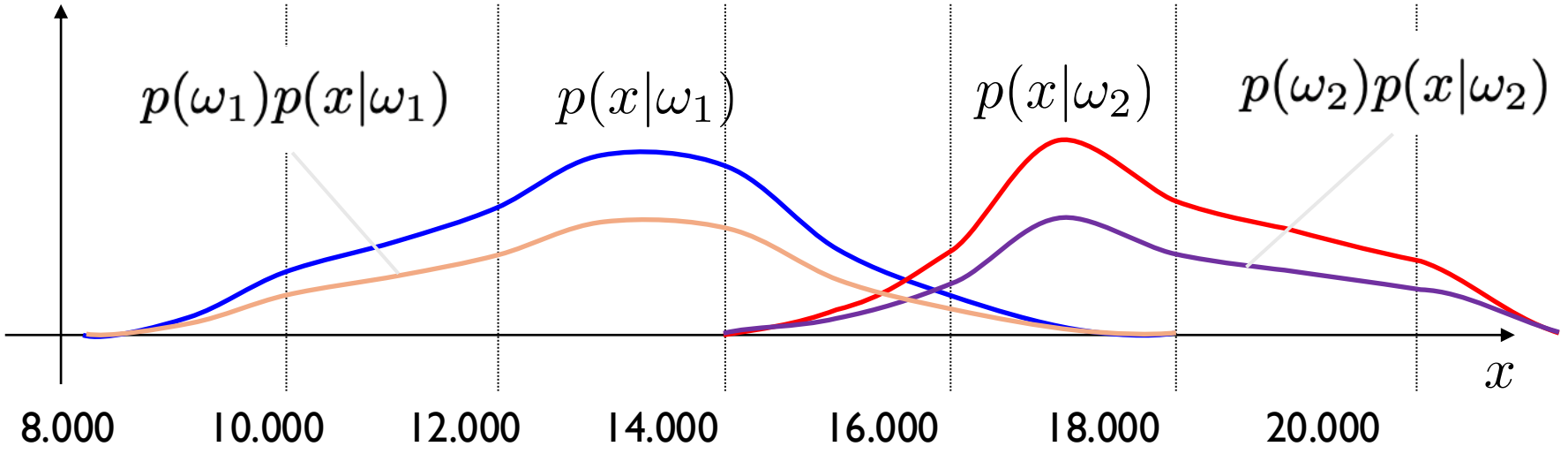
PDF: Probability Density Function



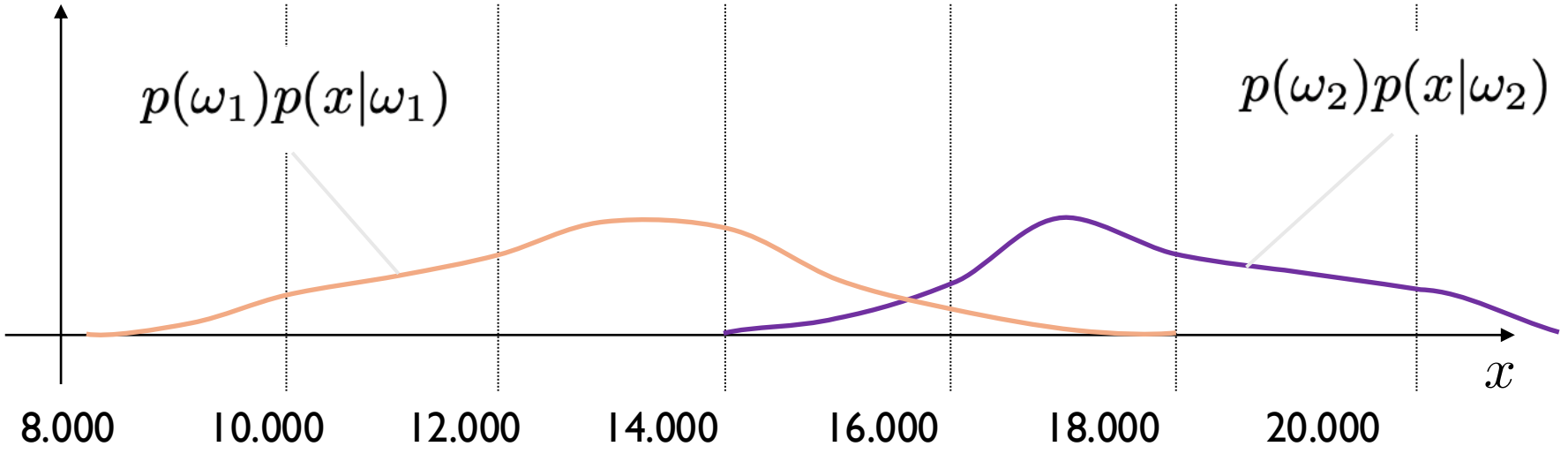
PDF: Probability Density Function



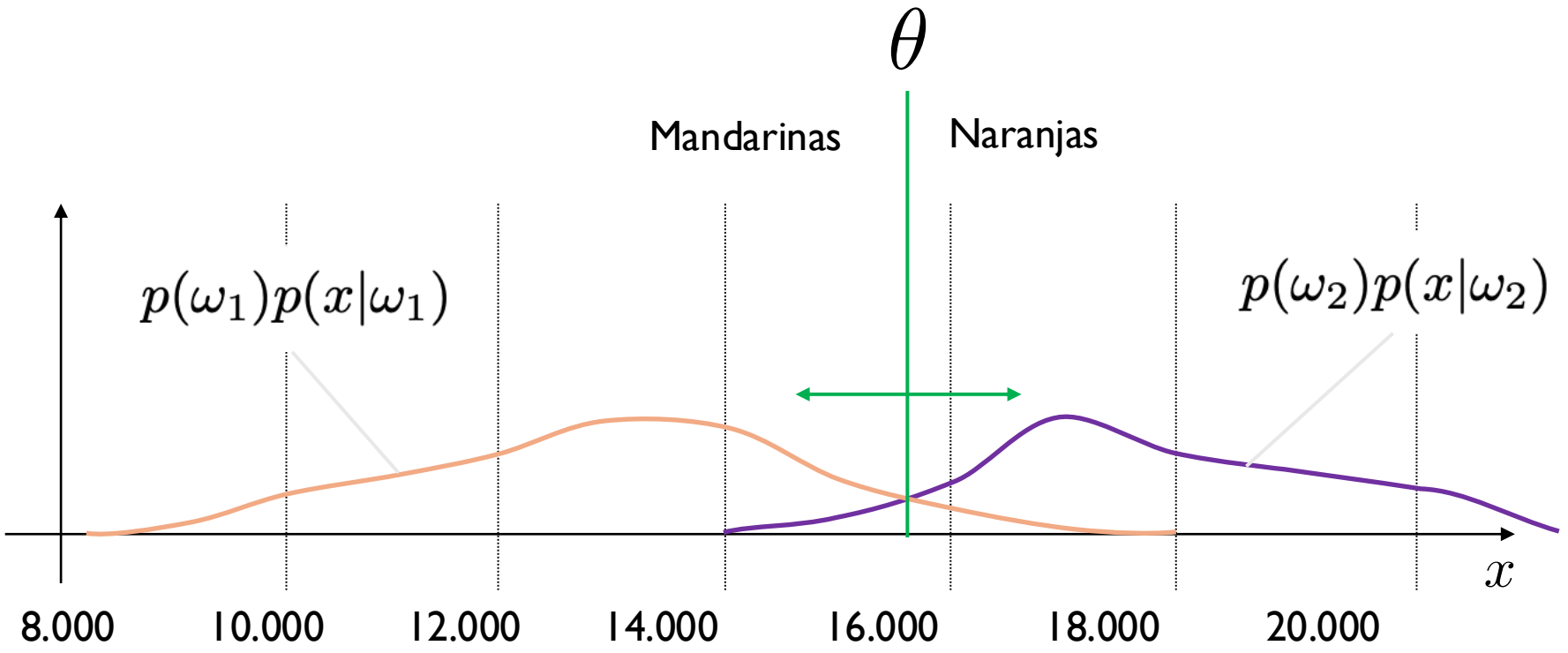
PDF: Probability Density Function



PDF: Probability Density Function



PDF: Probability Density Function



Clasificación según Bayes

ALGORITMO:

$$p(\omega_1|x) > p(\omega_2|x) \rightarrow \omega_1 \text{ (mandarina)}$$

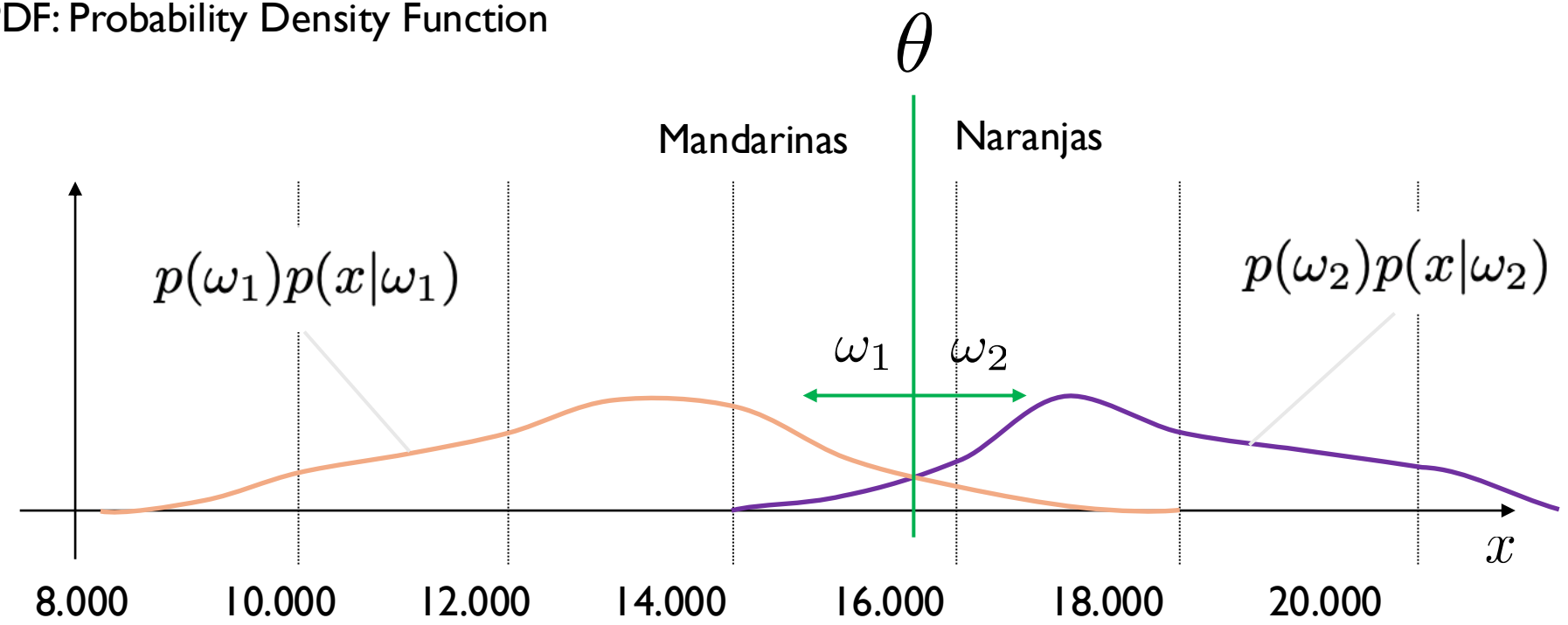
$$\text{else} \rightarrow \omega_2 \text{ (naranja)}$$

ALGORITMO:

$$p(\omega_1)p(x|\omega_1) \gtrless p(\omega_2)p(x|\omega_2) \rightarrow \omega_1 \text{ (mandarina)}$$

$$\text{else} \rightarrow \omega_2 \text{ (naranja)}$$

PDF: Probability Density Function



ALGORITMO:

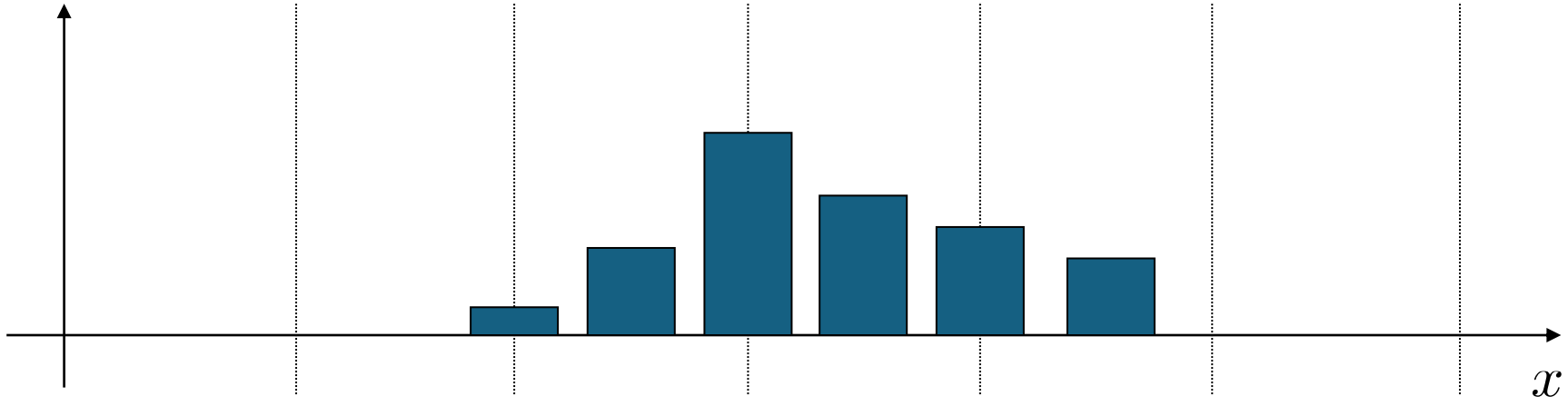
$x < \theta \rightarrow \omega_1$ (mandarina)

else $\rightarrow \omega_2$ (naranja)

Cómo calcular la PDF en 1D?
KDE

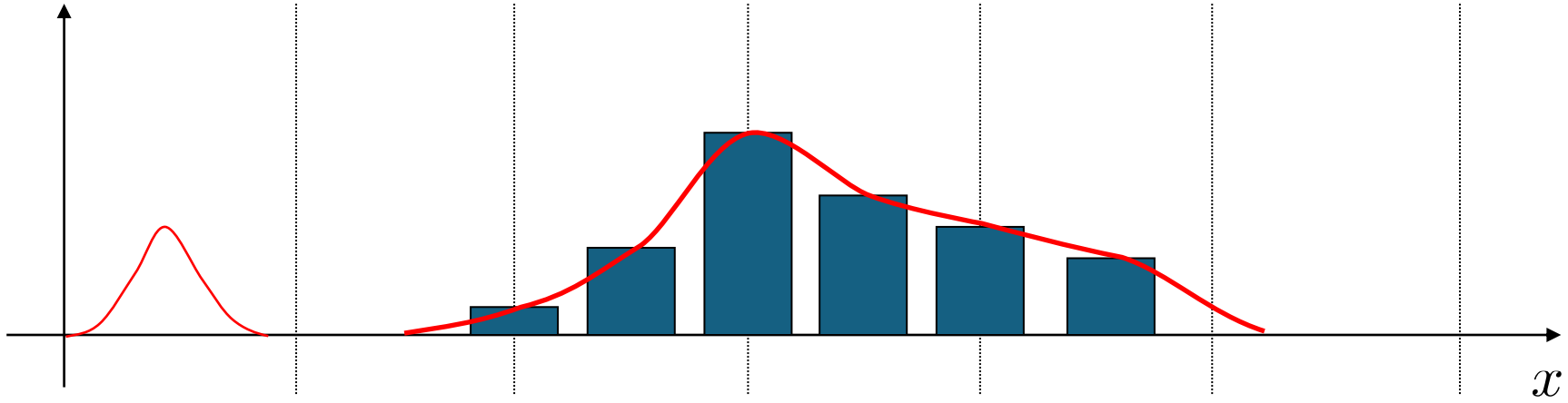
Cómo calcular la PDF?

PASO 1: Histograma



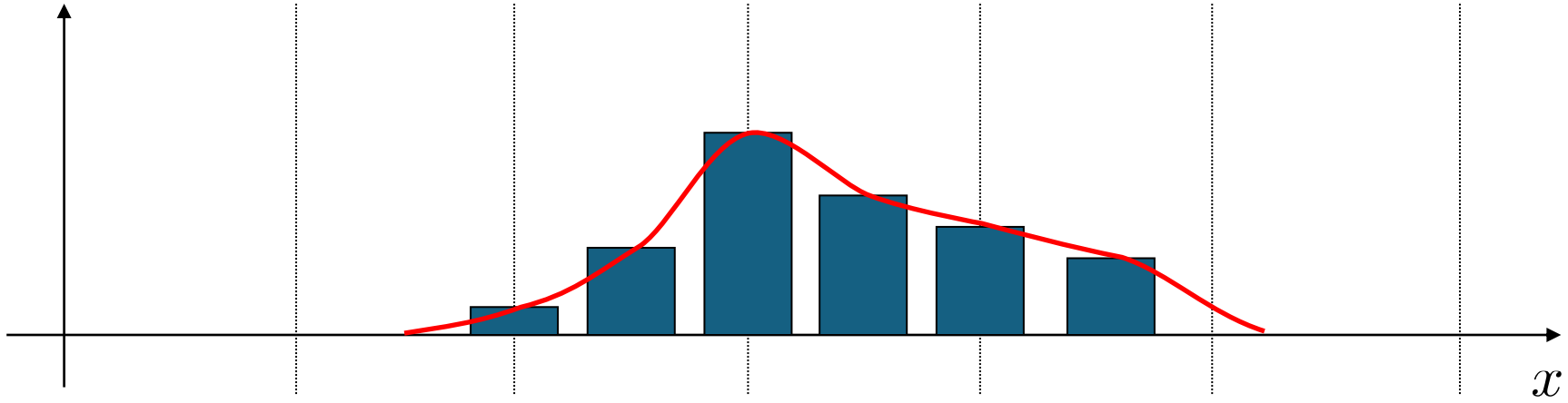
Cómo calcular la PDF?

PASO 2: Convolución con una Gaussiana (Kernel)



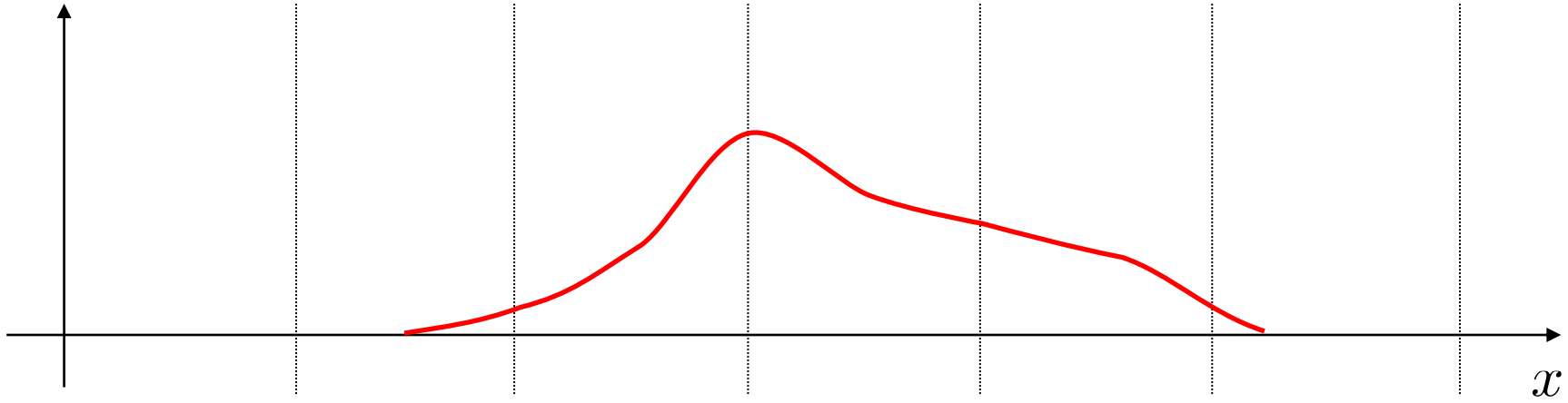
Cómo calcular la PDF?

PASO 2: Convolución con una Gaussiana (Kernel)



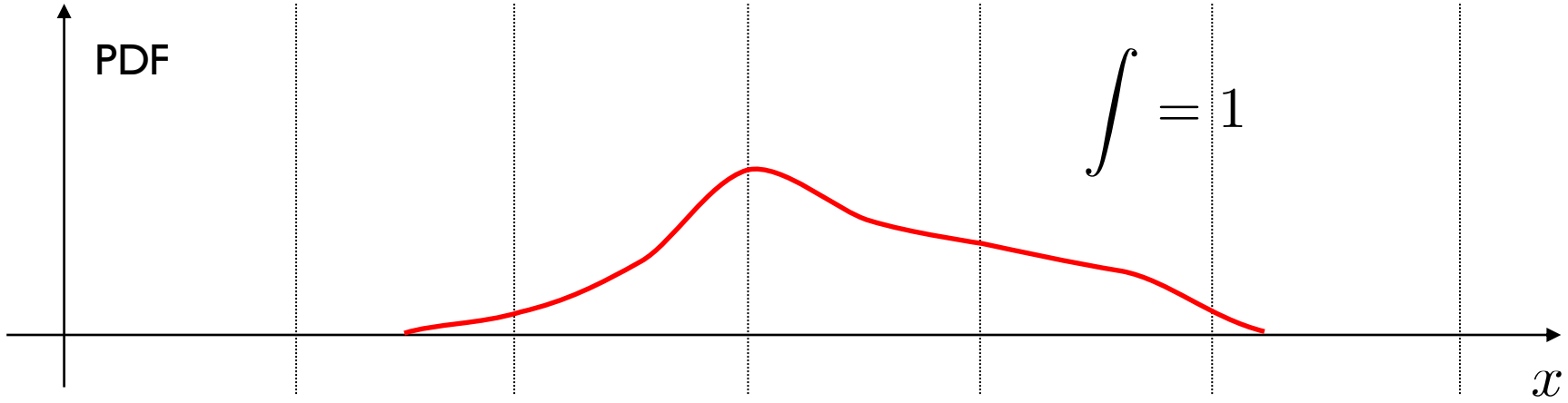
Cómo calcular la PDF?

PASO 2: Convolución con una Gaussiana (Kernel)

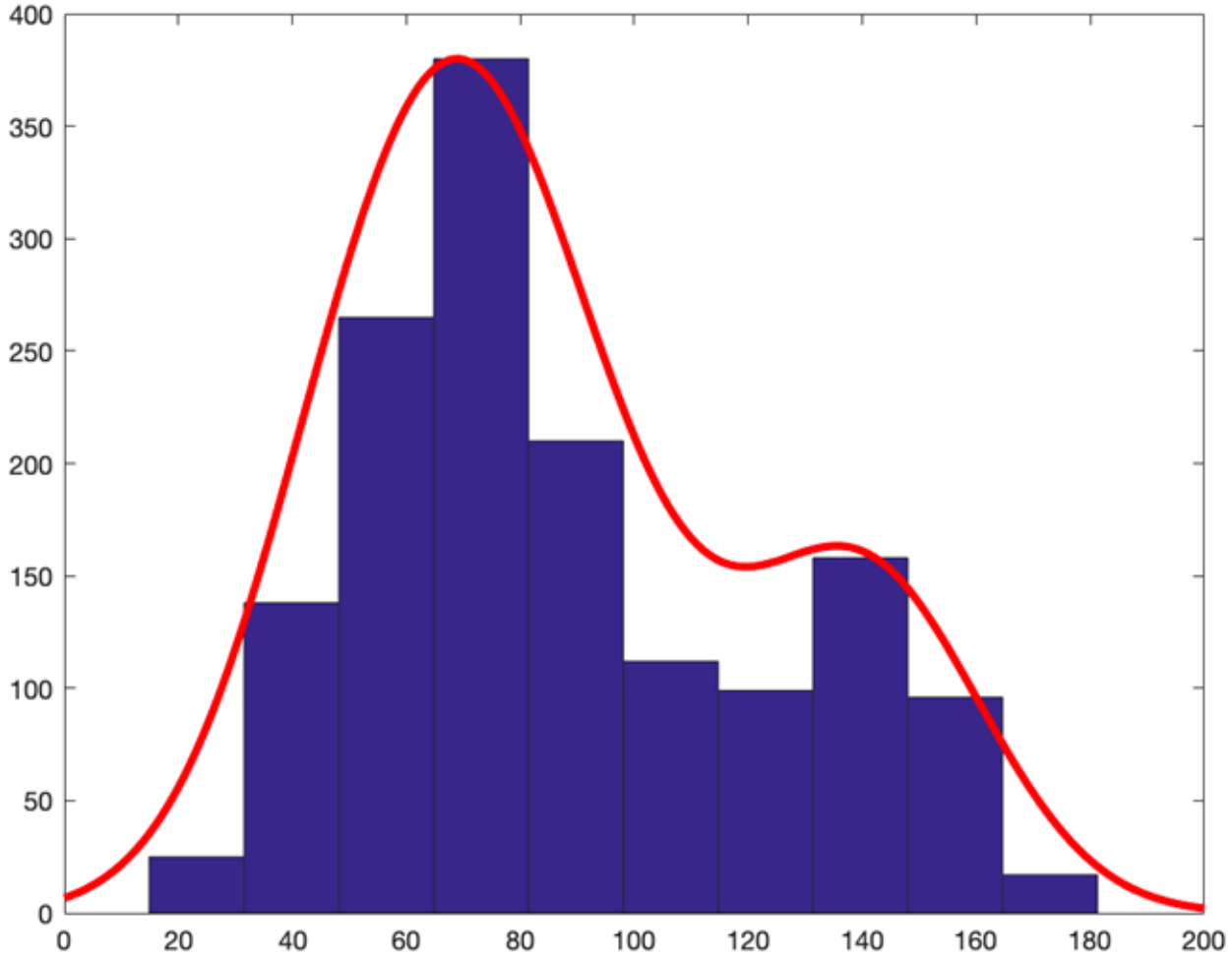


Cómo calcular la PDF?

PASO 3: División por el área

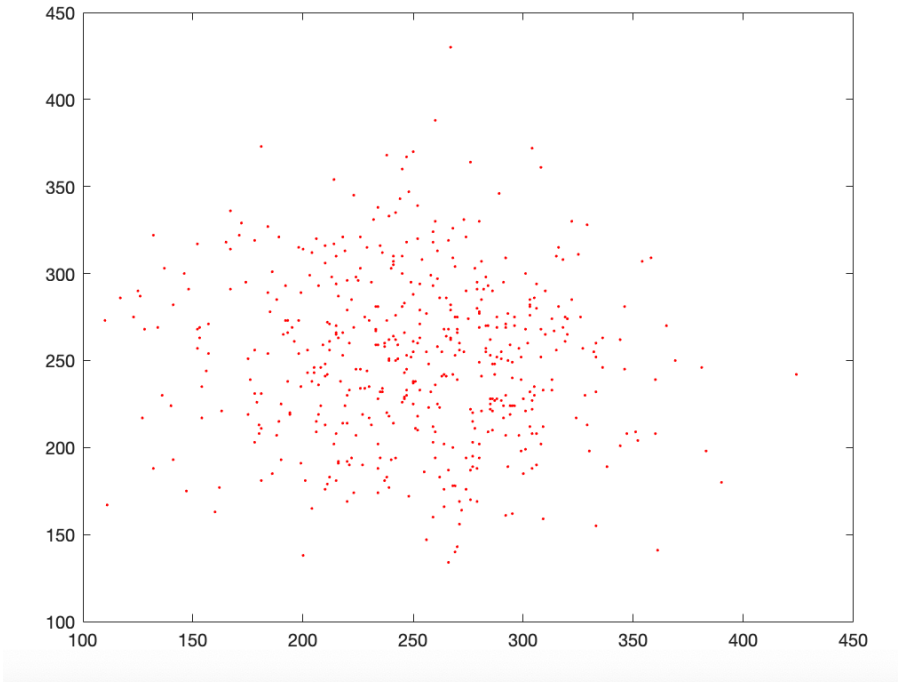


PDF en 1D

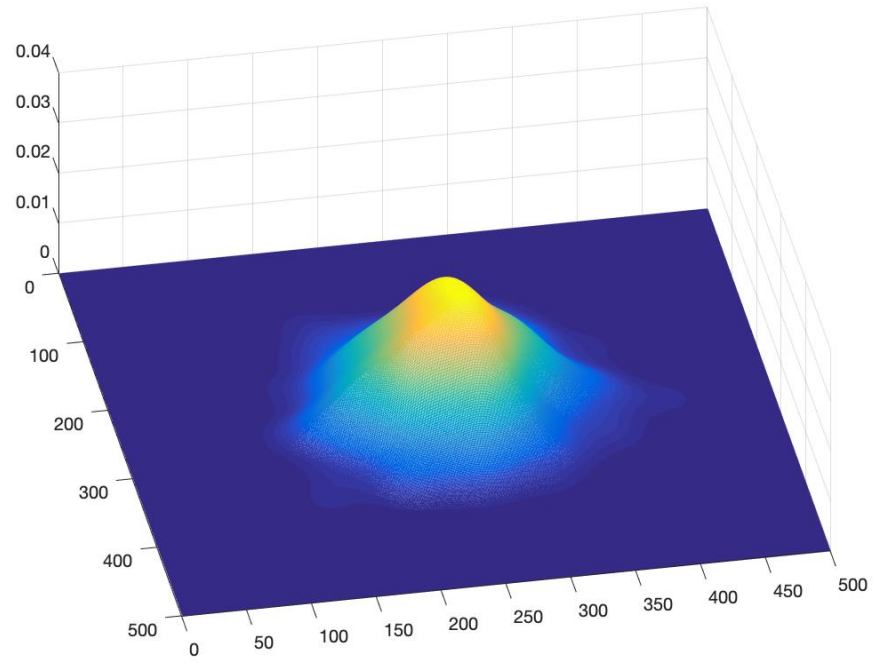

400
350
300
250
200
150
100
50
0
0
20
40
60
80
100
120
140
160
180
200

Cómo calcular la PDF en 2D?
KDE

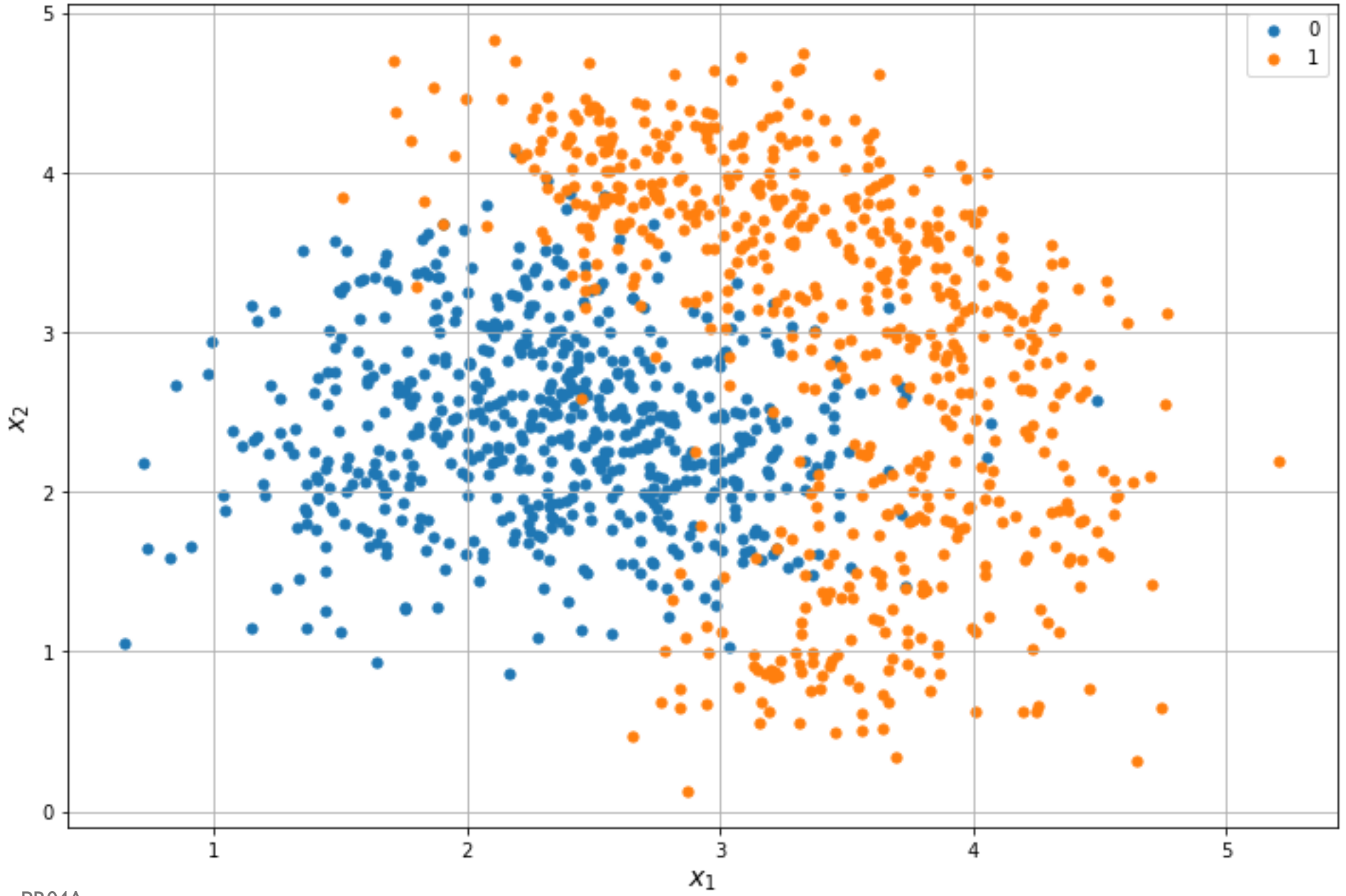
PDF en 2D

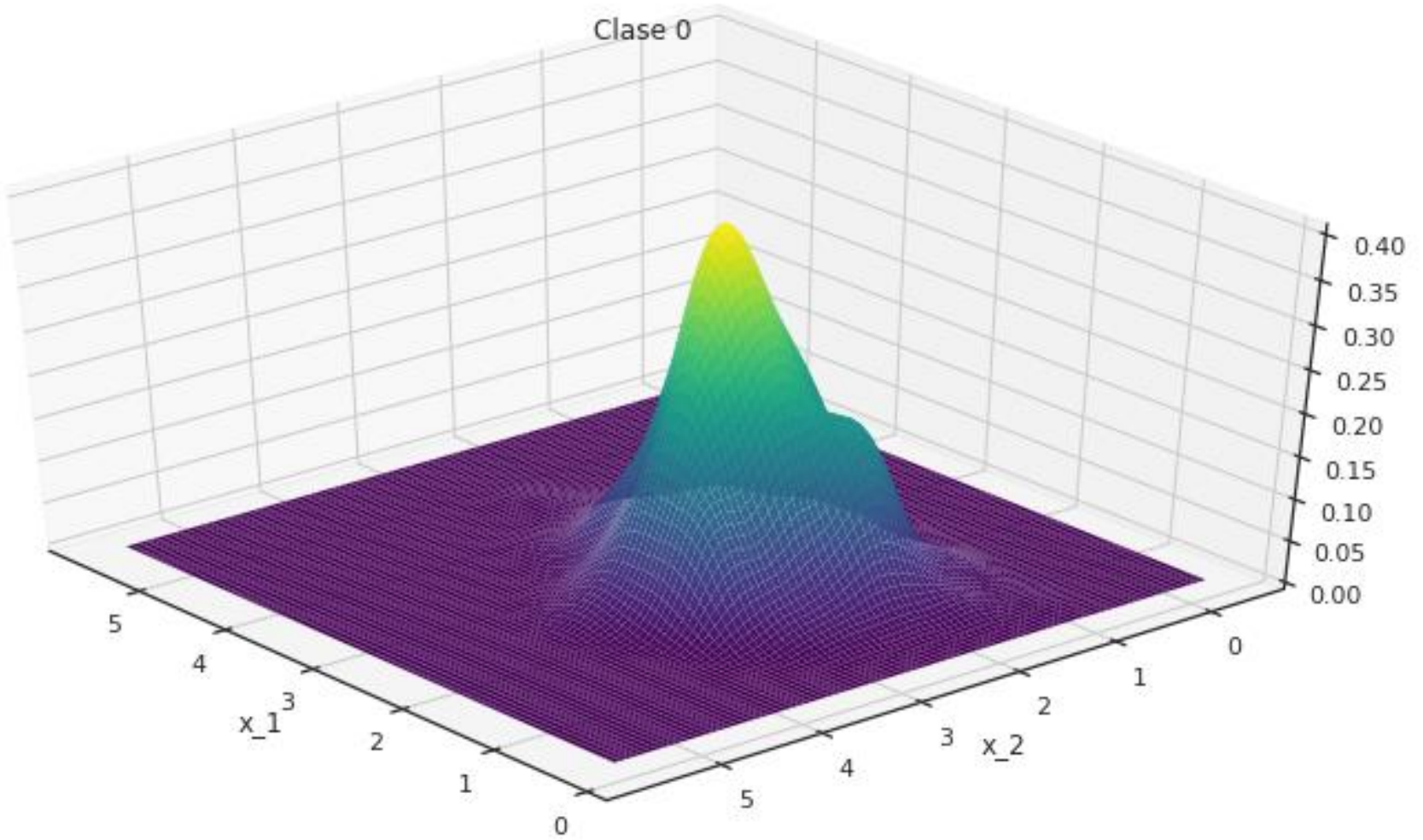


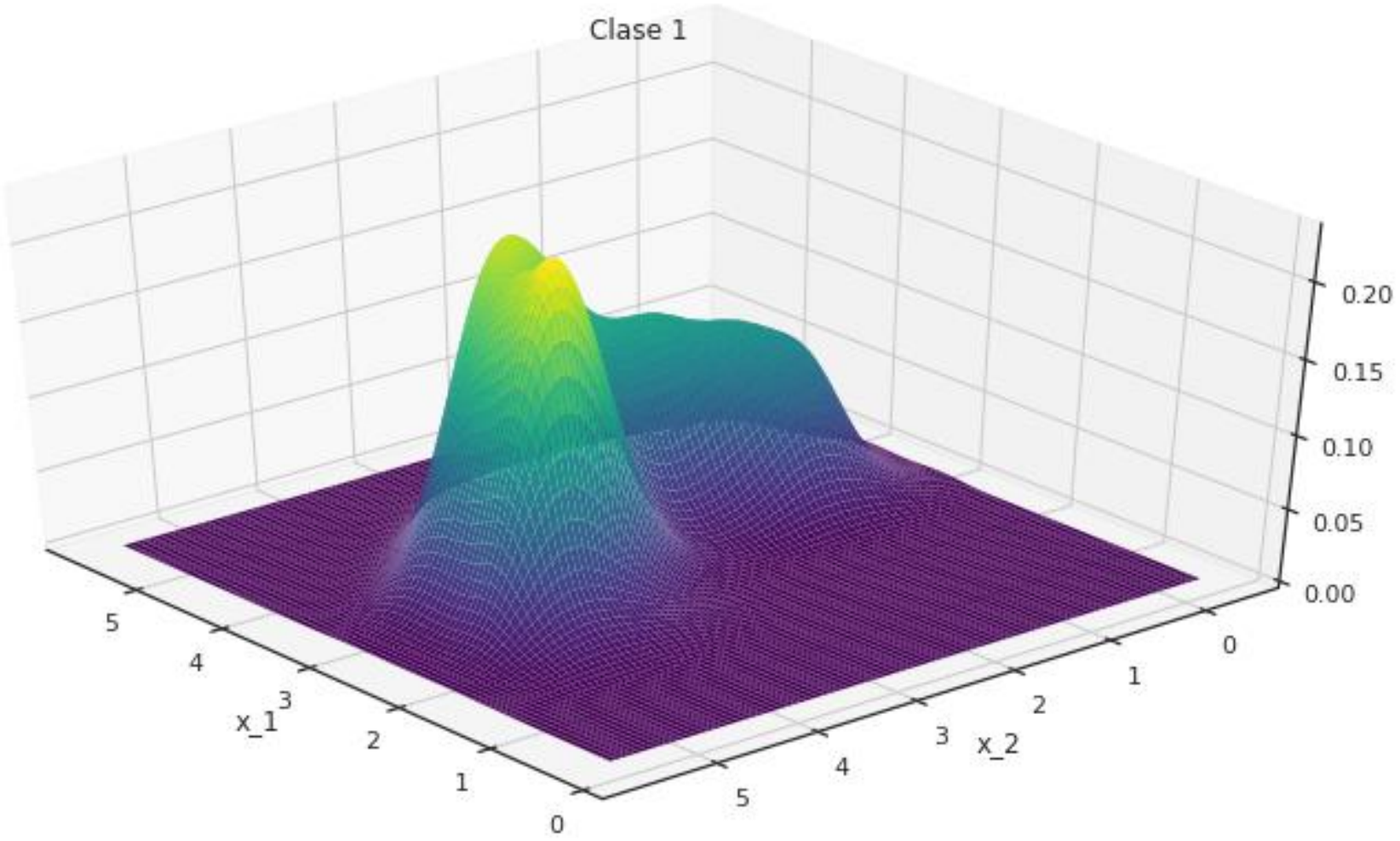
Muestras

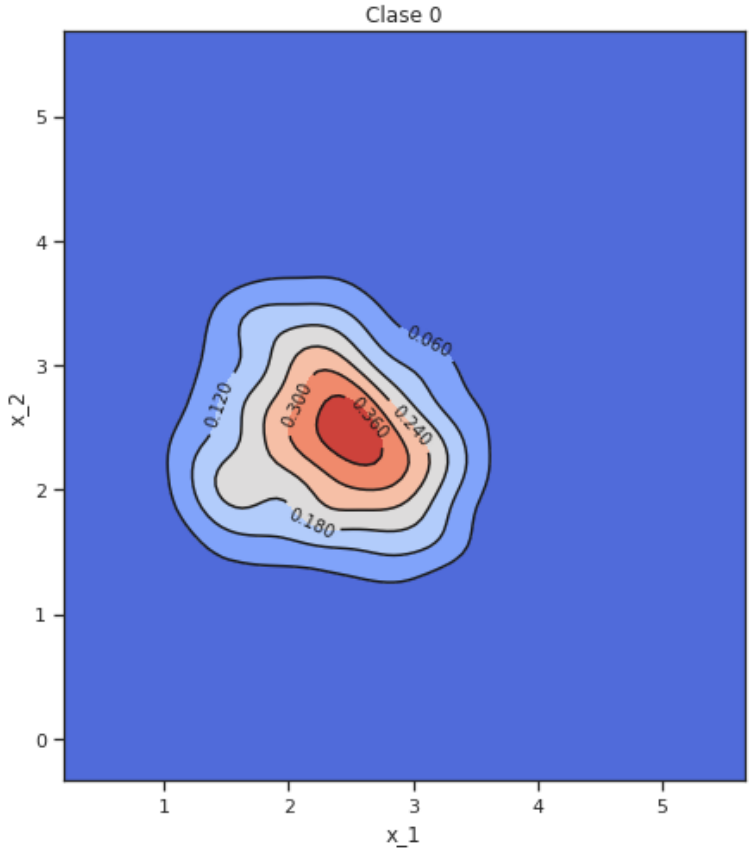


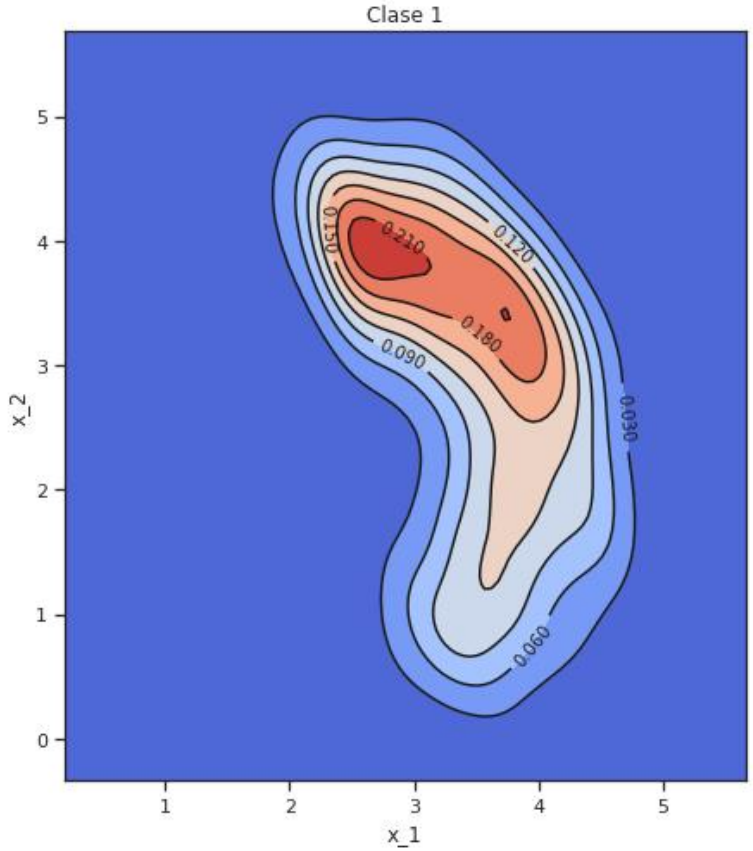
PDF

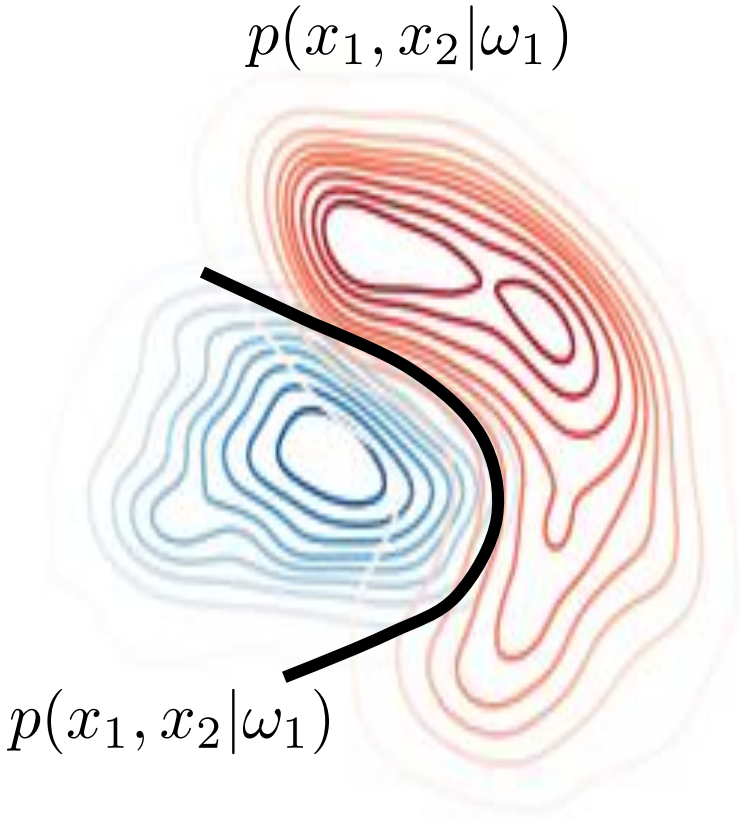

0
1
5
4
3
2
1
0
x_2
1
2
3
4
5
x_1

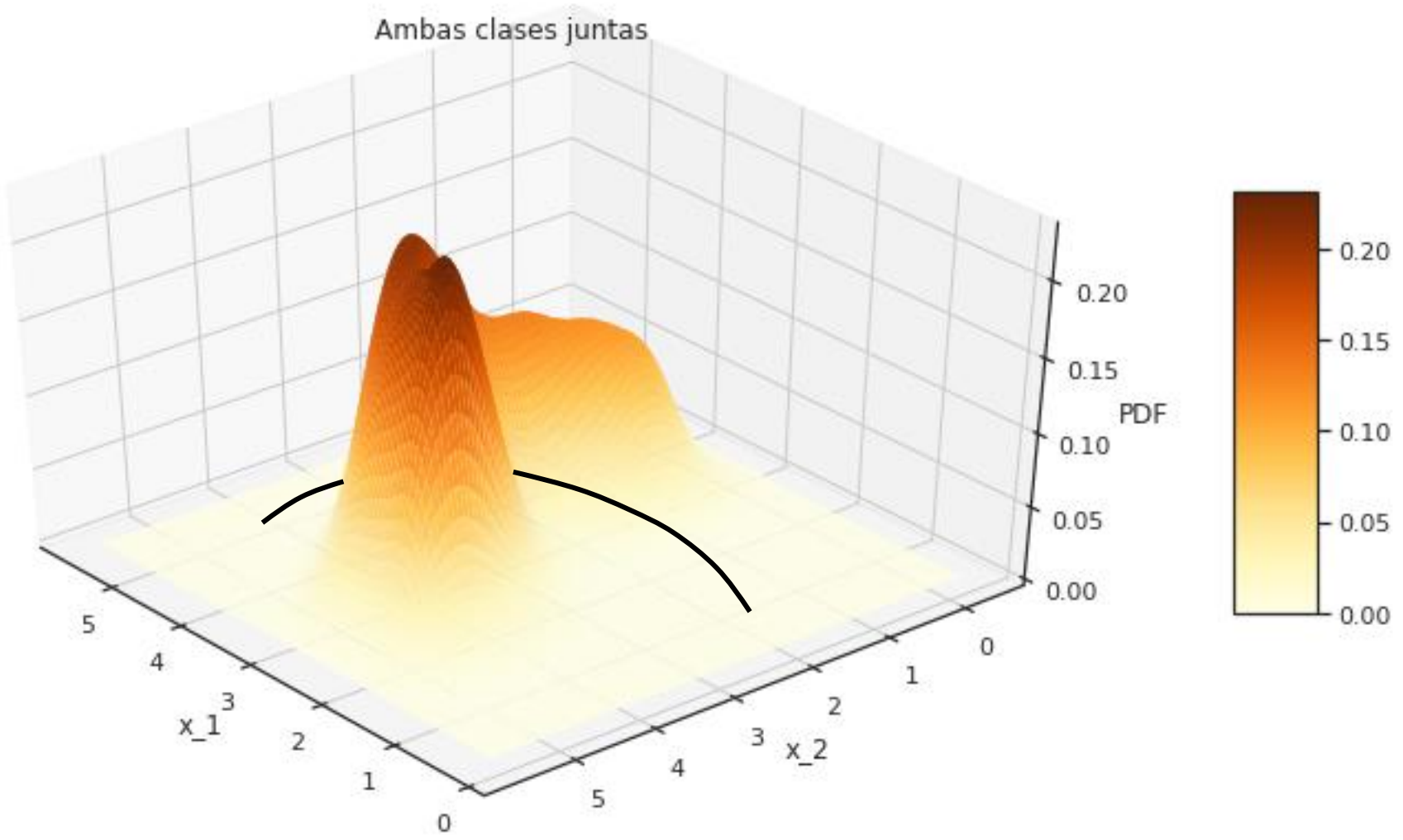

Clase 0
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
x_1
x_2
0
1
2
3
4
5


Clase 1
0.20
0.15
0.10
0.05
0.00
x_1
x_2
0
1
2
3
4
5


Clase 0
x_2
x_1
0.060
0.120
0.300
0.360
0.240
0.180


Clase 1
x_2
x_1
0.150
0.210
0.120
0.180
0.090
0.030
0.060


$p(x_1, x_2|\omega_1)$
$p(x_1, x_2|\omega_1)$

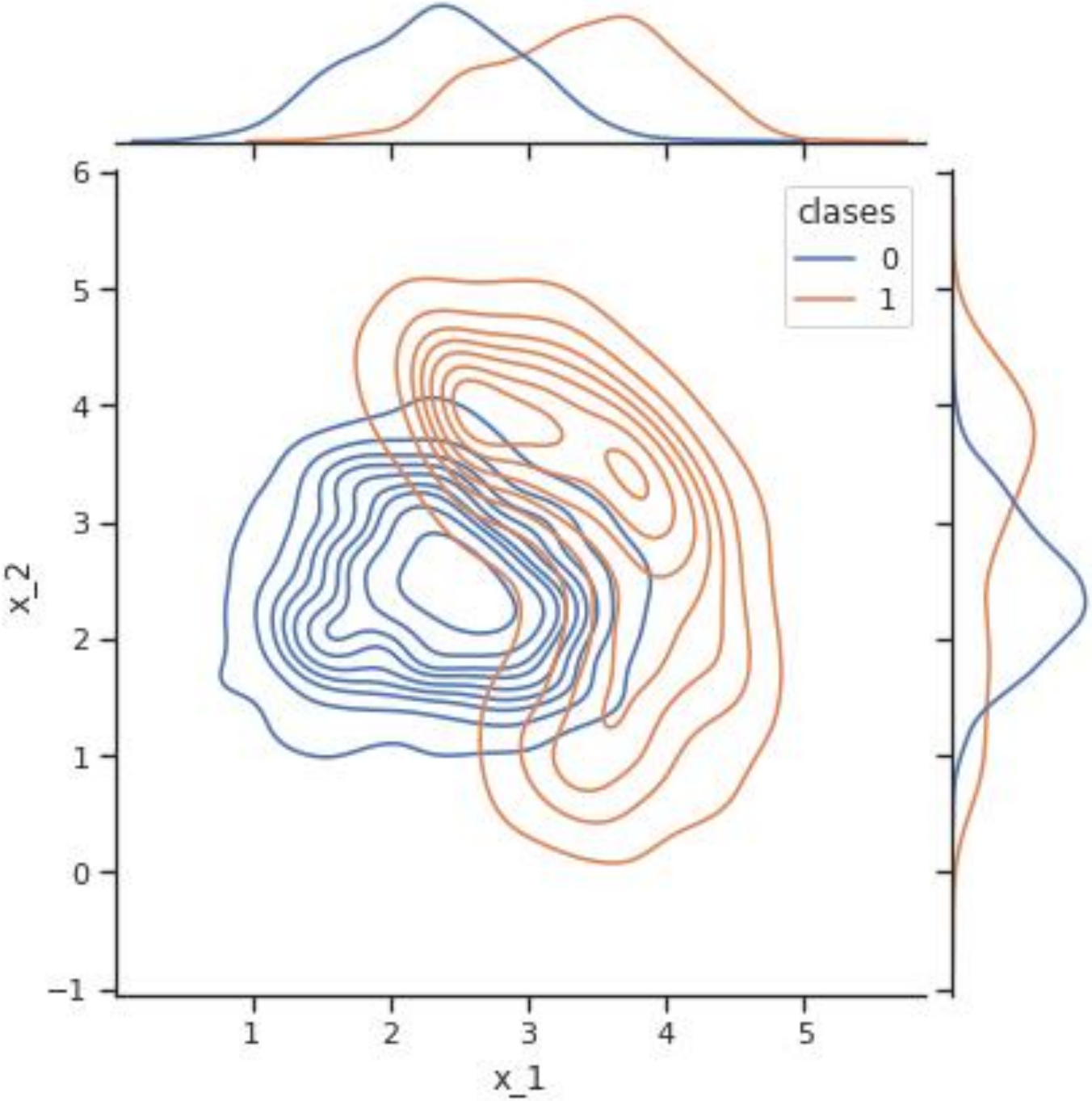

Ambas clases juntas
x_1
x_2
PDF
0.20
0.15
0.10
0.05
0.00

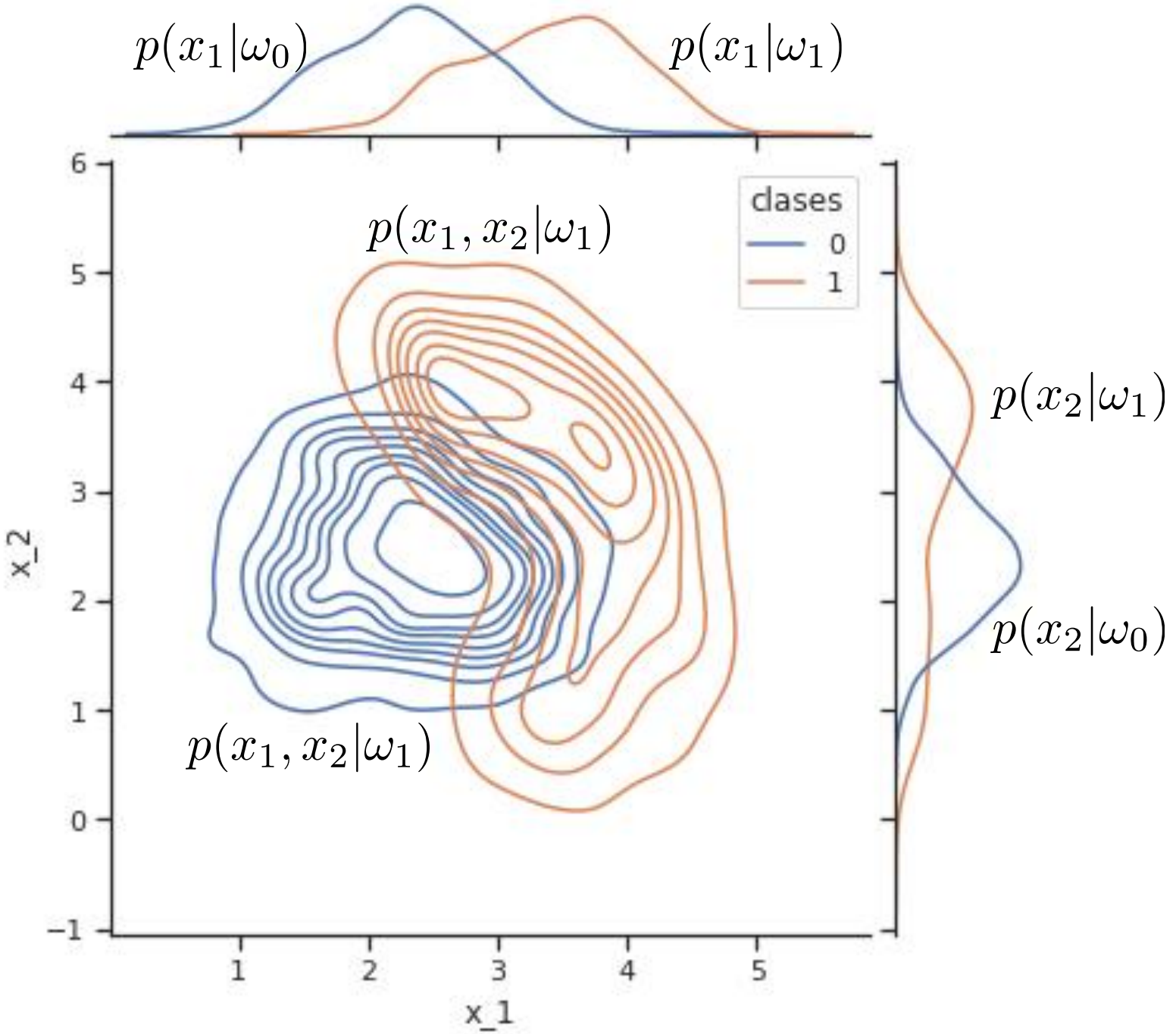
Naïve Bayes

$$p(x_1, \cdots, x_n|\omega_0) = p(x_1|\omega_0)p(x_2|\omega_0)\cdots p(x_n|\omega_0)$$

$$p(x_1, \cdots, x_n|\omega_1) = p(x_1|\omega_1)p(x_2|\omega_1)\cdots p(x_n|\omega_1)$$

Eventos independientes


clases
0
1
x_1
x_2
1
2
3
4
5
6
5
4
3
2
1
0
−1


$p(x_1|\omega_0)$
$p(x_1|\omega_1)$
clases
0
1
$p(x_1, x_2|\omega_1)$
$p(x_1, x_2|\omega_1)$
$p(x_2|\omega_1)$
$p(x_2|\omega_0)$
x_2
x_1
6
5
4
3
2
1
0
−1
1
2
3
4
5

Naïve Bayes

$$p(x_1, \cdots, x_n|\omega_0) = p(x_1|\omega_0)p(x_2|\omega_0)\cdots p(x_n|\omega_0)$$

$$p(x_1, \cdots, x_n|\omega_1) = p(x_1|\omega_1)p(x_2|\omega_1)\cdots p(x_n|\omega_1)$$

Eventos independientes

LDA, QDA, Mahalanobis

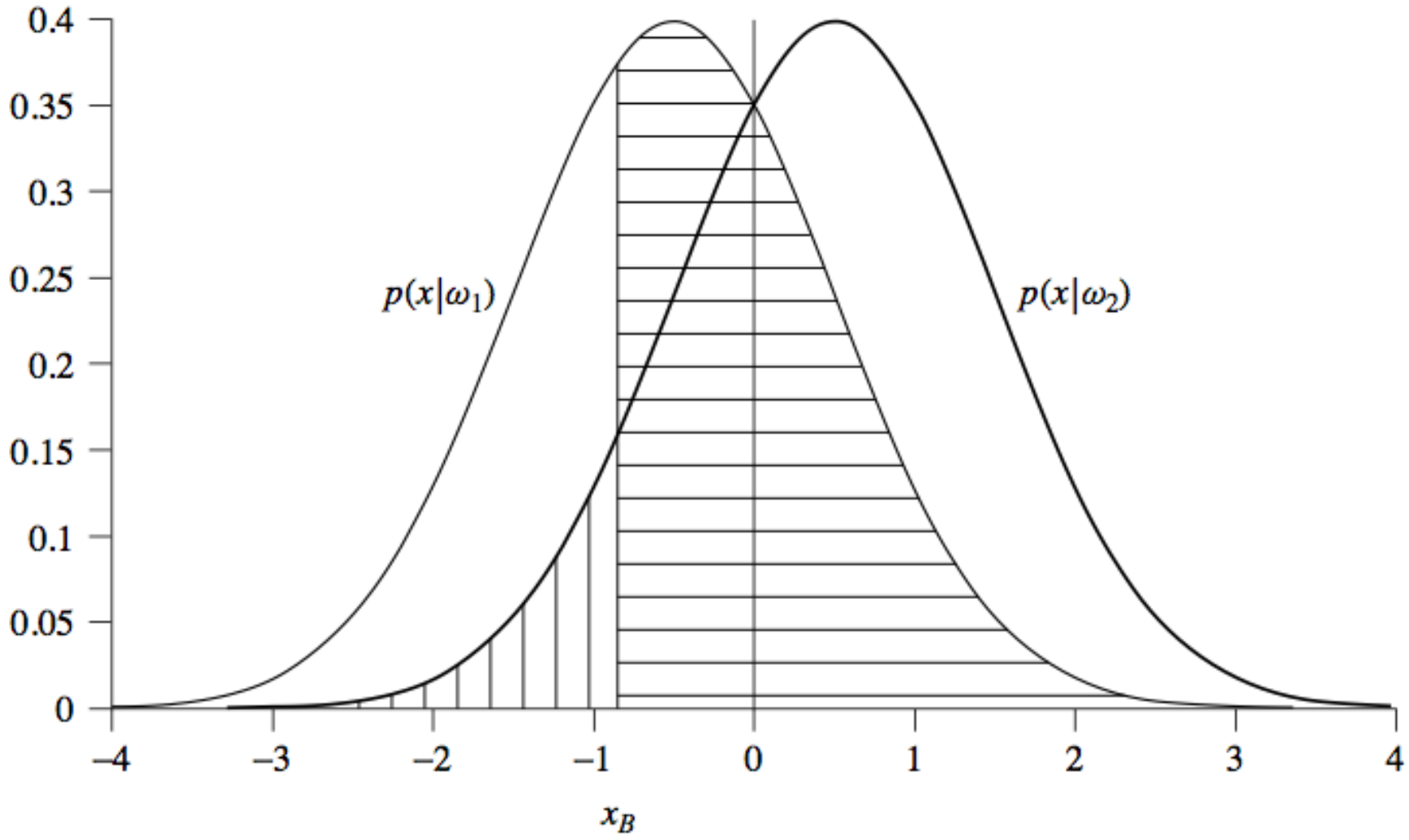
Clasificador de Bayes: $\boldsymbol{x}$ es clasificado como clase j si

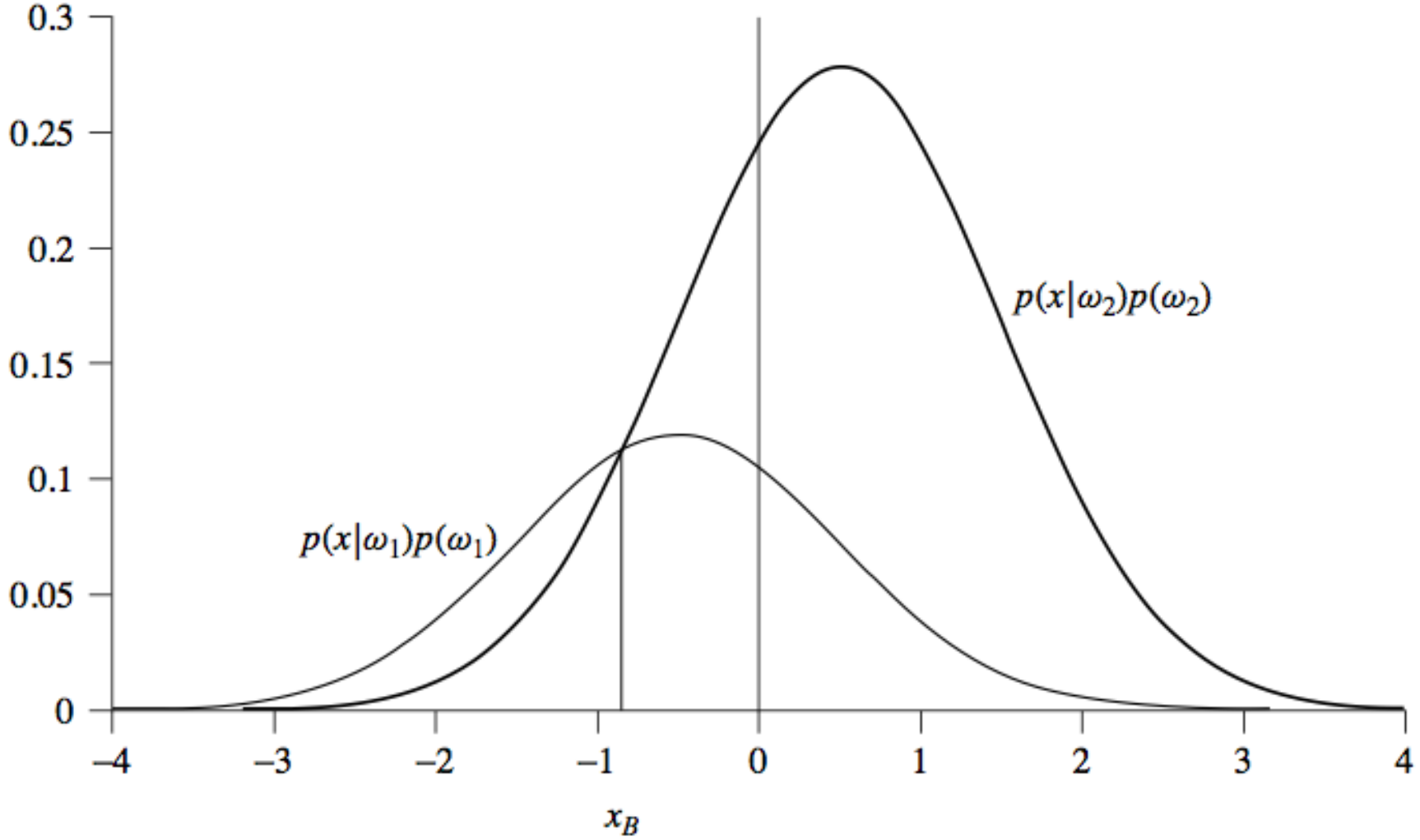
$$p(\omega_j|\boldsymbol{x}) > p(\omega_k|\boldsymbol{x}) \quad k = 1, \ldots, C; k \neq j$$

Usando el teorema de Bayes:

$$p(\omega_i|\boldsymbol{x}) = p(\omega_i)\frac{p(\boldsymbol{x}|\omega_i)}{p(\boldsymbol{x})}$$

$$p(\boldsymbol{x}|\omega_j)p(\omega_j) > p(\boldsymbol{x}|\omega_k)p(\omega_k) \quad k = 1, \ldots, C; k \neq j$$


0.4
0.35
0.3
0.25
0.2
0.15
0.1
0.05
0
−4
−3
−2
−1
0
1
2
3
4
$p(x|\omega_1)$
$p(x|\omega_2)$
x_B


0.3
0.25
0.2
0.15
0.1
0.05
0
−4
−3
−2
−1
0
1
2
3
4
x_B
$p(x|\omega_2)p(\omega_2)$
$p(x|\omega_1)p(\omega_1)$

Para distribuciones Gaussianas

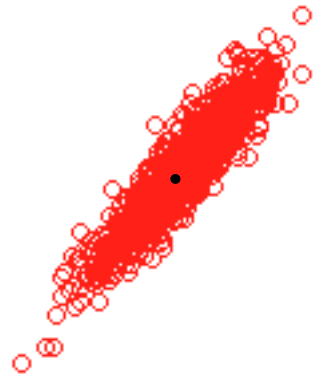
$$p(\boldsymbol{x}|\omega_i) = \frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}_i|^{1/2}} \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i)\right\}$$

Estimador de Matriz de Covarianza: $$\hat{\boldsymbol{\Sigma}} = \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{x}_i - \boldsymbol{m})(\boldsymbol{x}_i - \boldsymbol{m})^T$$

Ejemplos de Σ y μ en 2D:







6.0057	-0.1020
-0.1020	1.0632

7.7947	-0.7772
-0.7772	43.1459

15.1951	21.8267
21.8267	37.6734

Clasificador de Bayes

$$p(\omega_j|\boldsymbol{x}) > p(\omega_k|\boldsymbol{x}) \quad k = 1, \ldots, C; k \neq j$$

$$p(\boldsymbol{x}|\omega_j)p(\omega_j) > p(\boldsymbol{x}|\omega_k)p(\omega_k) \quad k = 1, \ldots, C; k \neq j$$

$$\log\{p(\boldsymbol{x}|\omega_j)p(\omega_j)\} > \log\{p(\boldsymbol{x}|\omega_k)p(\omega_k)\} \quad k = 1, \ldots, C; k \neq j$$

Para distribuciones Gaussianas:

$$p(\boldsymbol{x}|\omega_i) = \frac{1}{(2\pi)^{p/2}|\boldsymbol{\Sigma}_i|^{1/2}} \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i)\right\}$$

$$\begin{aligned}\log(p(\boldsymbol{x}|\omega_i)\,p(\omega_i)) &= \log(p(\boldsymbol{x}|\omega_i)) + \log(p(\omega_i)) \\ &= -\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i) - \frac{1}{2}\log(|\boldsymbol{\Sigma}_i|) \\ &\quad - \frac{p}{2}\log(2\pi) + \log(p(\omega_i))\end{aligned}$$

$$\log\{p(\boldsymbol{x}|\omega_j)p(\omega_j)\} > \log\{p(\boldsymbol{x}|\omega_k)p(\omega_k)\} \quad k = 1, \ldots, C; k \neq j$$

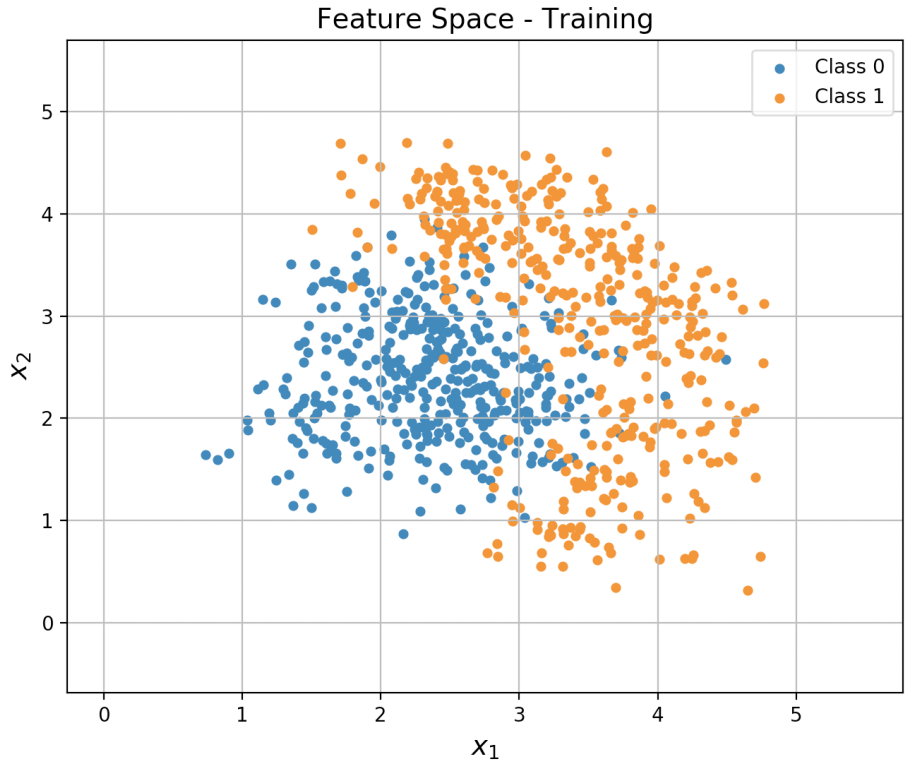
$$\begin{aligned}
\log(p(\boldsymbol{x}|\omega_i)\, p(\omega_i)) &= \log(p(\boldsymbol{x}|\omega_i)) + \log(p(\omega_i)) \\
&= -\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_i) - \frac{1}{2}\log(|\boldsymbol{\Sigma}_i|) \\
&\quad - \frac{p}{2}\log(2\pi) + \log(p(\omega_i))
\end{aligned}$$

Este término es constante, se puede eliminar de la desigualdad.

Ejemplo

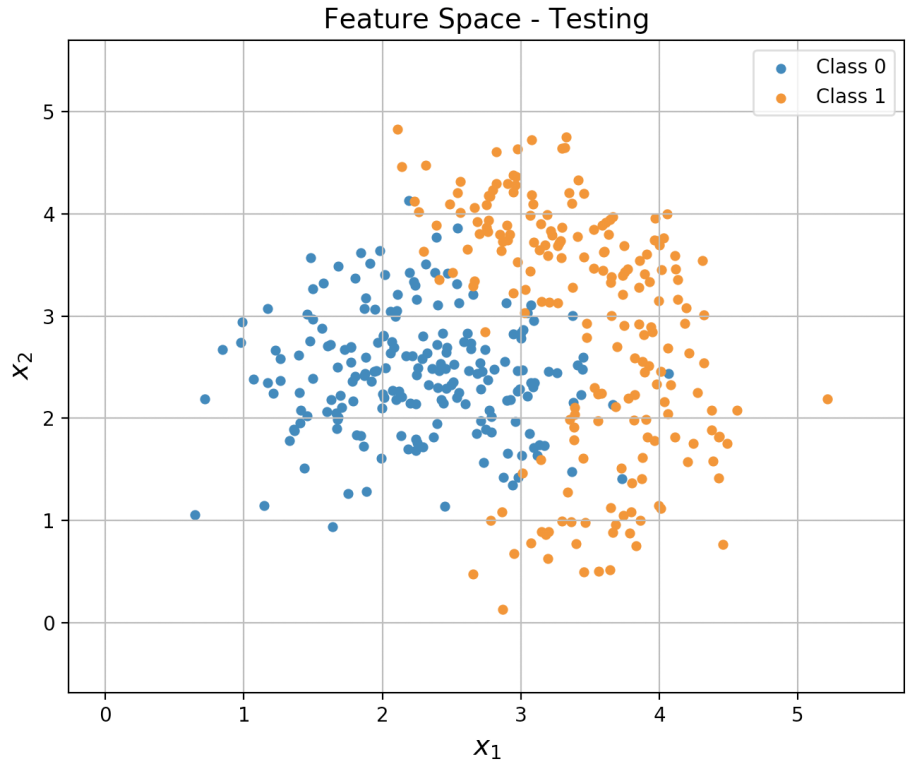
(X,d)

TRAINING



(X_t,d_t)

TESTING



LDA

$$\log\{p(\boldsymbol{x}|\omega_j)p(\omega_j)\} > \log\{p(\boldsymbol{x}|\omega_k)p(\omega_k)\} \quad k = 1, \ldots, C; k \neq j$$

$$\begin{aligned}\log(p(\boldsymbol{x}|\omega_i)\,p(\omega_i)) &= \log(p(\boldsymbol{x}|\omega_i)) + \log(p(\omega_i)) \\ &= -\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^T\boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i) - \frac{1}{2}\log(|\boldsymbol{\Sigma}_i|) \\ &\quad - \frac{p}{2}\log(2\pi) + \log(p(\omega_i))\end{aligned}$$

En LDA (Análisis Discriminante Lineal) se supone que $\Sigma_i = \Sigma$ (es constante) .

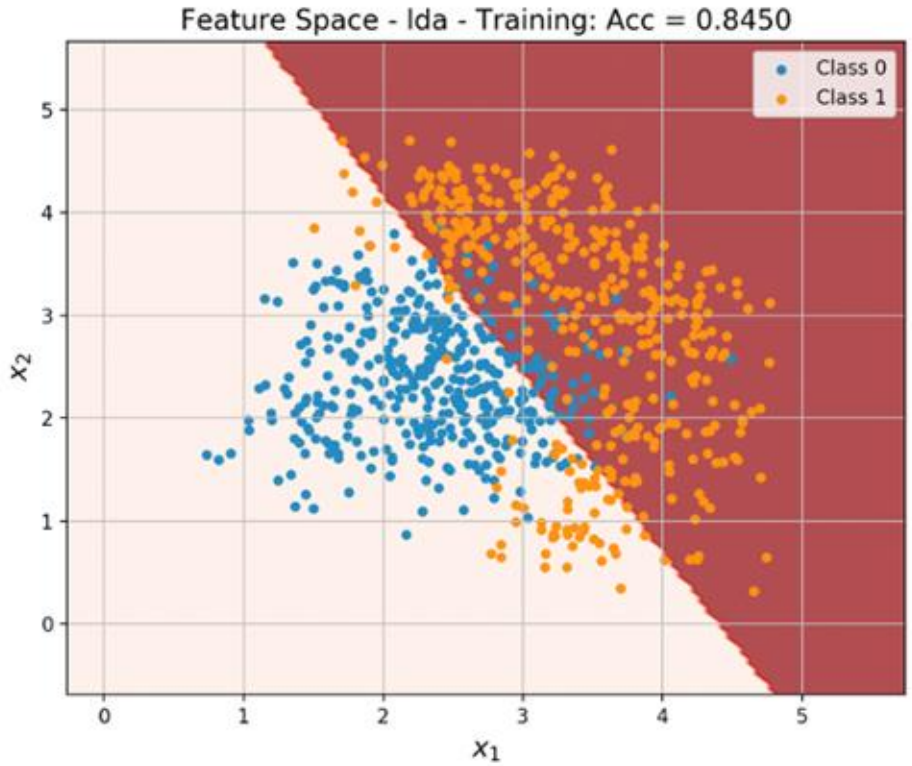
Σ se calcula a partir de datos de entrenamiento. Una buena estimación es el promedio de las matrices de covarianza individuales: $\Sigma = (\Sigma_1+\Sigma_2)/2$

Ejemplo

LDA

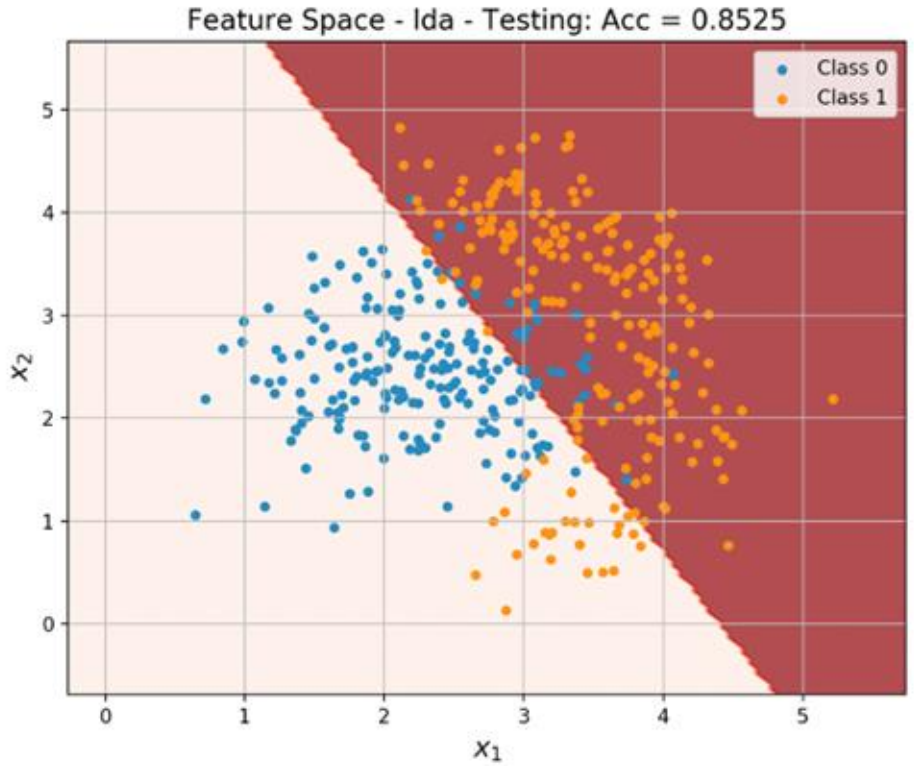
(X,d)

TRAINING



(X_t, d_t)

TESTING



MAHALANOBIS

$$\log\{p(\boldsymbol{x}|\omega_j)p(\omega_j)\} > \log\{p(\boldsymbol{x}|\omega_k)p(\omega_k)\} \quad k = 1, \ldots, C; k \neq j$$

$$\begin{aligned}\log(p(\boldsymbol{x}|\omega_i)\,p(\omega_i)) &= \log(p(\boldsymbol{x}|\omega_i)) + \log(p(\omega_i)) \\ &= -\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^T\boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i) - \frac{1}{2}\log(|\boldsymbol{\Sigma}_i|) \\ &\quad - \frac{p}{2}\log(2\pi) + \log(p(\omega_i))\end{aligned}$$

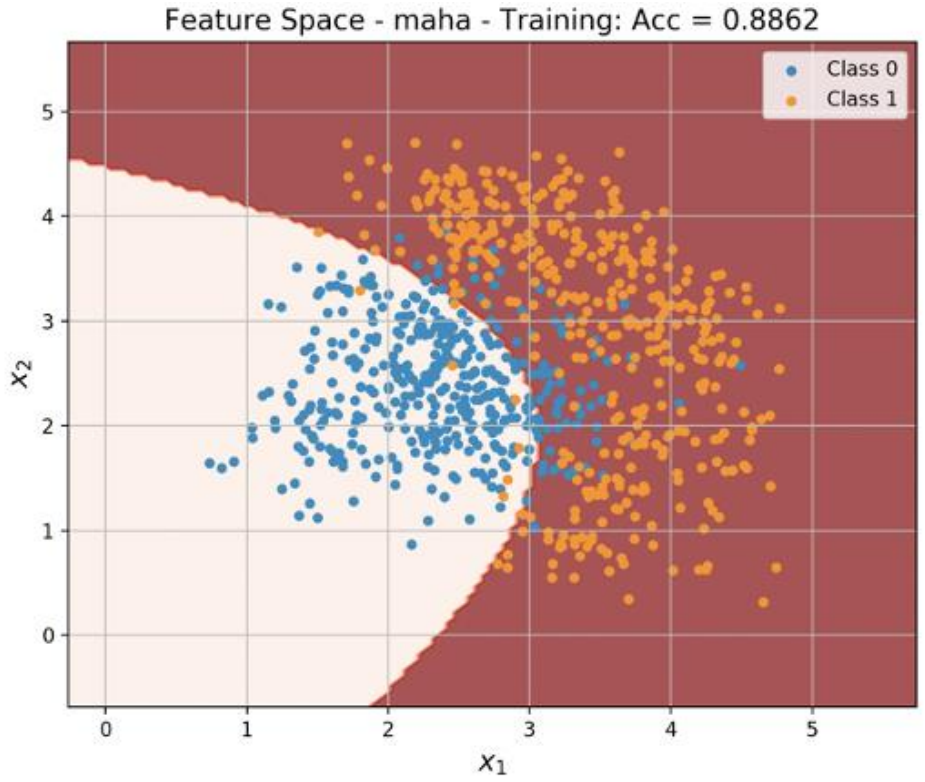
En el Clasificador Mahalanobis se asume $p(w_i) = p$
y las matrices Σ_i son distintas.

Ejemplo

Mahalanobis

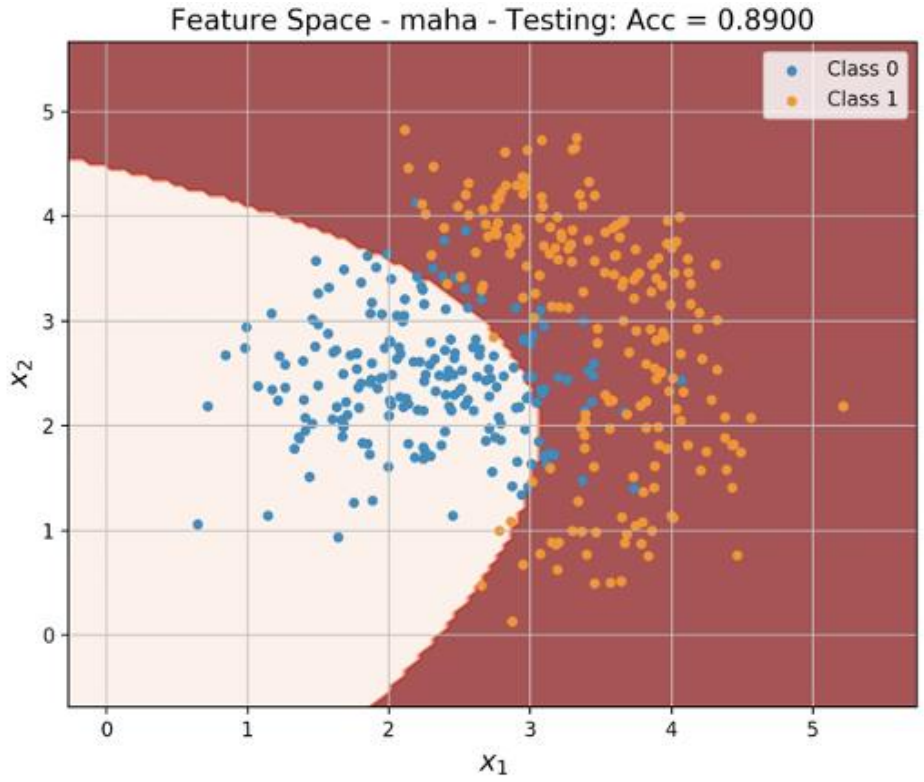
(X,d)

TRAINING



(X_t, d_t)

TESTING



MAHALANOBIS-0

$$\log\{p(\boldsymbol{x}|\omega_j)p(\omega_j)\} > \log\{p(\boldsymbol{x}|\omega_k)p(\omega_k)\} \quad k = 1, \ldots, C; k \neq j$$

$$\log(p(\boldsymbol{x}|\omega_i)\, p(\omega_i)) = \log(p(\boldsymbol{x}|\omega_i)) + \log(p(\omega_i))$$

$$= -\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_i) - \frac{1}{2}\log(|\boldsymbol{\Sigma}_i|)$$

$$- \frac{p}{2}\log(2\pi) + \log(p(\omega_i))$$

En el Clasificador Mahalanobis se asume $p(w_i) = p$.

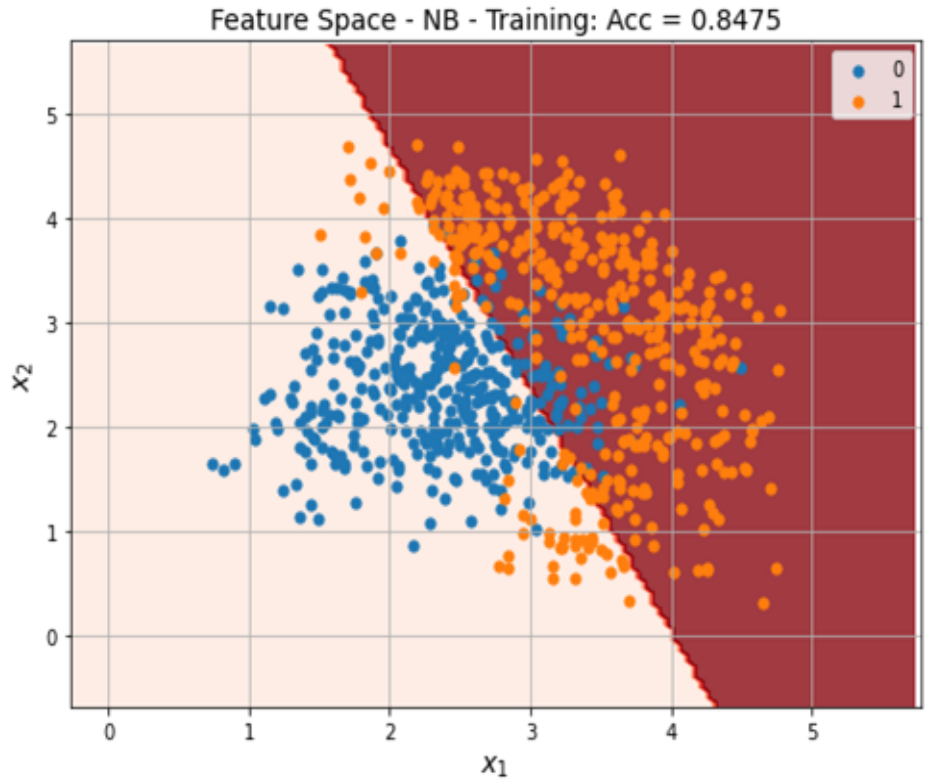
Hay una variante de Mahalanobis en la que se supone que $\Sigma_i = \Sigma$.

Ejemplo

Mahalanobis-0 ($\Sigma_i = \Sigma$)

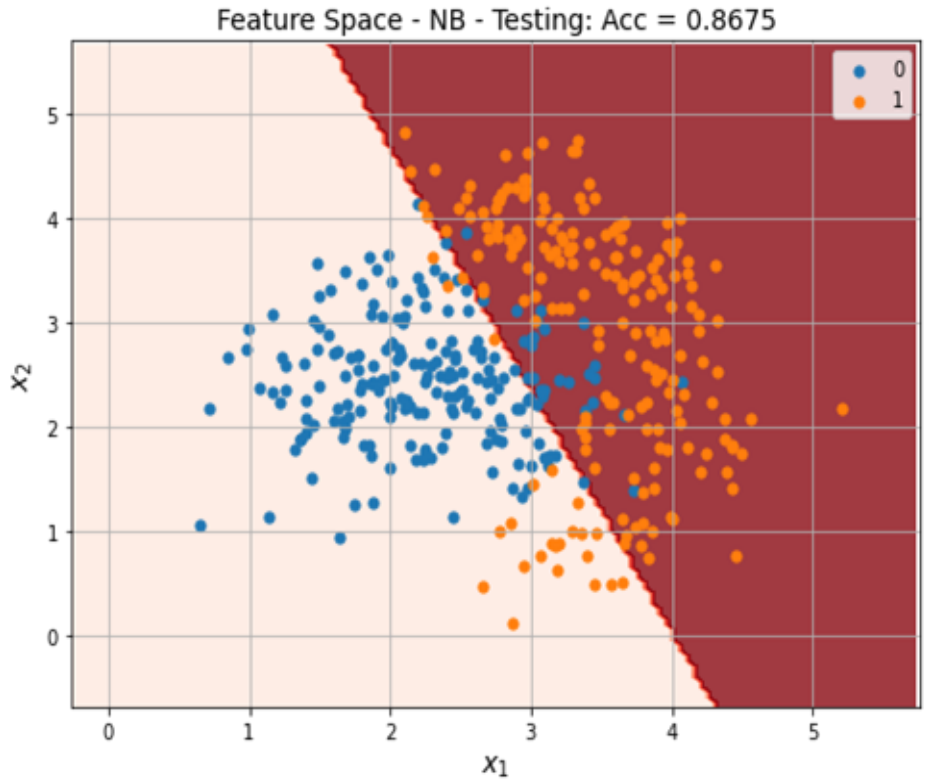
(X,d)

TRAINING



(X_t,d_t)

TESTING



QDA

$$\log\{p(\boldsymbol{x}|\omega_j)p(\omega_j)\} > \log\{p(\boldsymbol{x}|\omega_k)p(\omega_k)\} \quad k = 1, \ldots, C; k \neq j$$

$$\begin{aligned}
\log(p(\boldsymbol{x}|\omega_i)\, p(\omega_i)) &= \log(p(\boldsymbol{x}|\omega_i)) + \log(p(\omega_i)) \\
&= -\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_i) - \frac{1}{2}\log(|\boldsymbol{\Sigma}_i|) \\
&\quad - \frac{p}{2}\log(2\pi) + \log(p(\omega_i))
\end{aligned}$$

En QDA (Análisis Discriminante Cuadrático) se supone que Σ_i y $p(w_i)$ son diferentes.

Ejemplo

QDA

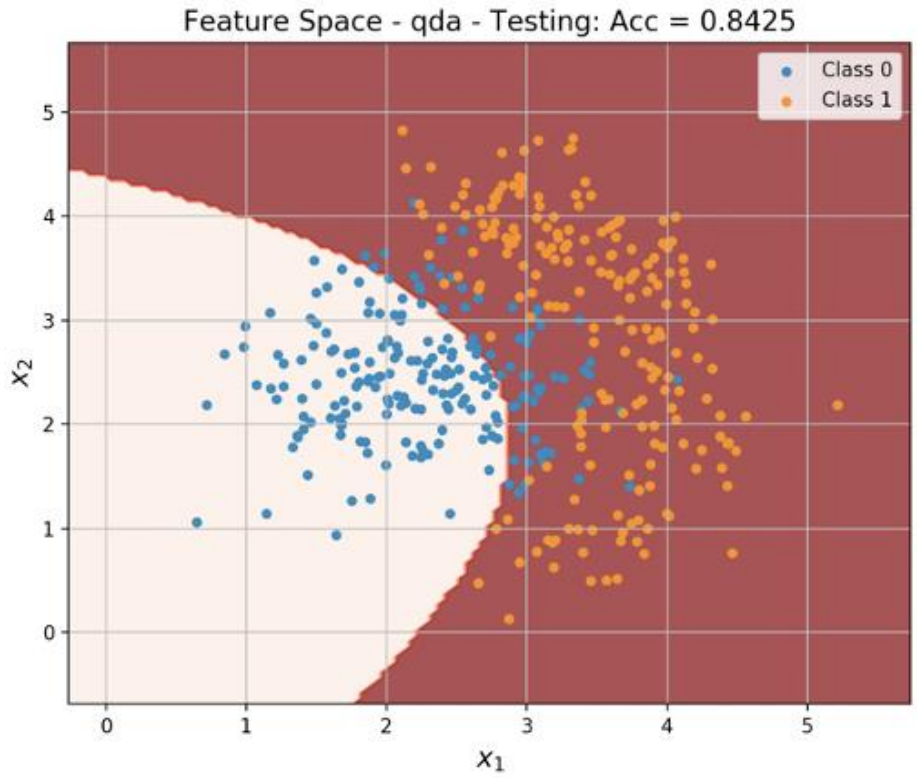
(X,d)

TRAINING



(X_t, d_t)

TESTING



$$\log(p(\boldsymbol{x}|\omega_i) = -\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^T\boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i) - \frac{1}{2}\log(|\boldsymbol{\Sigma}_i|) - \frac{p}{2}\log(2\pi) + \log(p(\omega_i))$$

$\Sigma_i = \Sigma = cte$ cte

LDA $$= -\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^T\boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i) - \frac{1}{2}\log(|\boldsymbol{\Sigma}_i|) - \frac{p}{2}\log(2\pi) + \log(p(\omega_i))$$

$\Sigma_i = \Sigma = cte$ cte $p(w_i) = cte$

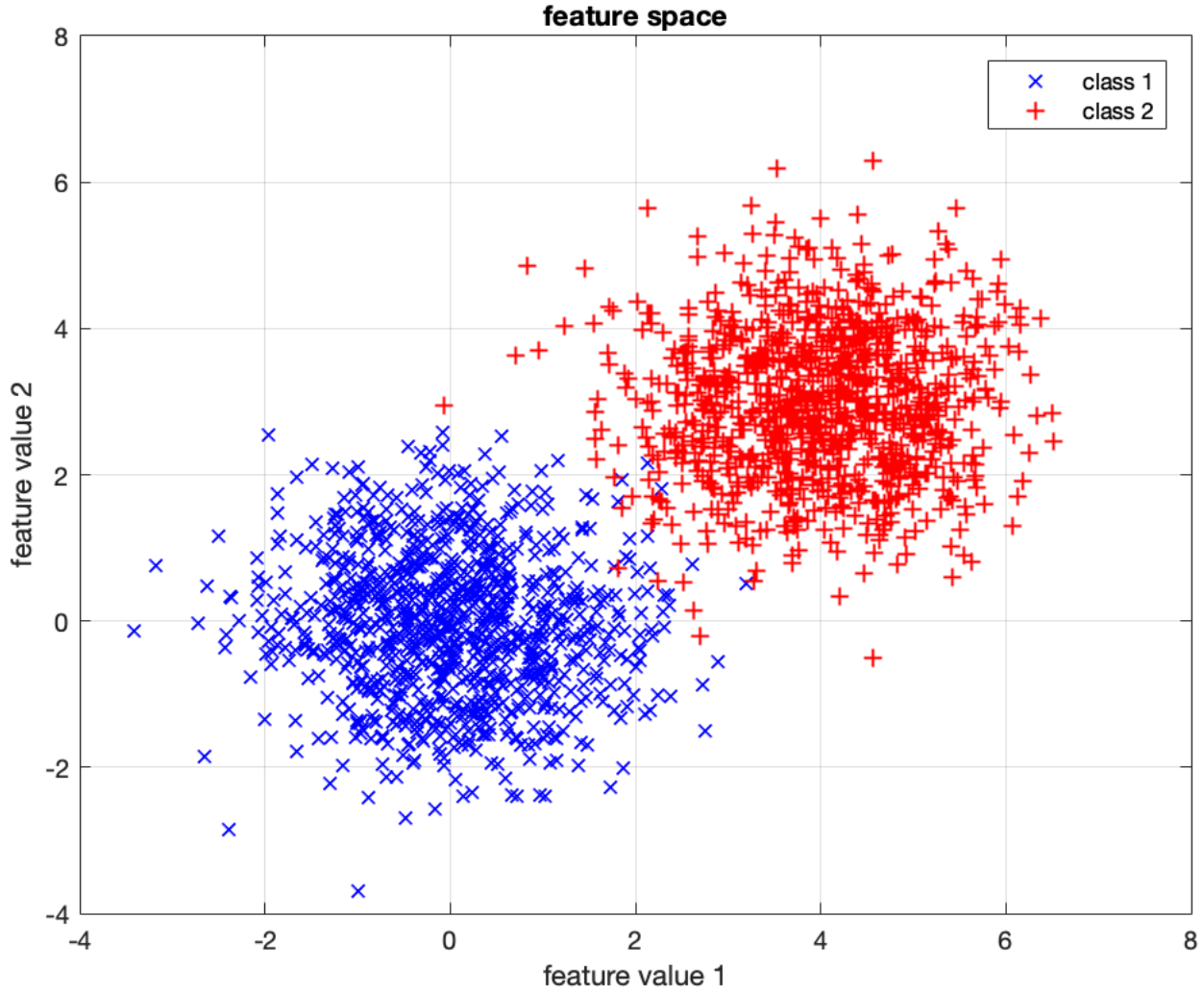
Mahalanobis $$= -\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^T\boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i) - \frac{1}{2}\log(|\boldsymbol{\Sigma}_i|) - \frac{p}{2}\log(2\pi) + \log(p(\omega_i))$$

cte

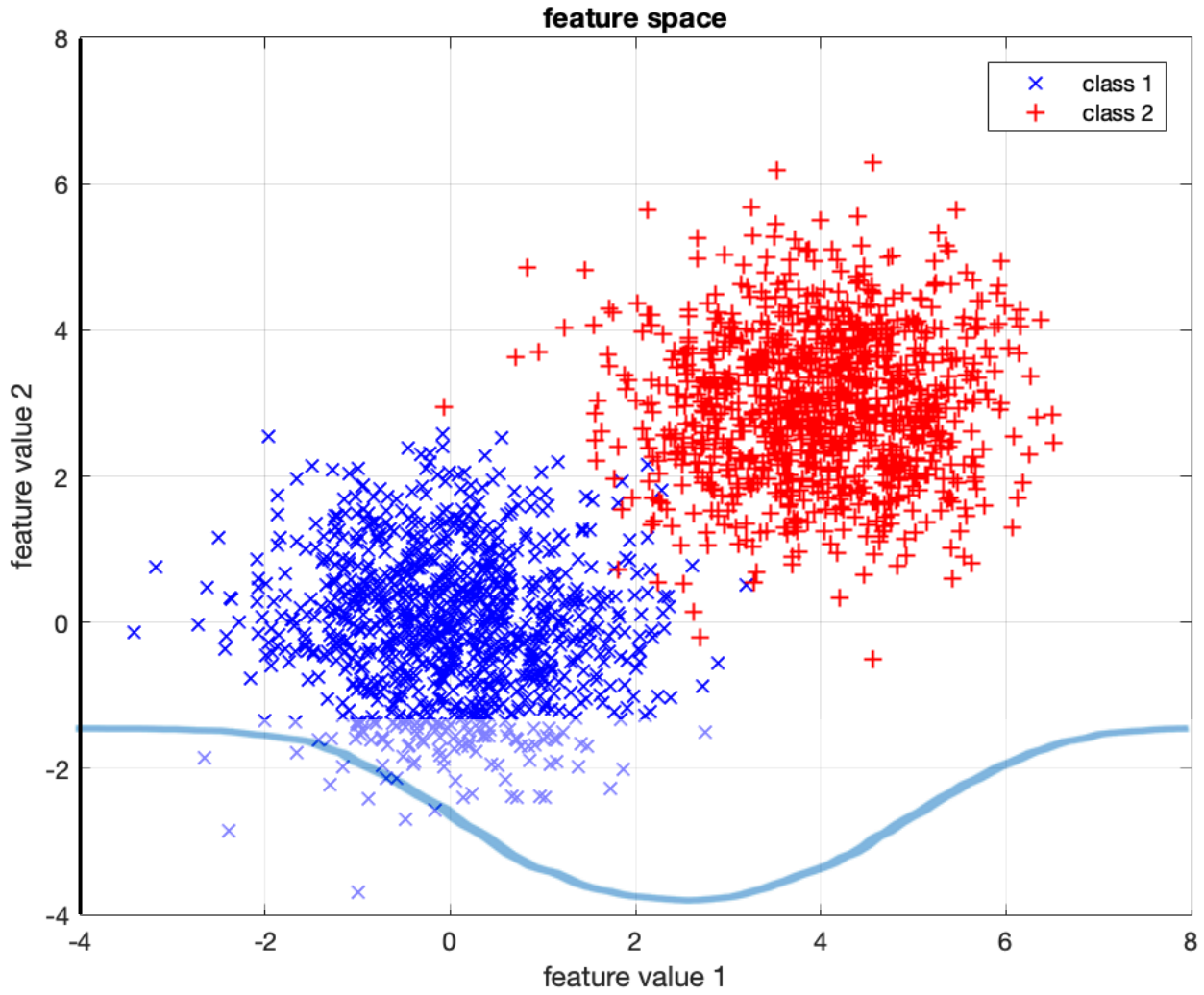
QDA $$= -\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_i)^T\boldsymbol{\Sigma}_i^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_i) - \frac{1}{2}\log(|\boldsymbol{\Sigma}_i|) - \frac{p}{2}\log(2\pi) + \log(p(\omega_i))$$

Árboles de Decisión

Árboles de Decisión (Training data)

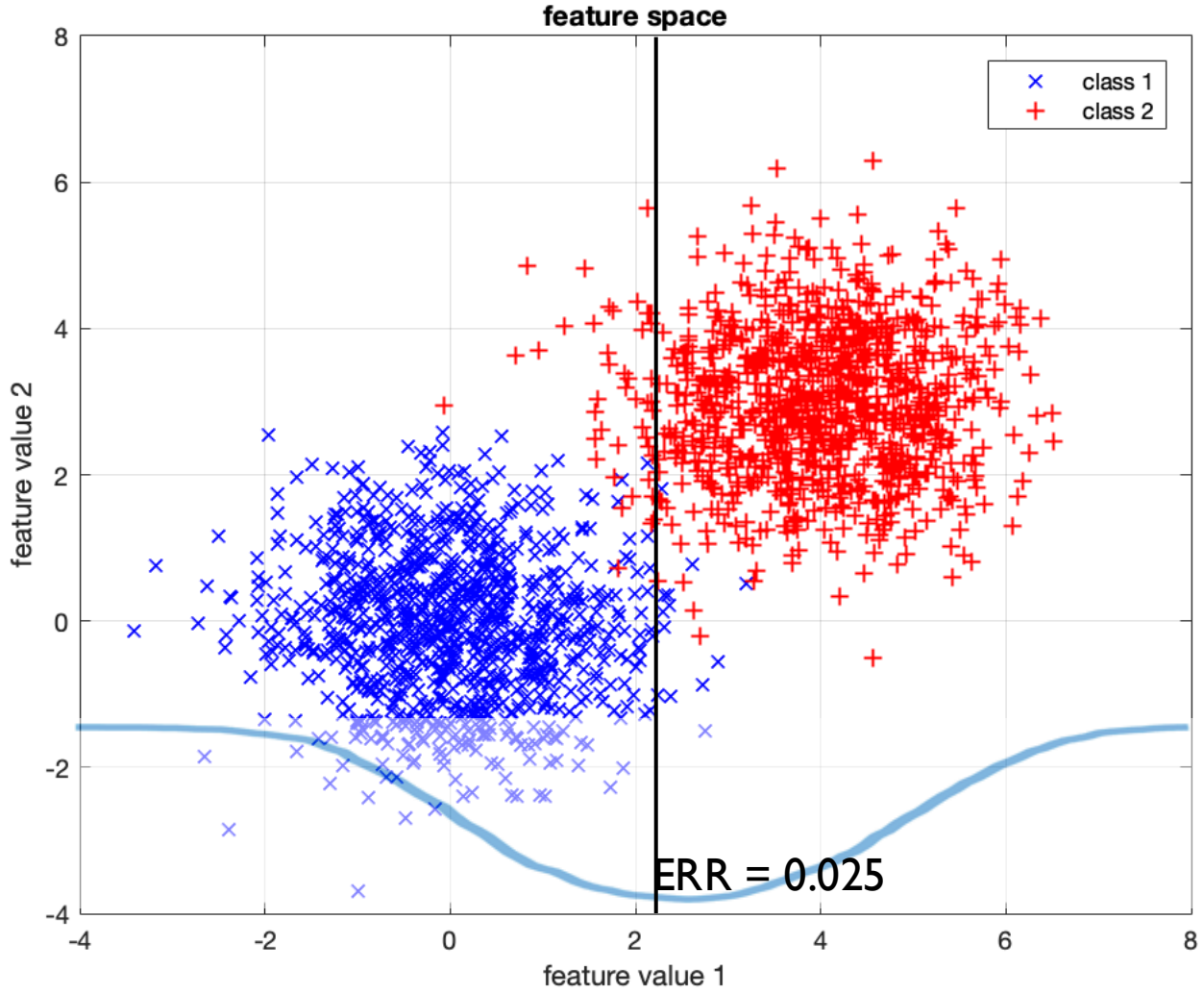


Árboles de Decisión (Training data)



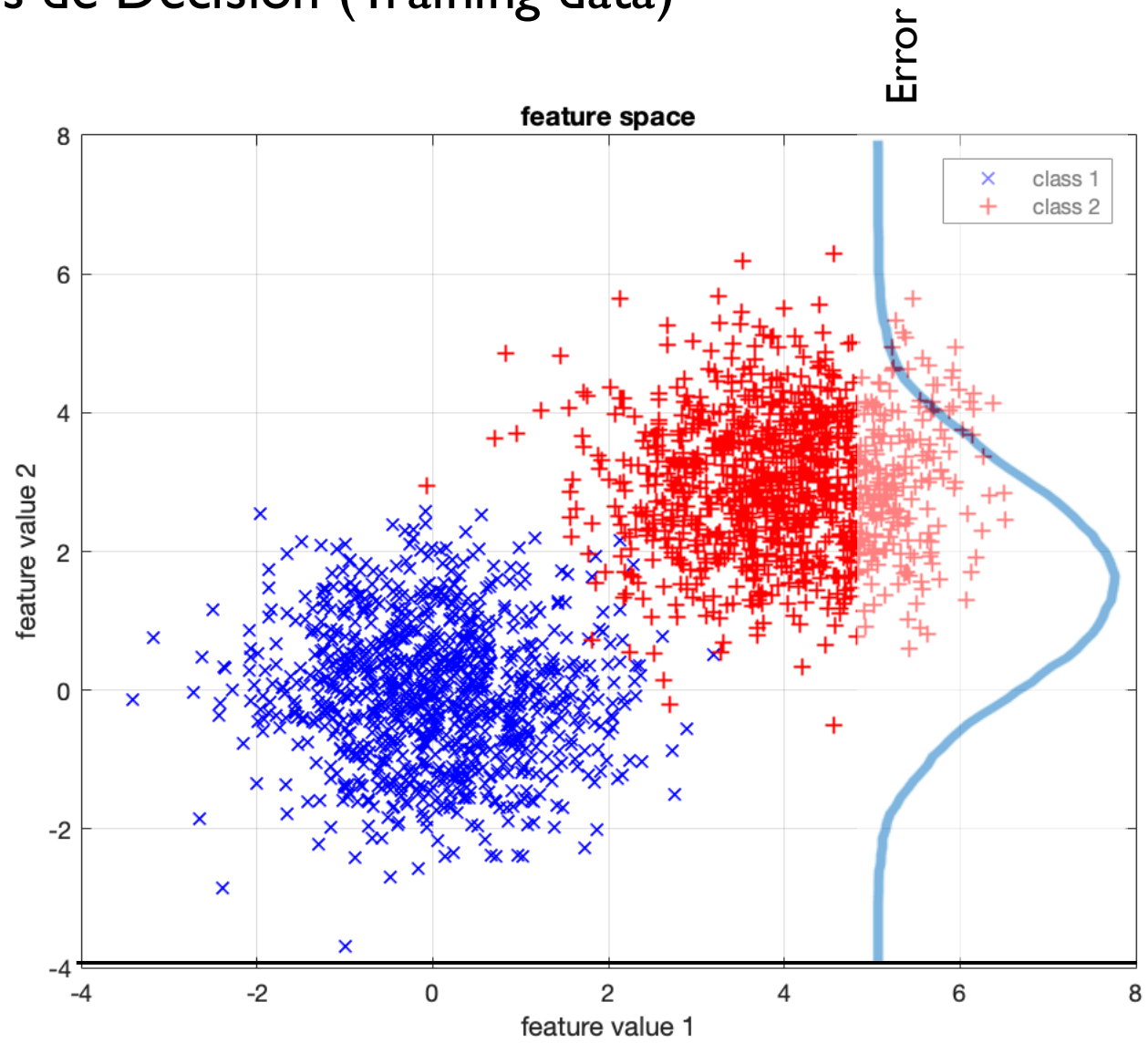
Error

Árboles de Decisión (Training data)

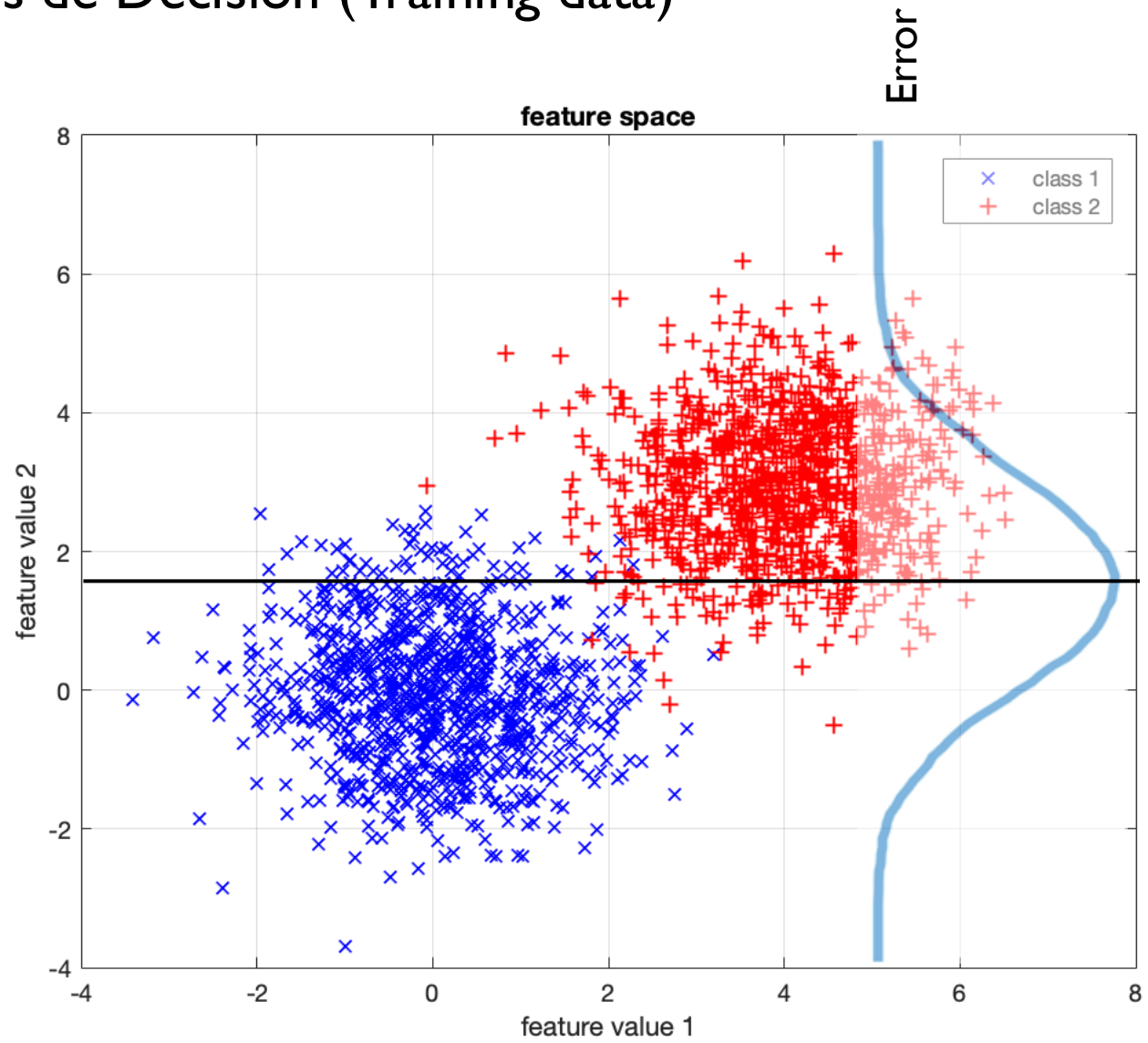


Error

Árboles de Decisión (Training data)

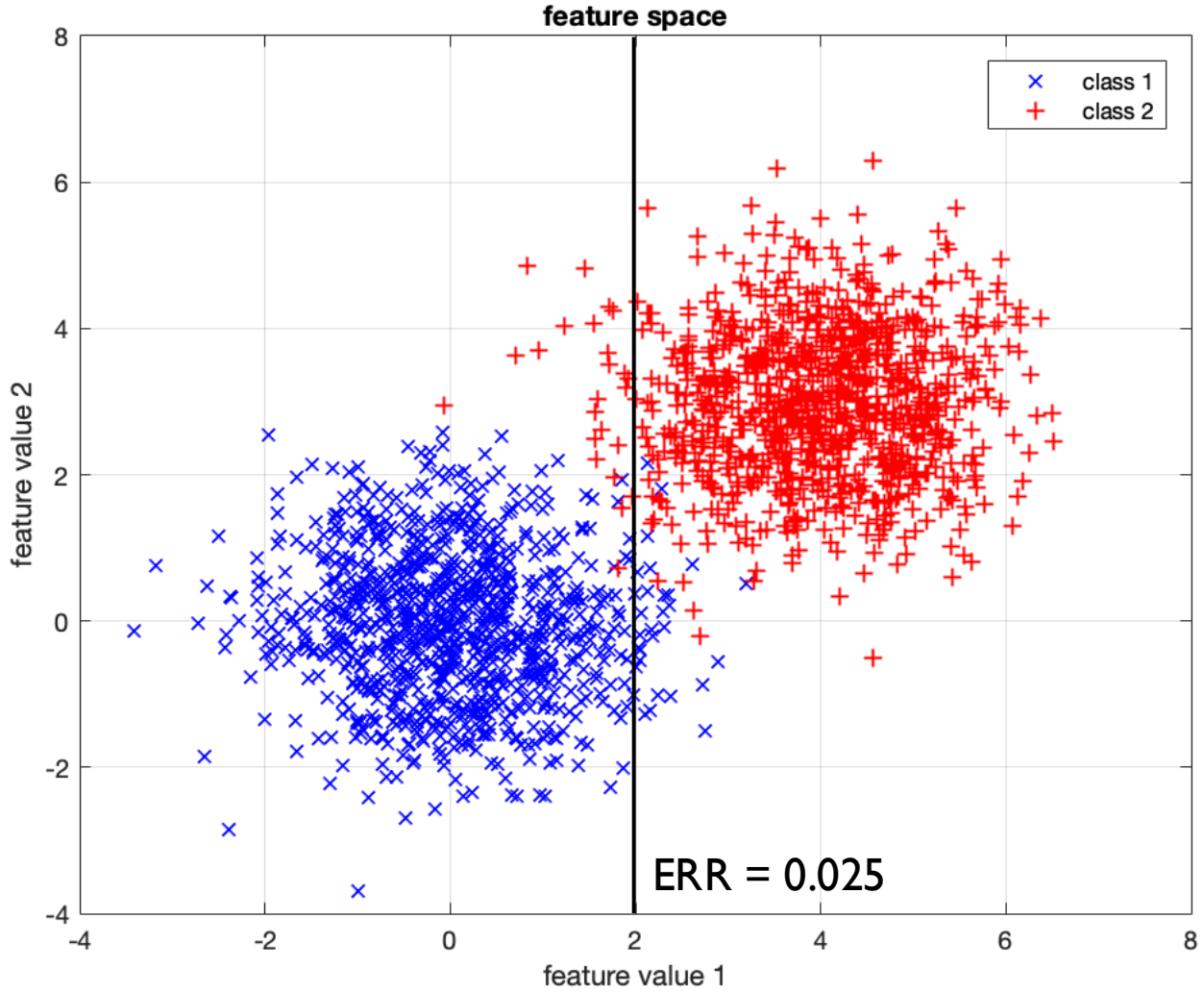


Árboles de Decisión (Training data)

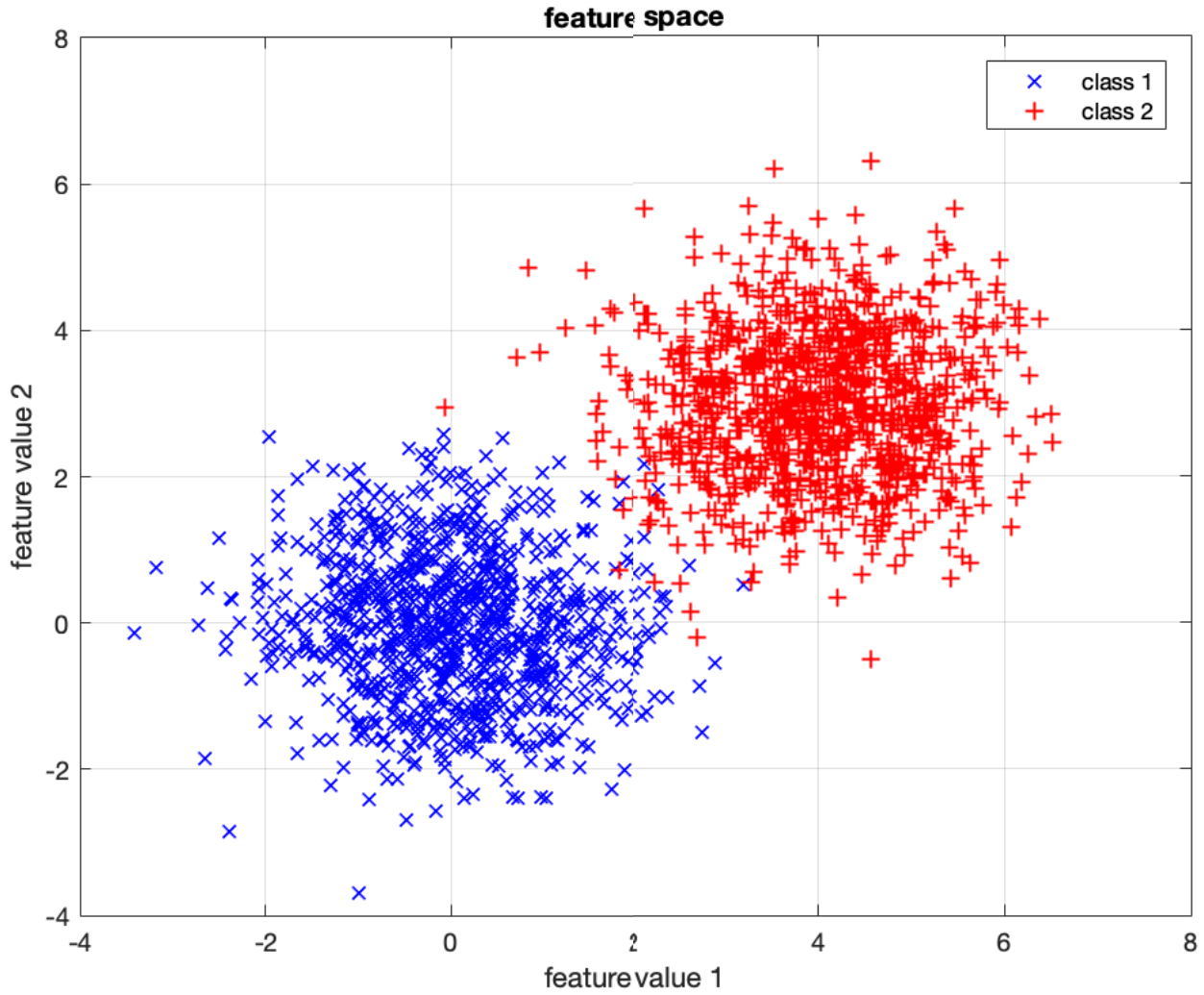


ERR = 0.066

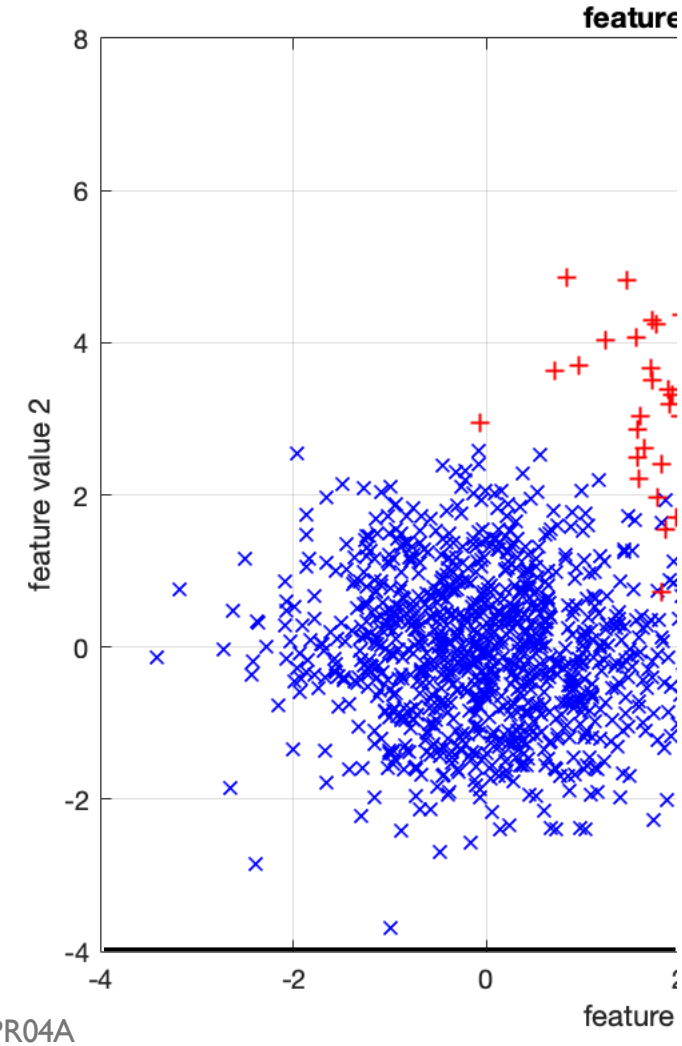
Árboles de Decisión (Training data)

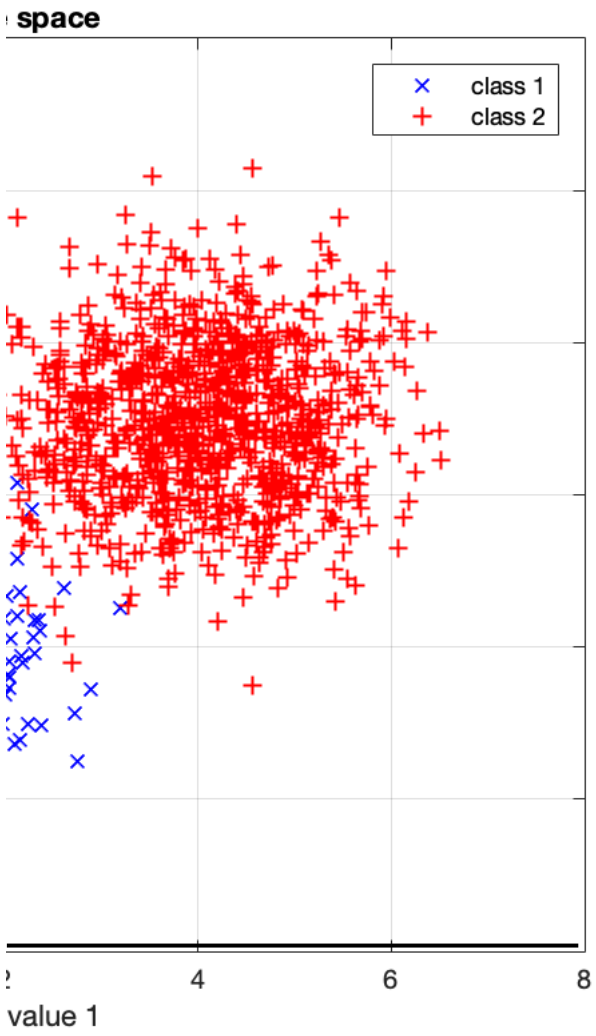


Árboles de Decisión (Training data)

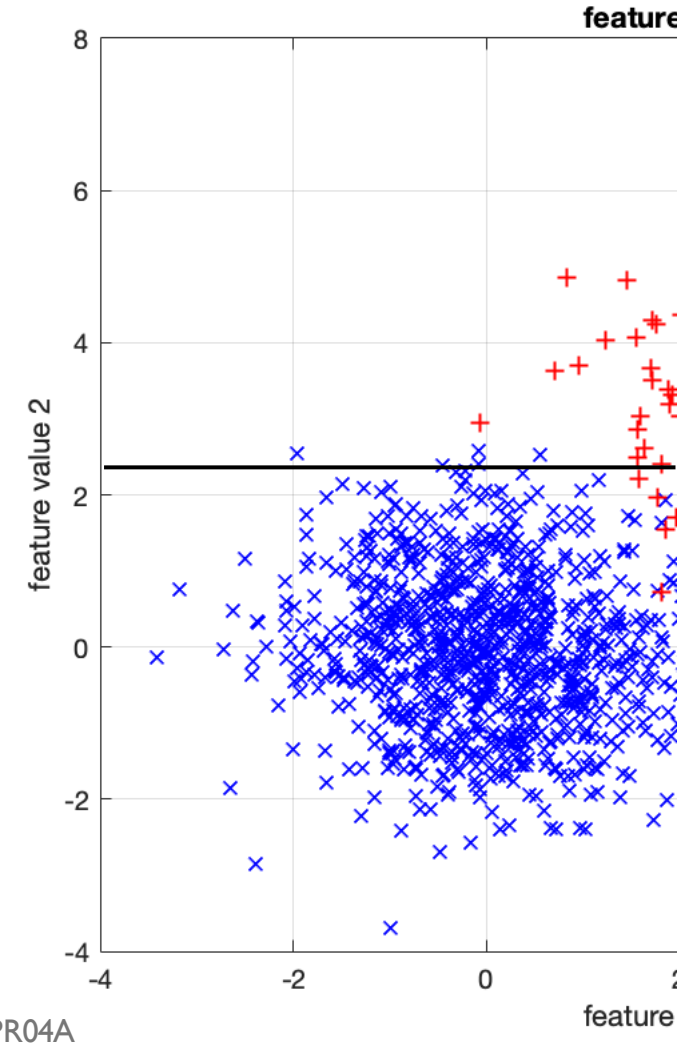


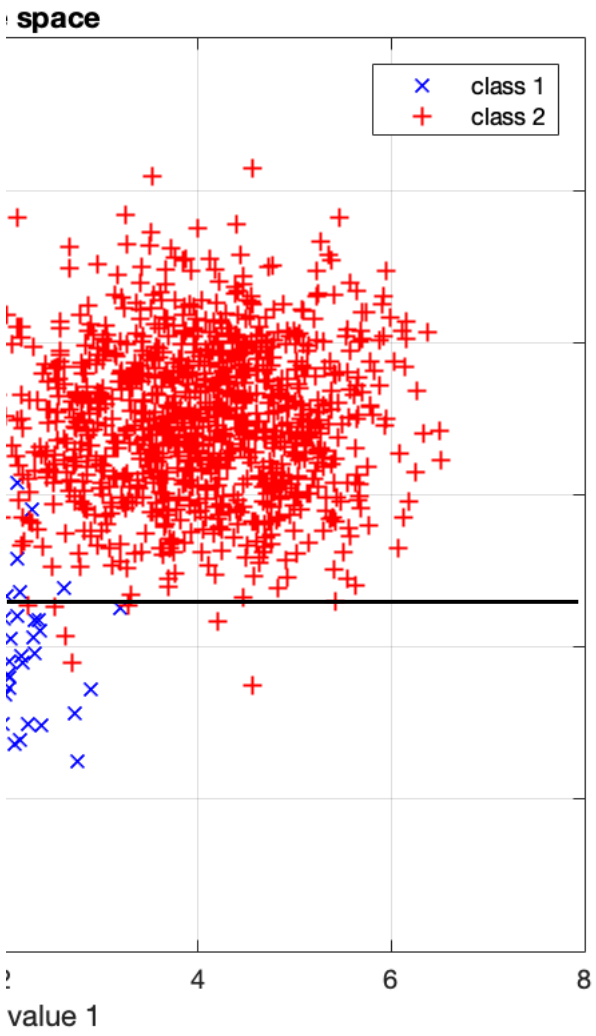
Árboles de Decisión (Training data)



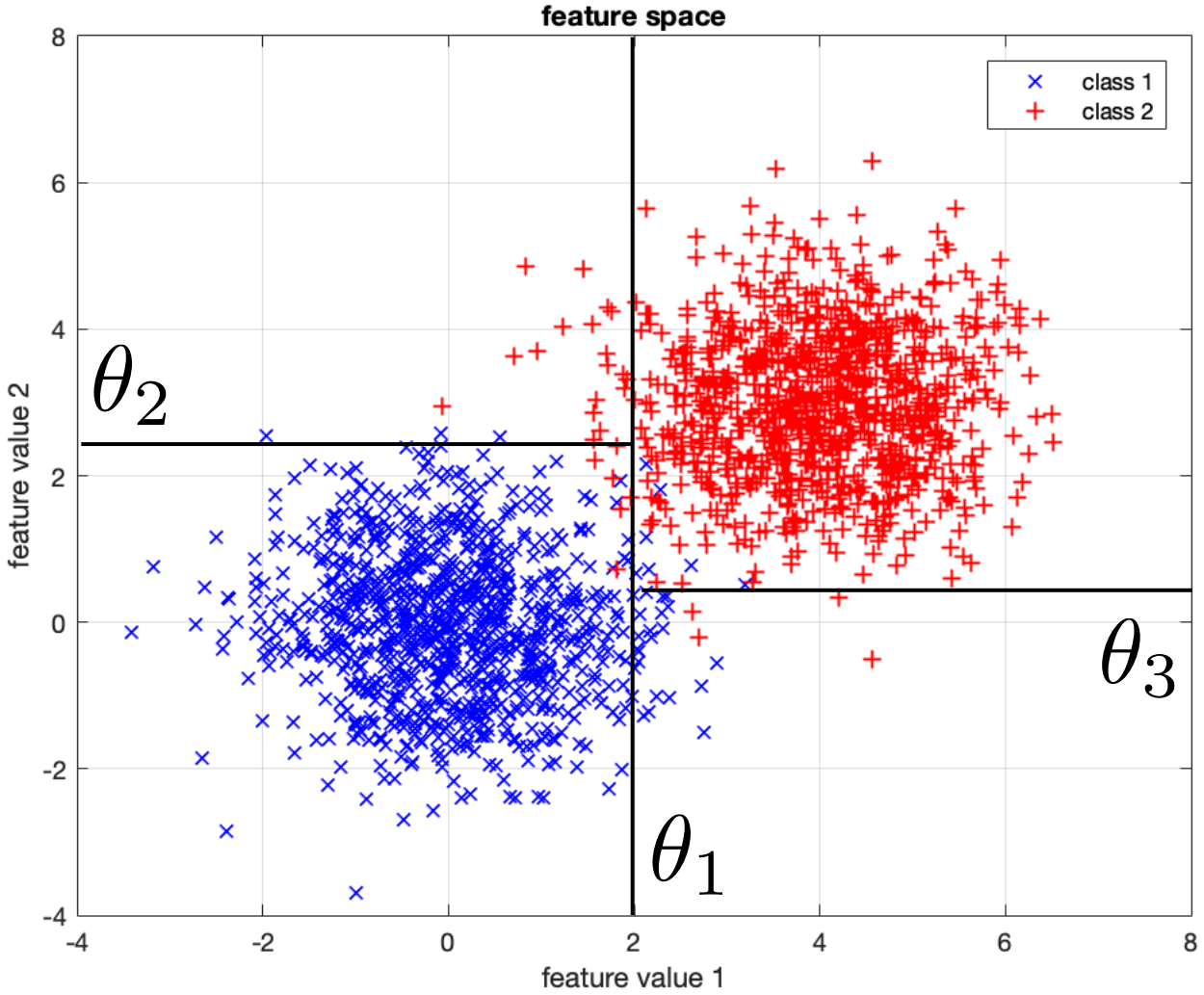


Árboles de Decisión (Training data)



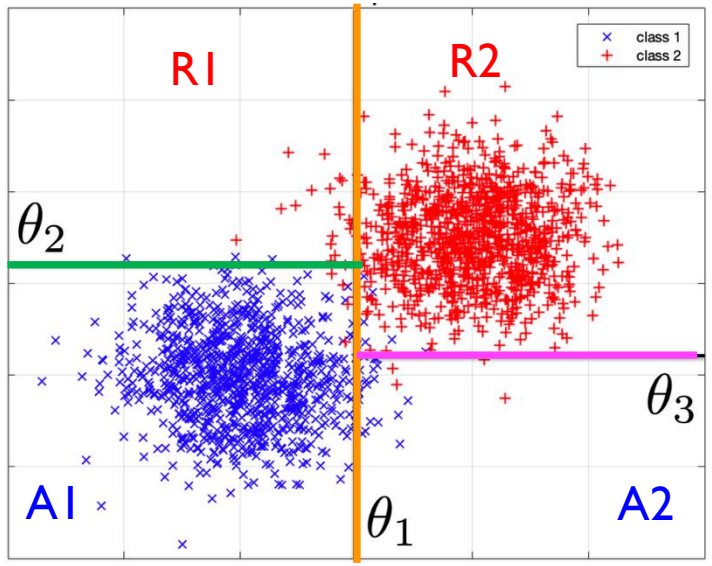


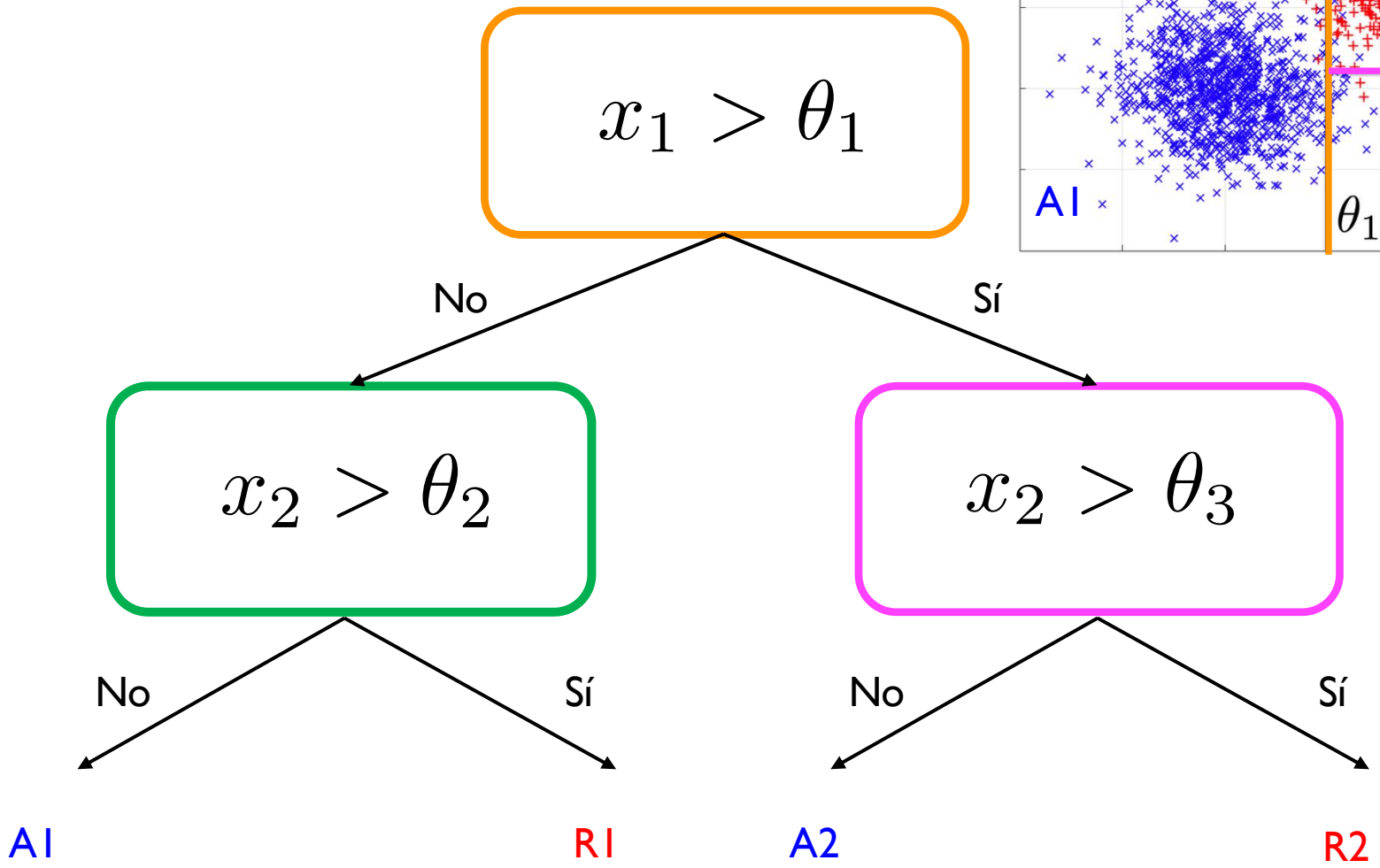
Árboles de Decisión (Training data)



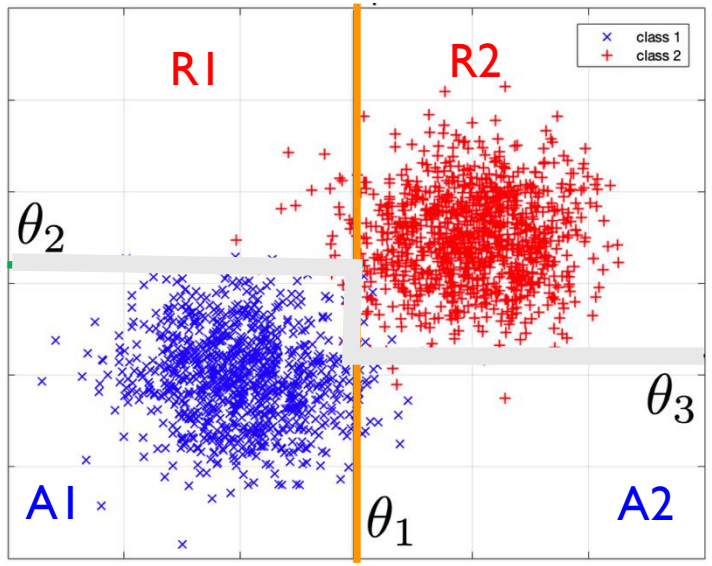
etc...

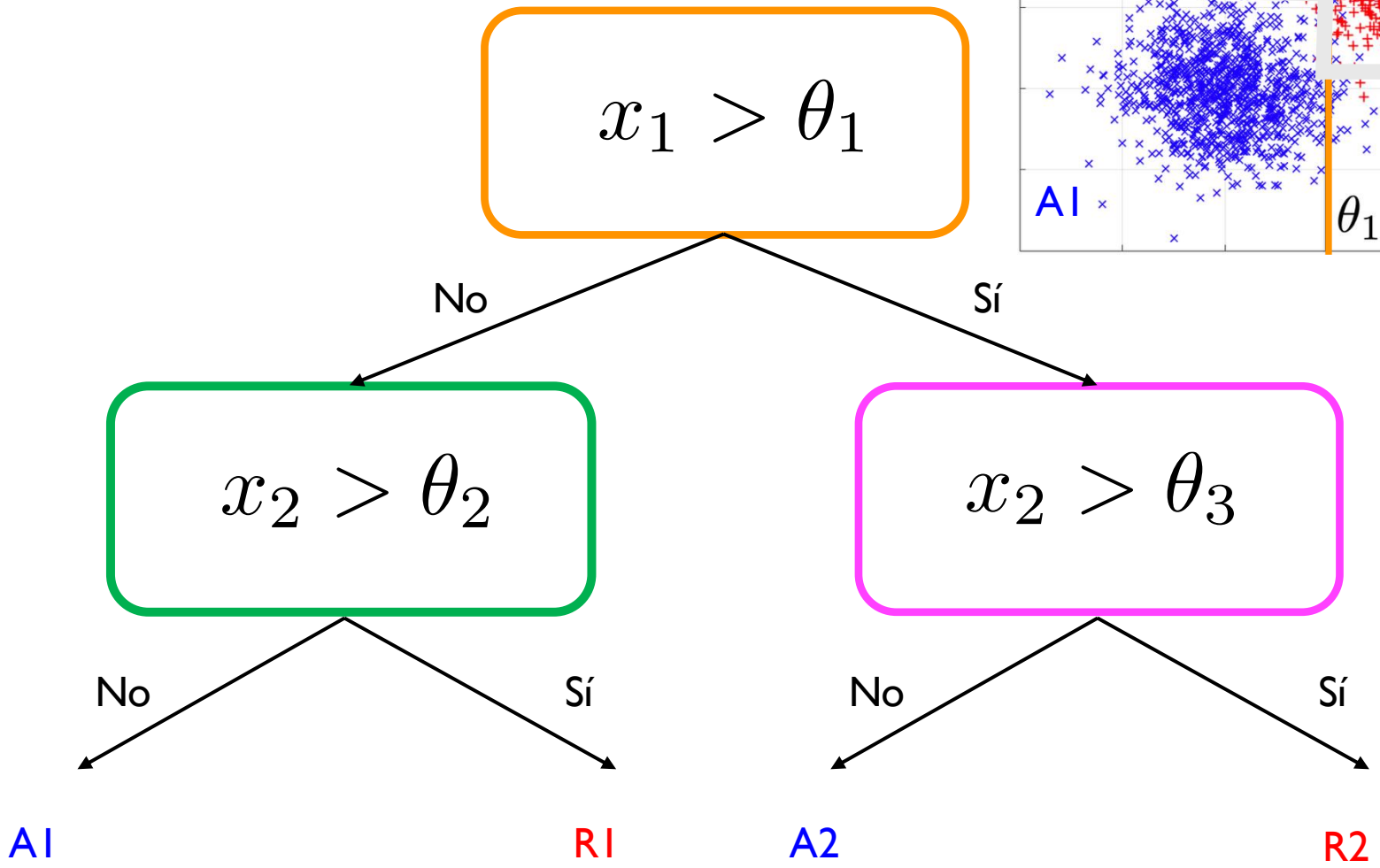
Árbol


class 1
class 2
R1
R2
θ_2
θ_3
A1
θ_1
A2


$x_1 > \theta_1$
No
Sí
$x_2 > \theta_2$
$x_2 > \theta_3$
No
Sí
No
Sí
A1
R1
A2
R2

Árbol


class 1
class 2
R1
R2
θ_2
θ_3
A1
θ_1
A2


$x_1 > \theta_1$
No
Sí
$x_2 > \theta_2$
$x_2 > \theta_3$
No
Sí
No
Sí
A1
R1
A2
R2

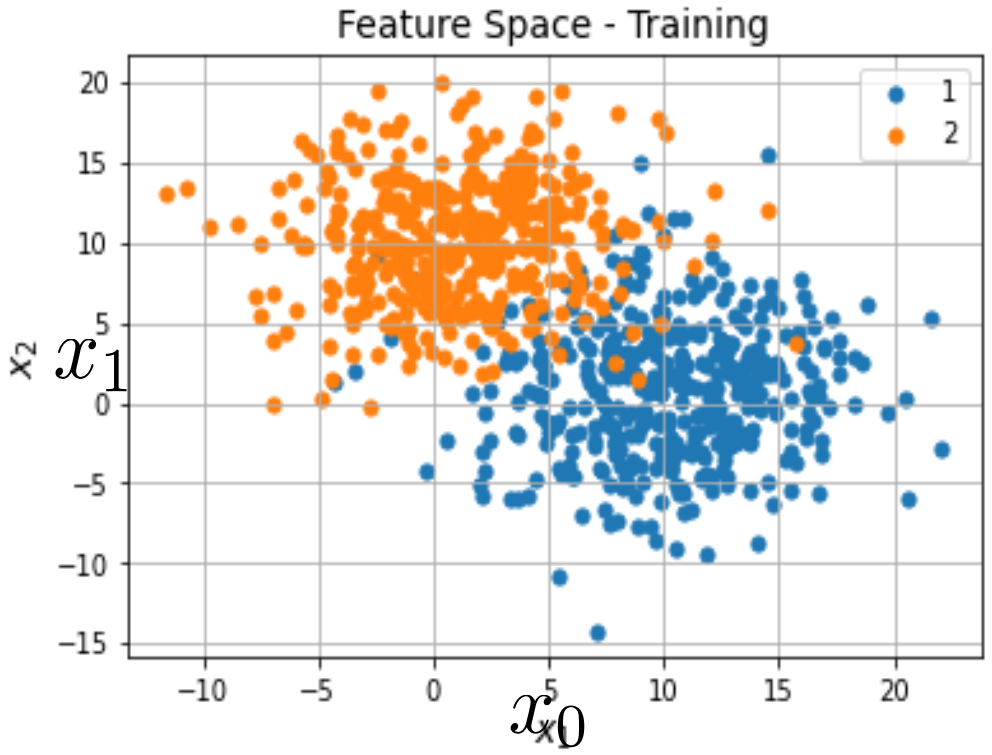
Métricas usadas para el error:

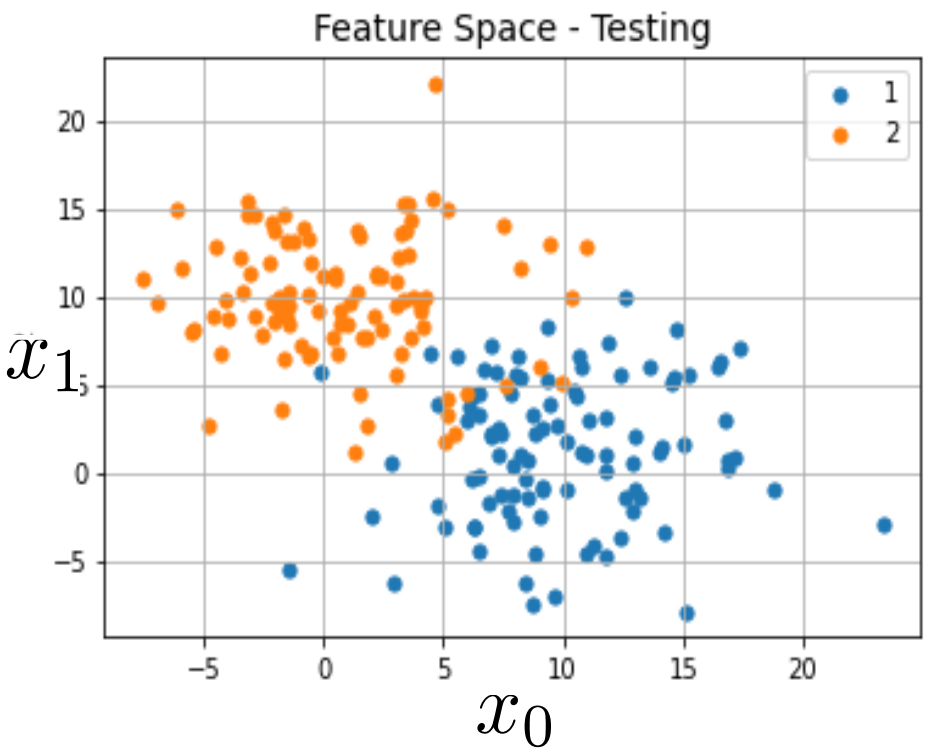
- Error de clasificación 1 - Accuracy
- Entropía $-\sum_{k=1}^{K} p_k \log_2(p_k)$
- Índice Gini $\sum_{k=1}^{K} p_k(1-p_k)$

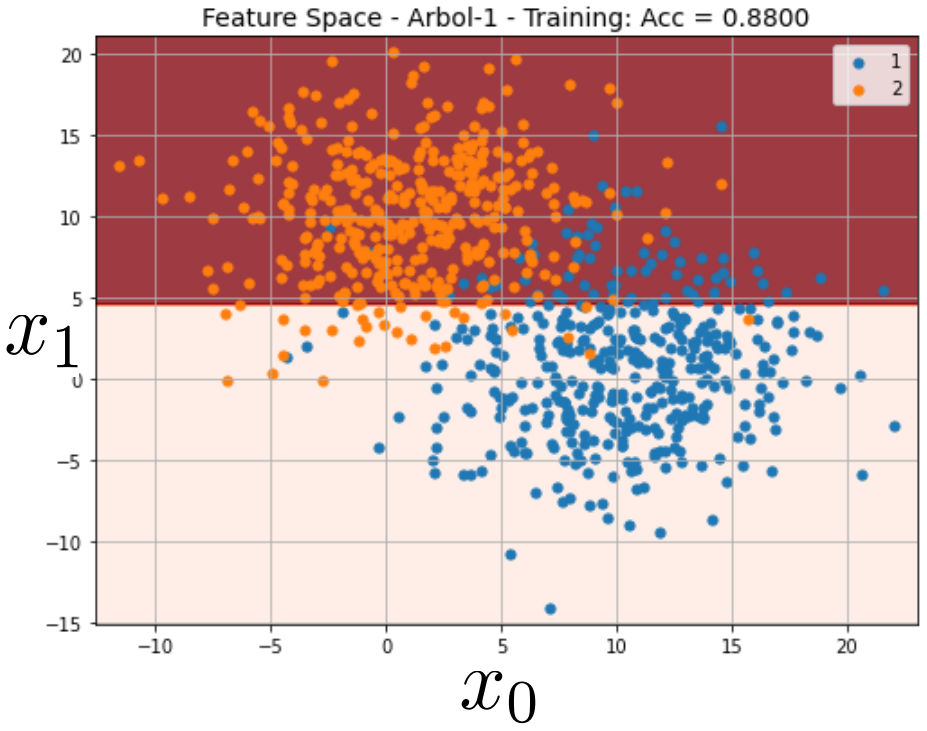
p_k Probabilidad de clasificar bien la clase k

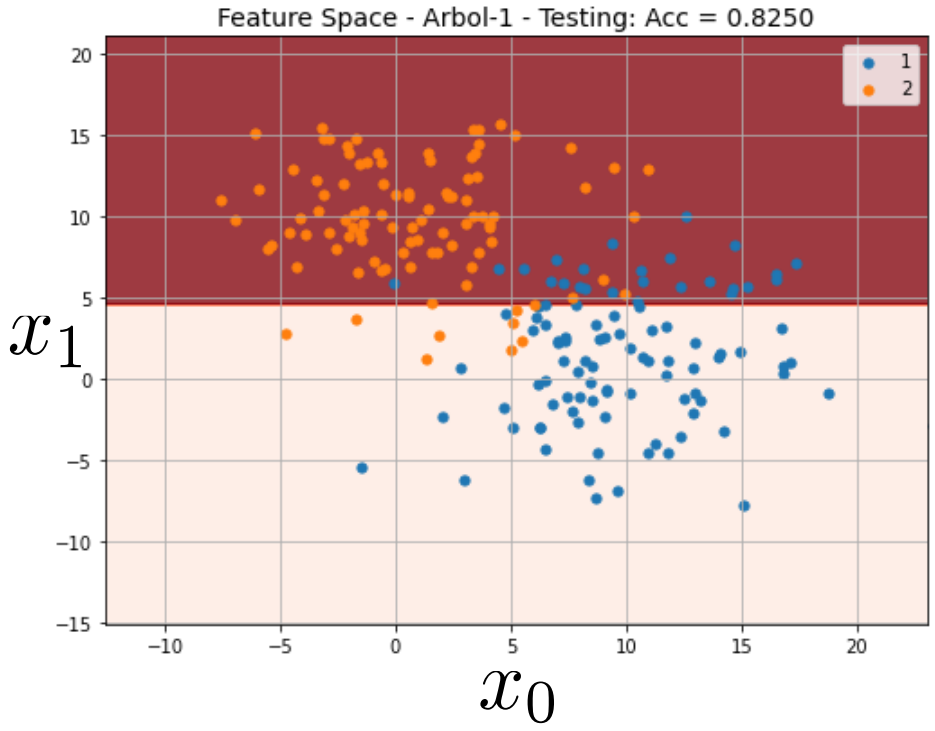
EJEMPLO

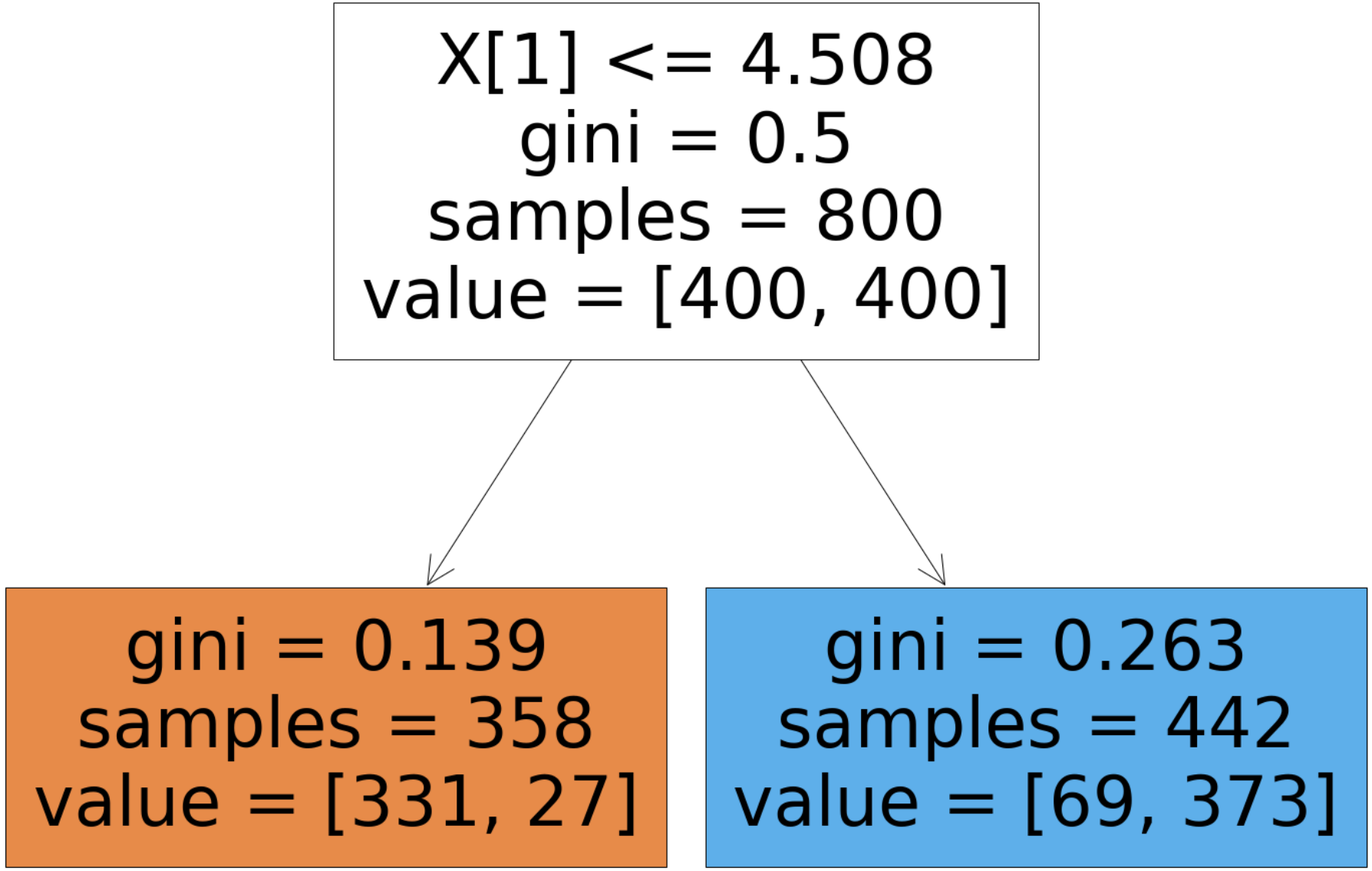
Datos Training/Testing

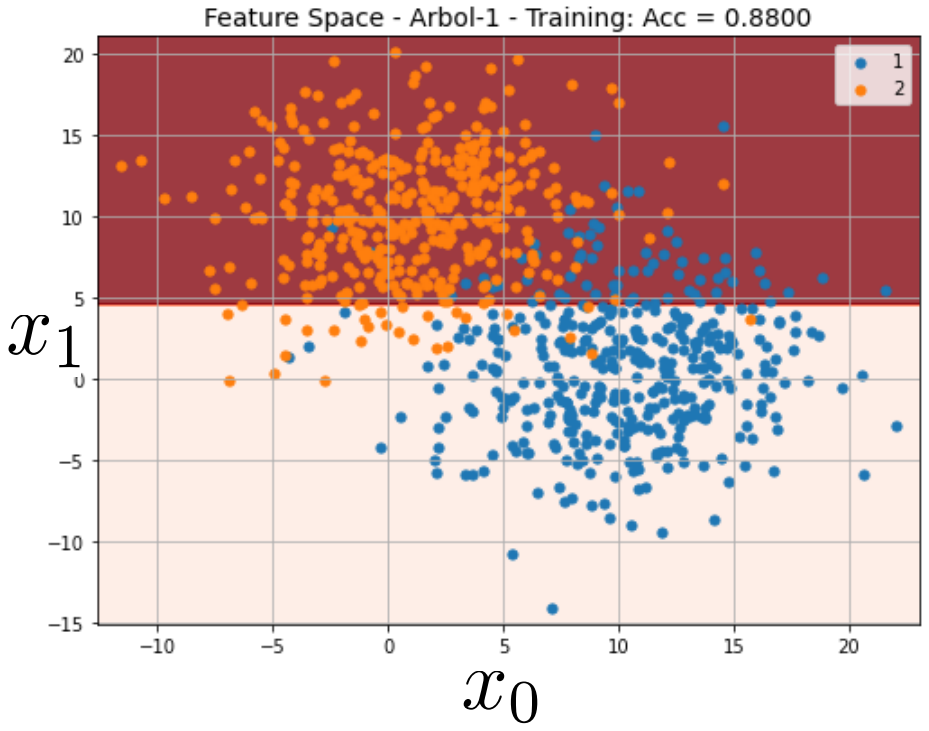


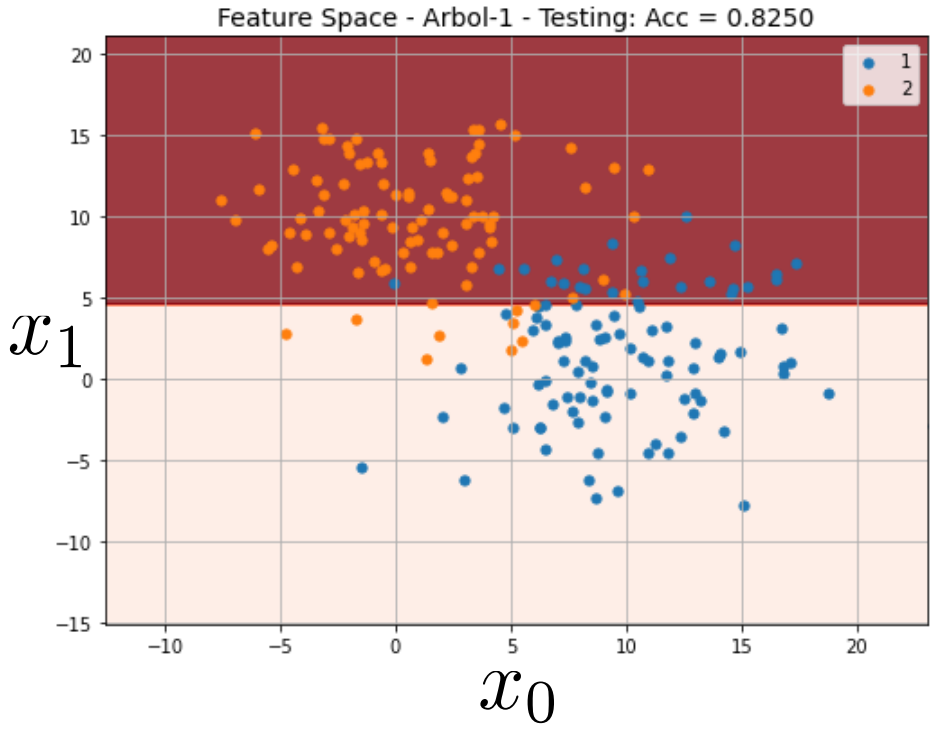


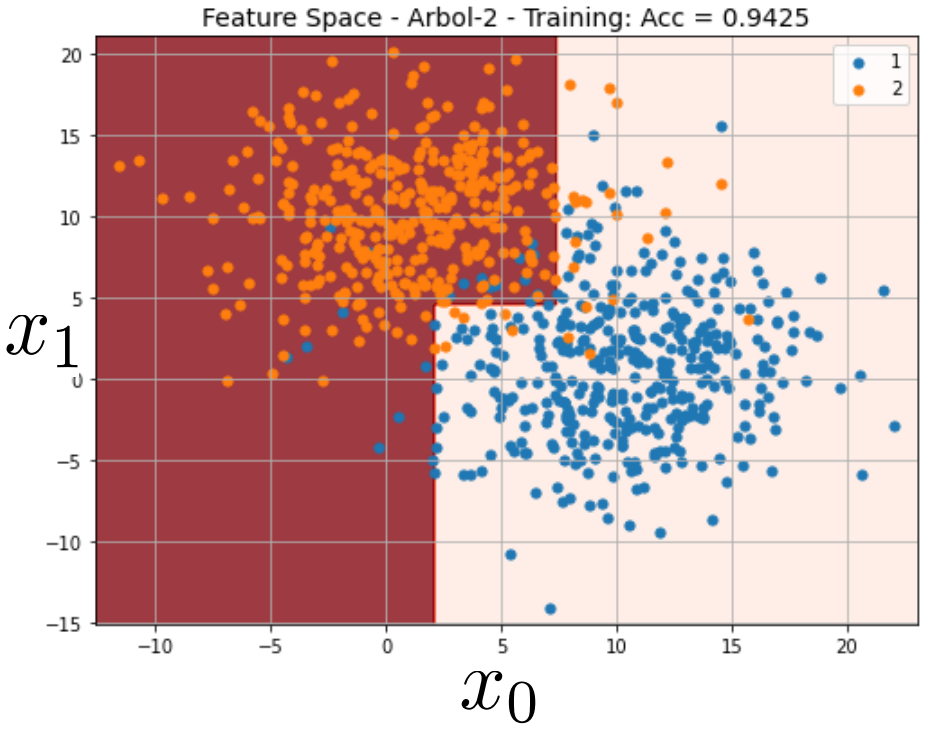

Feature Space - Arbol-1 - Training: Acc = 0.8800
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

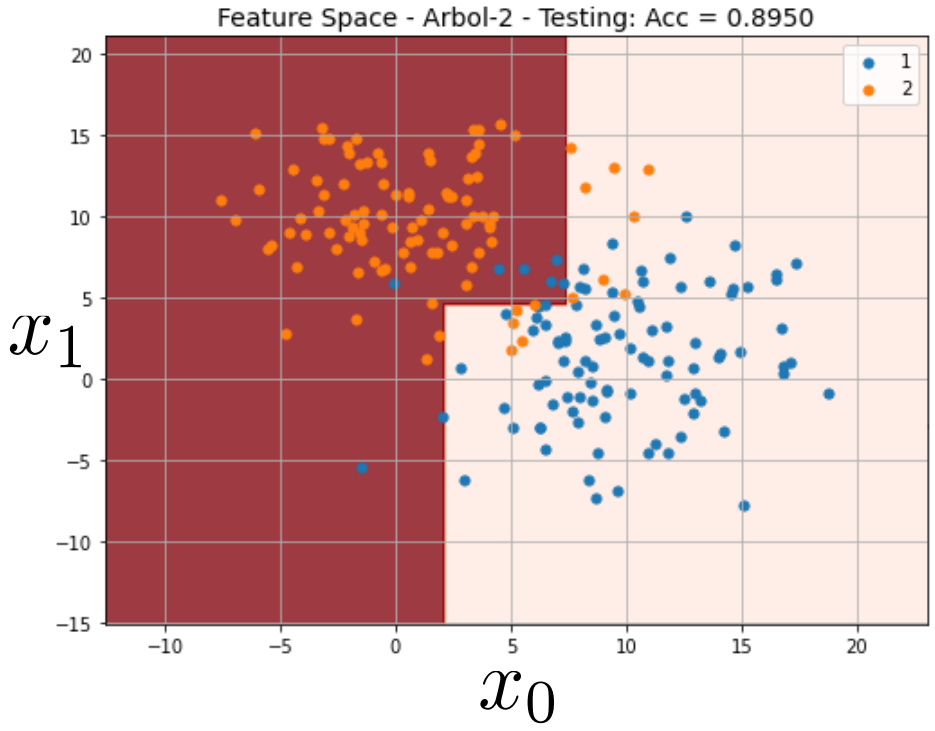

Feature Space - Arbol-1 - Testing: Acc = 0.8250
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

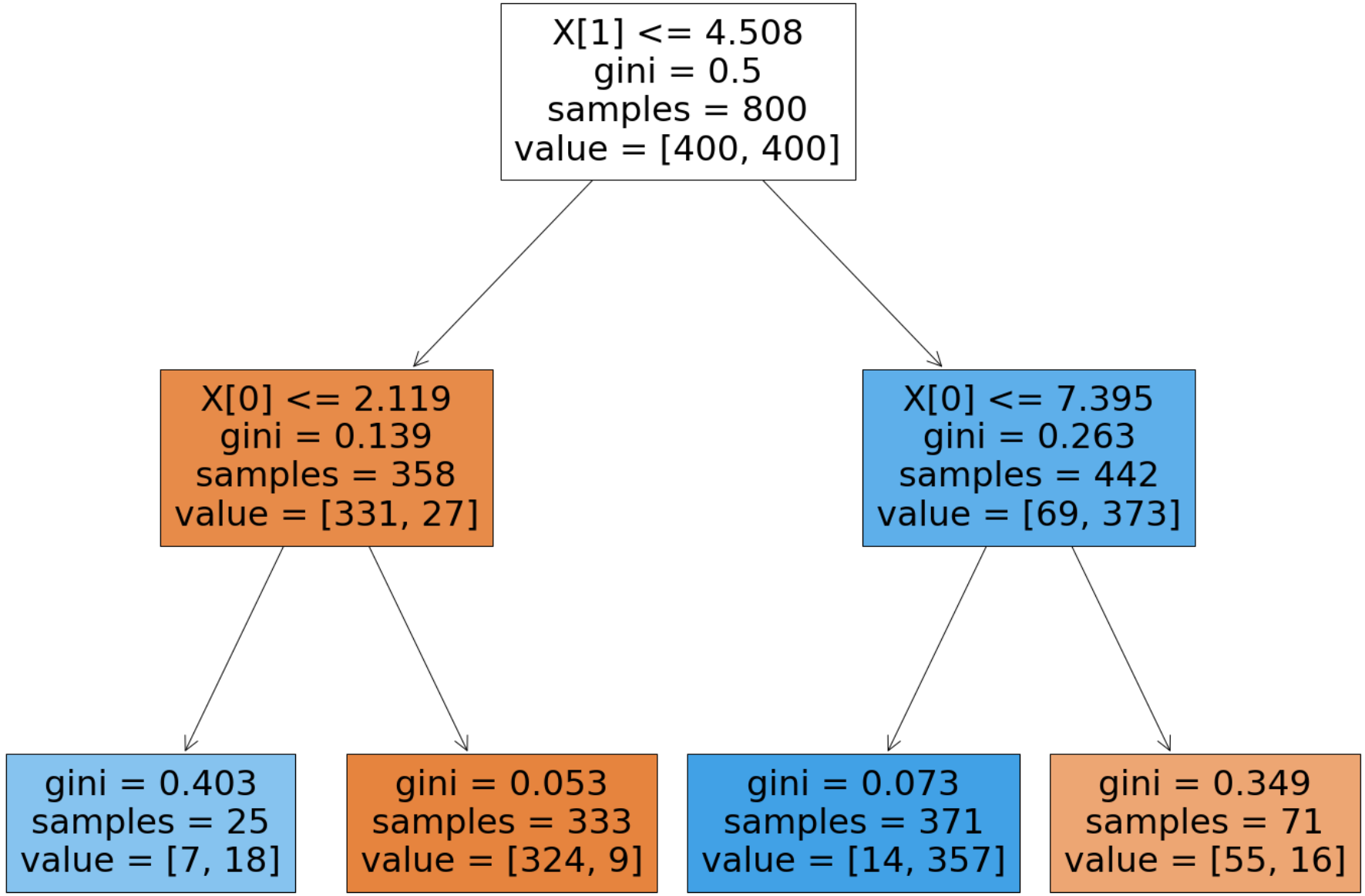

X[1] <= 4.508
gini = 0.5
samples = 800
value = [400, 400]
gini = 0.139
samples = 358
value = [331, 27]
gini = 0.263
samples = 442
value = [69, 373]

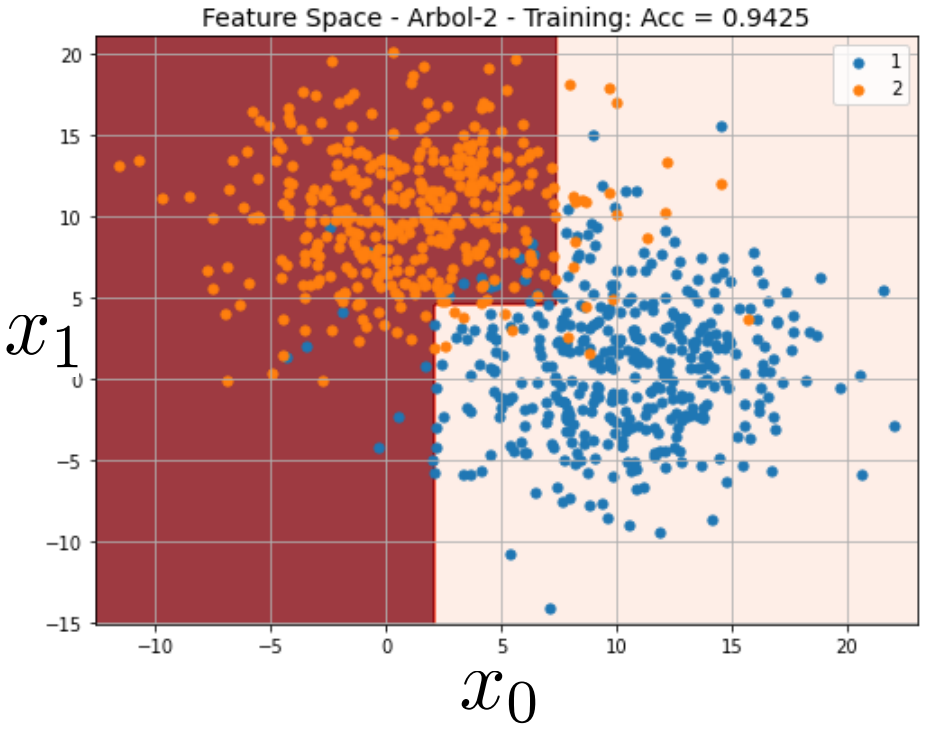

Feature Space - Arbol-1 - Training: Acc = 0.8800
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

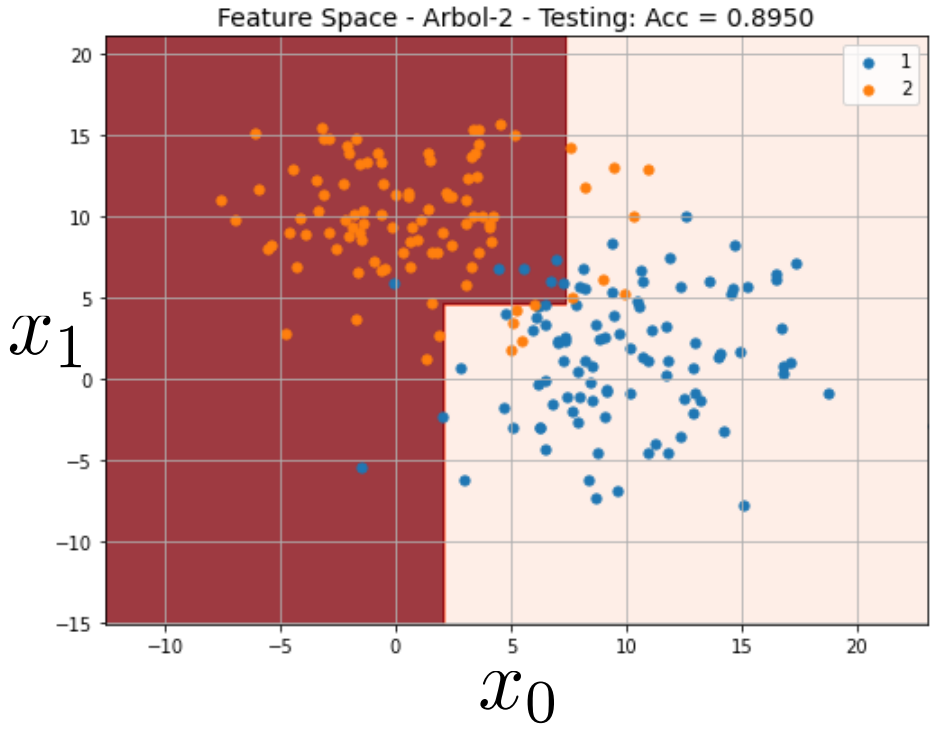

Feature Space - Arbol-1 - Testing: Acc = 0.8250
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

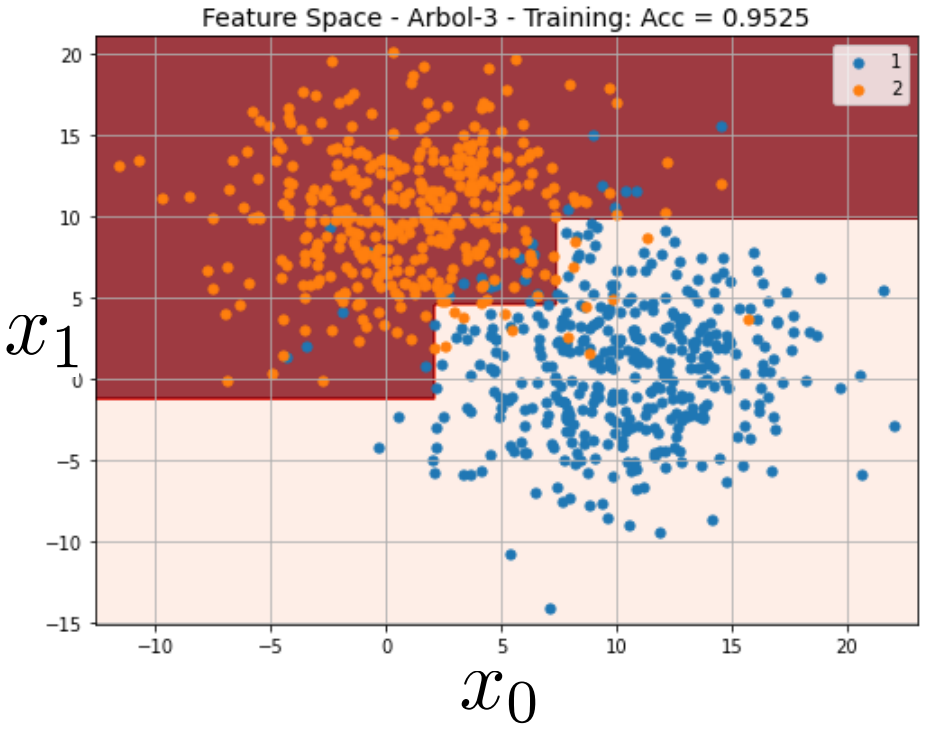

Feature Space - Arbol-2 - Training: Acc = 0.9425
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

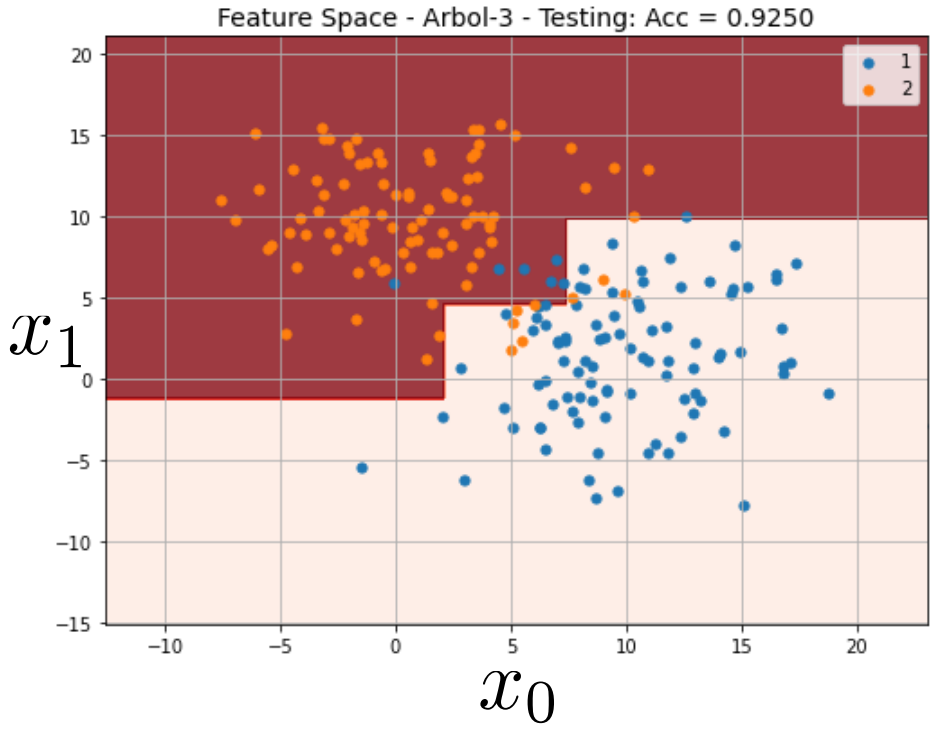

Feature Space - Arbol-2 - Testing: Acc = 0.8950
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

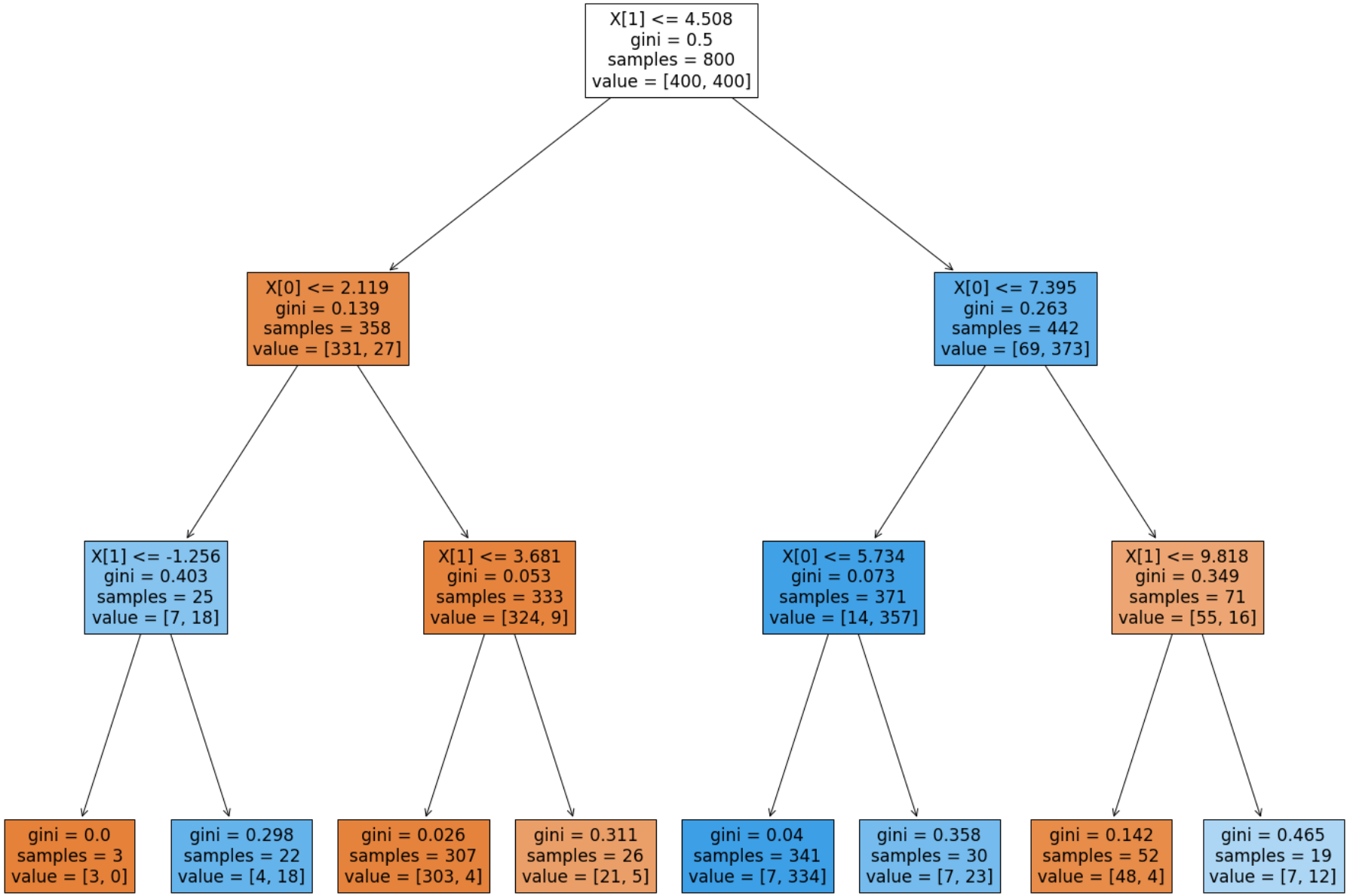

X[1] <= 4.508
gini = 0.5
samples = 800
value = [400, 400]
X[0] <= 2.119
gini = 0.139
samples = 358
value = [331, 27]
X[0] <= 7.395
gini = 0.263
samples = 442
value = [69, 373]
gini = 0.403
samples = 25
value = [7, 18]
gini = 0.053
samples = 333
value = [324, 9]
gini = 0.073
samples = 371
value = [14, 357]
gini = 0.349
samples = 71
value = [55, 16]

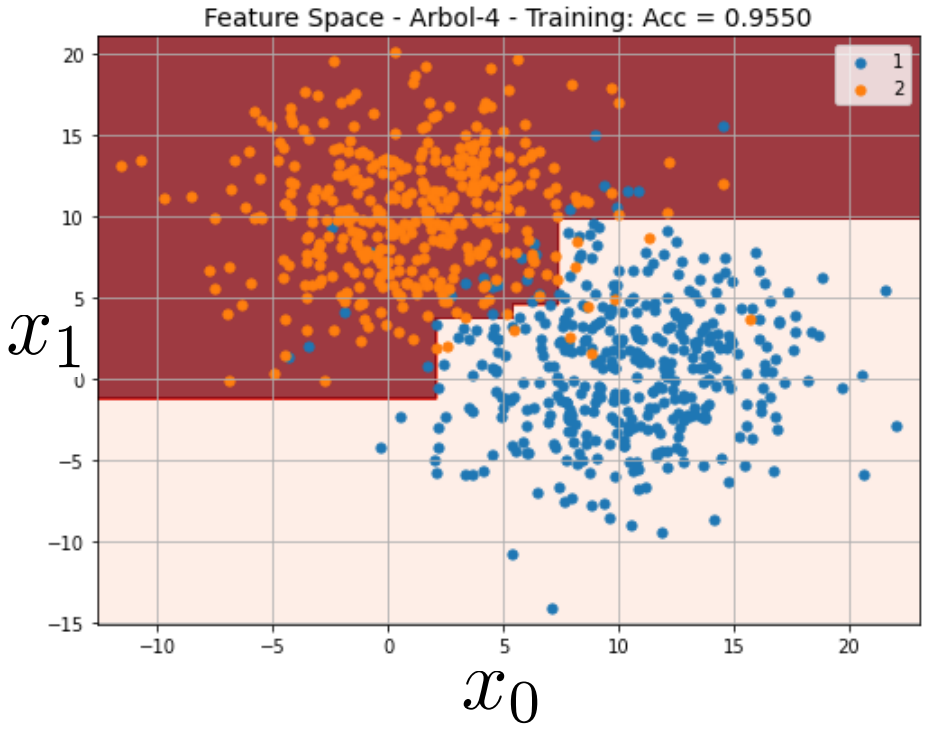

Feature Space - Arbol-2 - Training: Acc = 0.9425
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

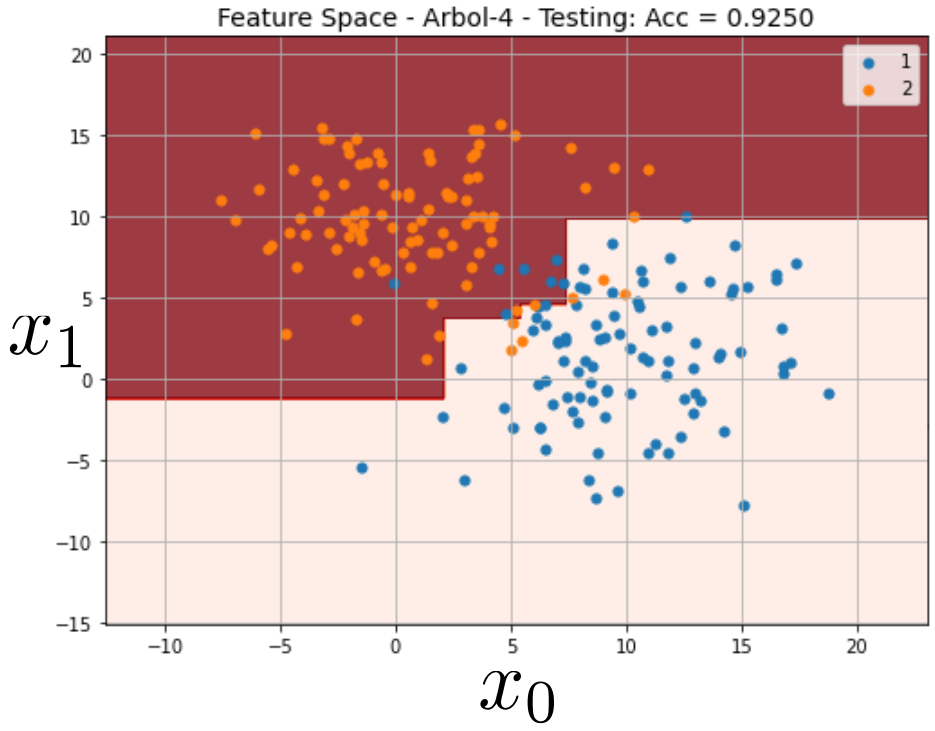

Feature Space - Arbol-2 - Testing: Acc = 0.8950
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

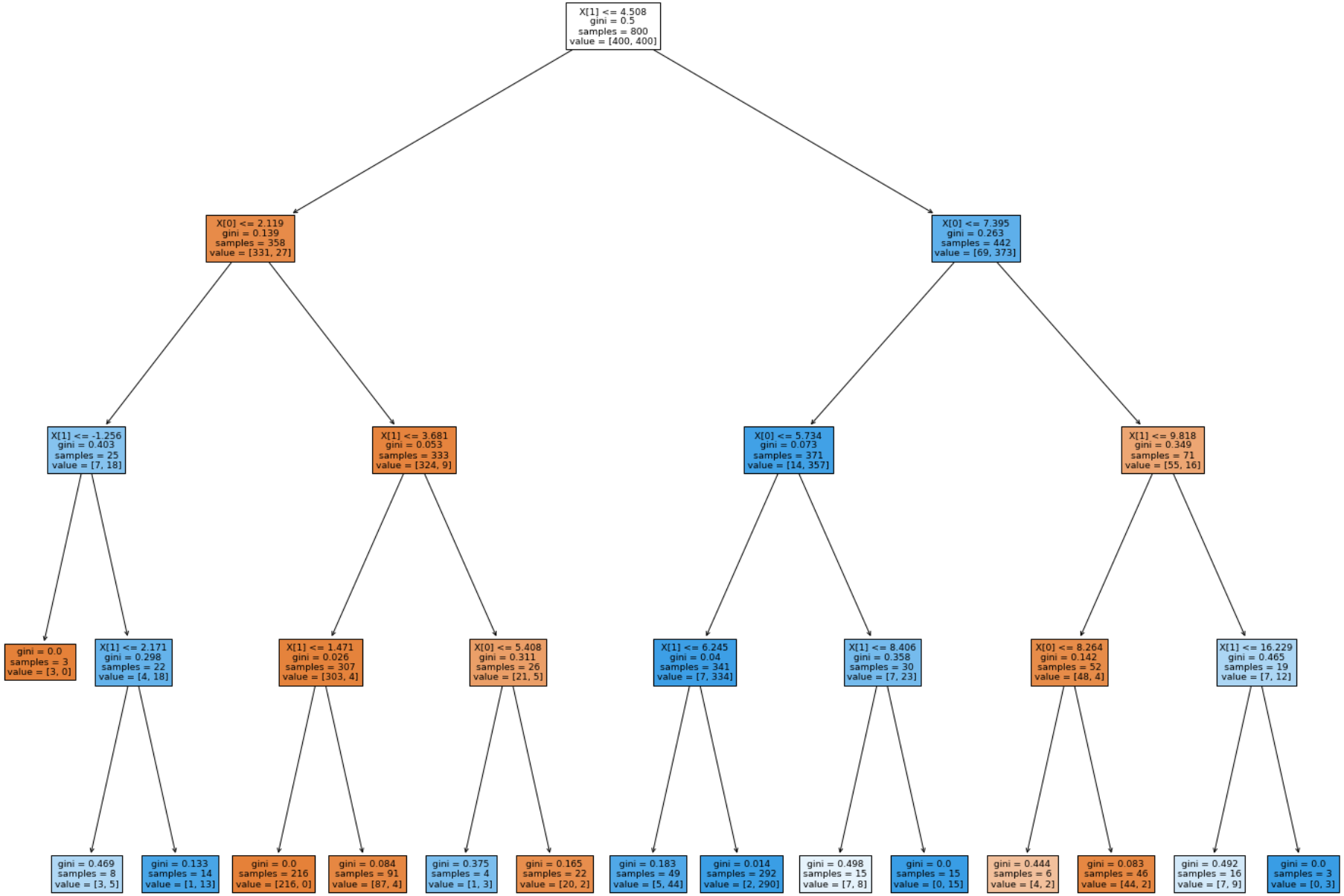

Feature Space - Arbol-3 - Training: Acc = 0.9525
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

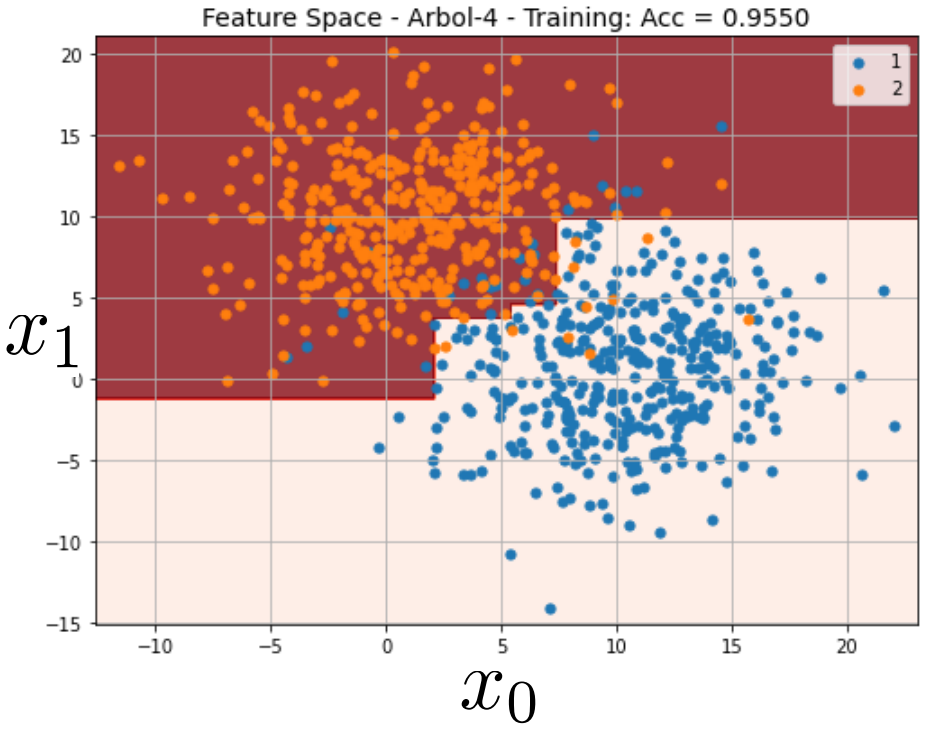

Feature Space - Arbol-3 - Testing: Acc = 0.9250
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

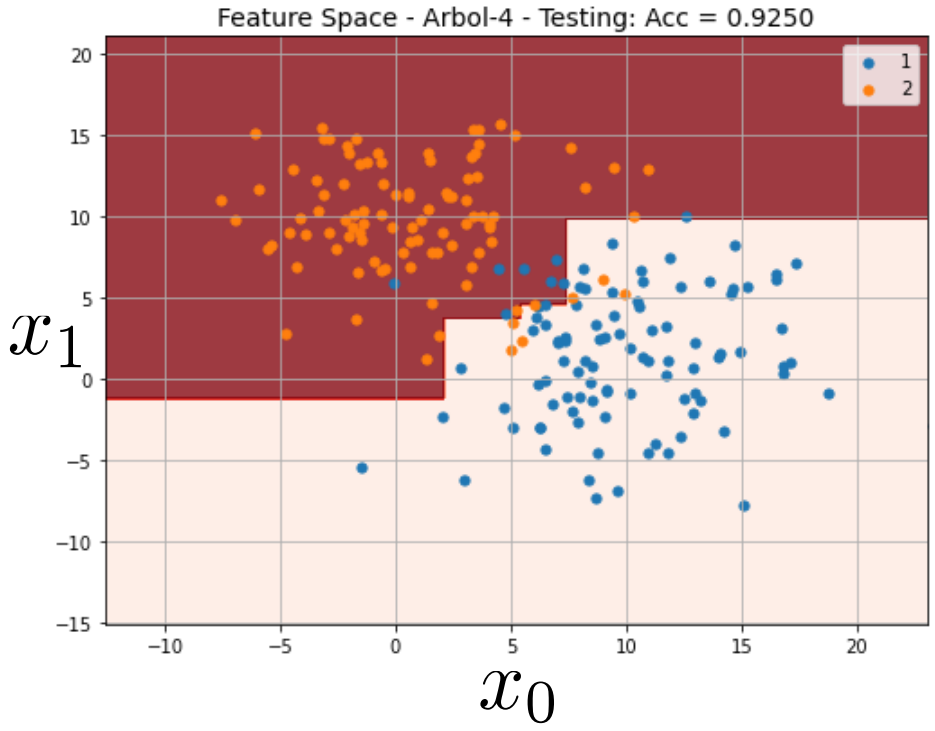

X[1] <= 4.508
gini = 0.5
samples = 800
value = [400, 400]
X[0] <= 2.119
gini = 0.139
samples = 358
value = [331, 27]
X[0] <= 7.395
gini = 0.263
samples = 442
value = [69, 373]
X[1] <= -1.256
gini = 0.403
samples = 25
value = [7, 18]
X[1] <= 3.681
gini = 0.053
samples = 333
value = [324, 9]
X[0] <= 5.734
gini = 0.073
samples = 371
value = [14, 357]
X[1] <= 9.818
gini = 0.349
samples = 71
value = [55, 16]
gini = 0.0
samples = 3
value = [3, 0]
gini = 0.298
samples = 22
value = [4, 18]
gini = 0.026
samples = 307
value = [303, 4]
gini = 0.311
samples = 26
value = [21, 5]
gini = 0.04
samples = 341
value = [7, 334]
gini = 0.358
samples = 30
value = [7, 23]
gini = 0.142
samples = 52
value = [48, 4]
gini = 0.465
samples = 19
value = [7, 12]

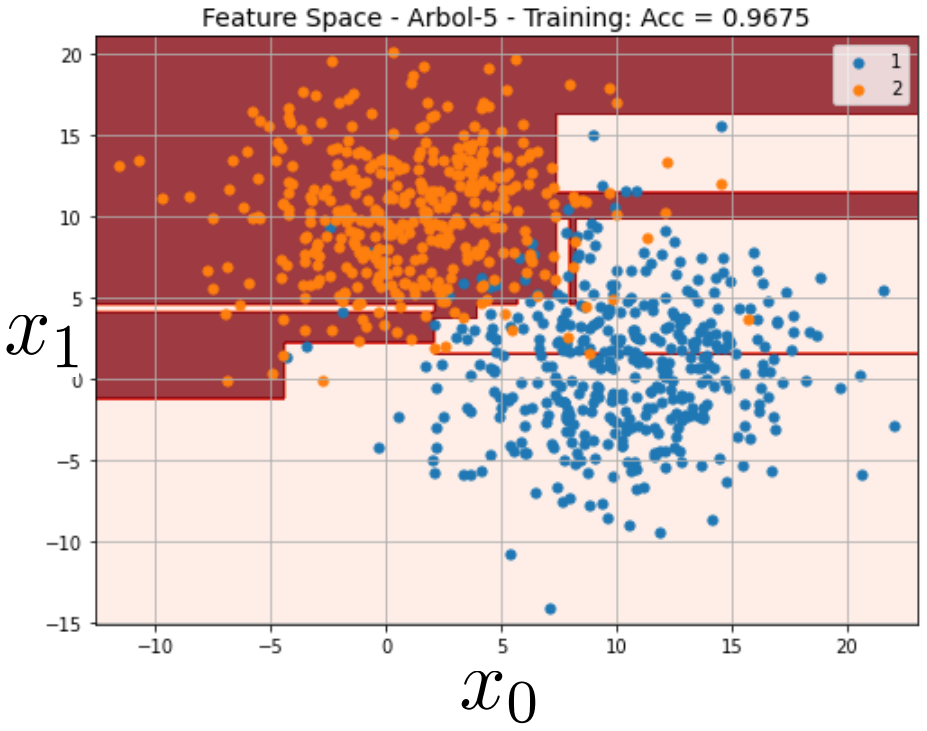

Feature Space - Arbol-4 - Training: Acc = 0.9550
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

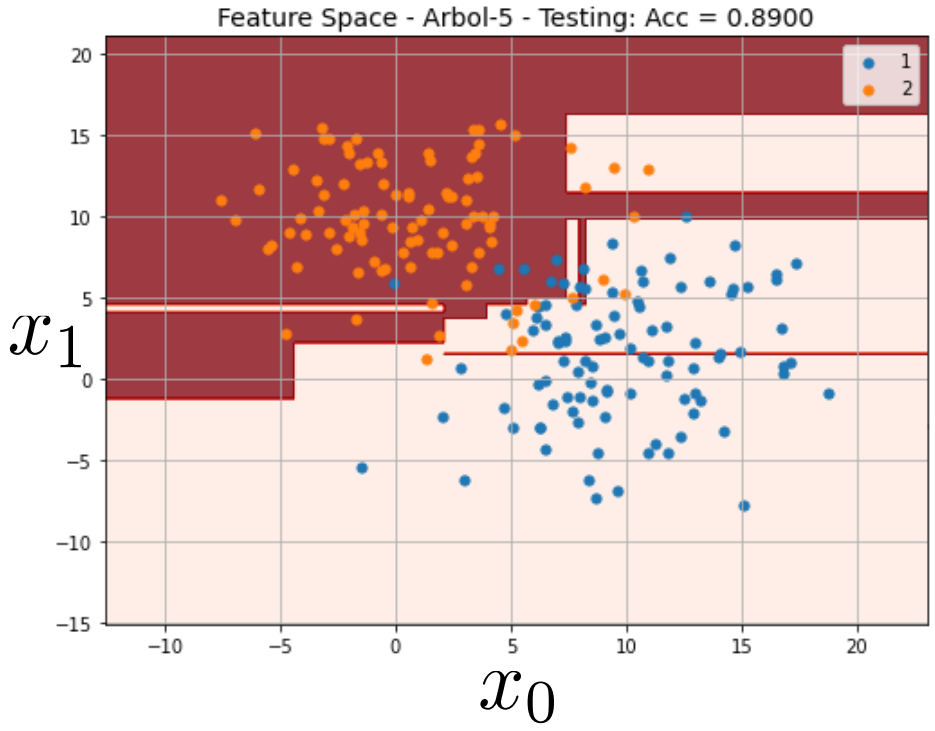

Feature Space - Arbol-4 - Testing: Acc = 0.9250
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0

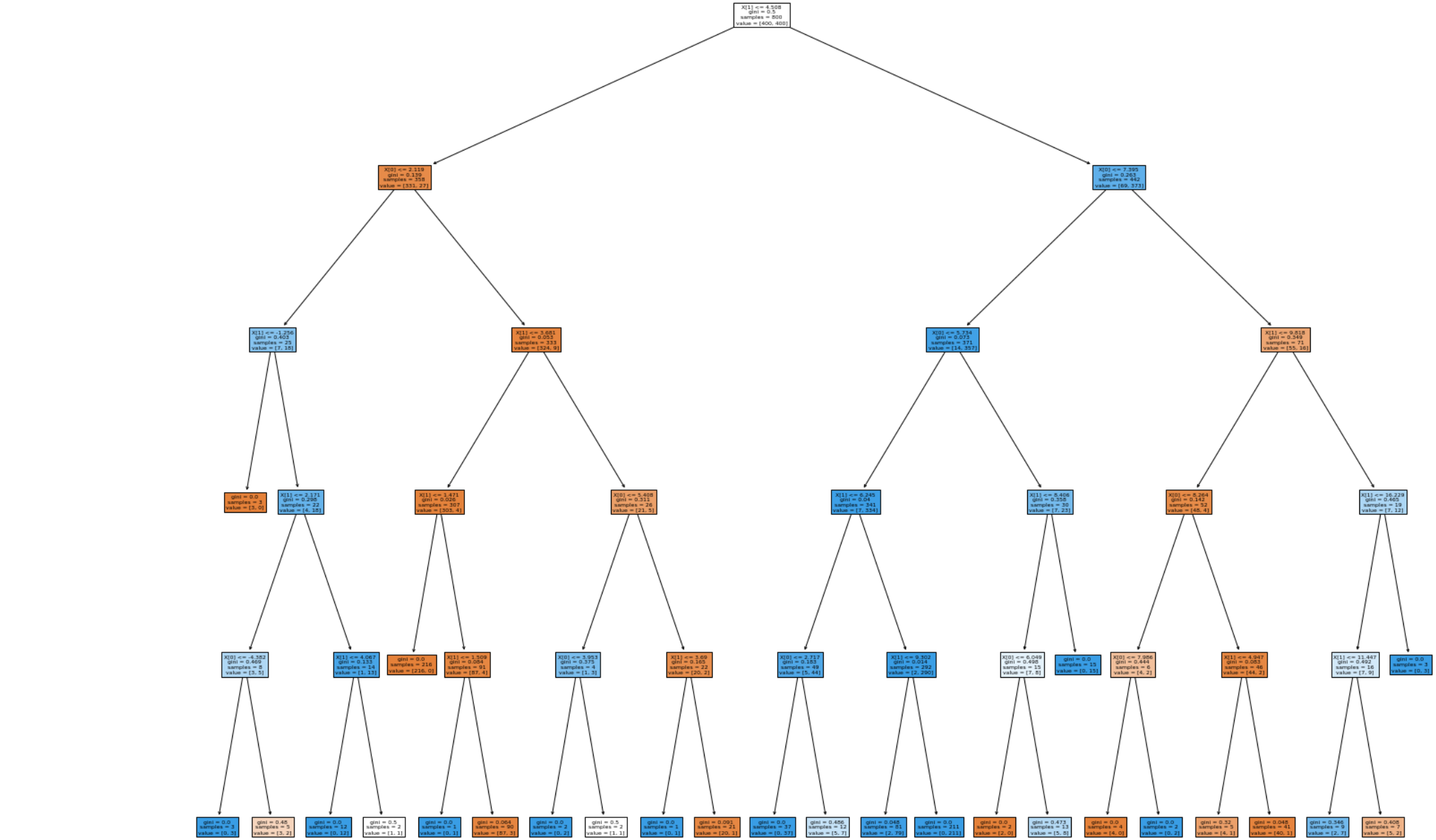

X[1] <= 4.508
gini = 0.5
samples = 800
value = [400, 400]
X[0] <= 2.119
gini = 0.139
samples = 358
value = [331, 27]
X[0] <= 7.395
gini = 0.263
samples = 442
value = [69, 373]
X[1] <= -1.256
gini = 0.403
samples = 25
value = [7, 18]
X[1] <= 3.681
gini = 0.053
samples = 333
value = [324, 9]
X[0] <= 5.734
gini = 0.073
samples = 371
value = [14, 357]
X[1] <= 9.818
gini = 0.349
samples = 71
value = [55, 16]
gini = 0.0
samples = 3
value = [3, 0]
X[1] <= 2.171
gini = 0.298
samples = 22
value = [4, 18]
X[1] <= 1.471
gini = 0.026
samples = 307
value = [303, 4]
X[0] <= 5.408
gini = 0.311
samples = 26
value = [21, 5]
X[1] <= 6.245
gini = 0.04
samples = 341
value = [7, 334]
X[1] <= 8.406
gini = 0.358
samples = 30
value = [7, 23]
X[0] <= 8.264
gini = 0.142
samples = 52
value = [48, 4]
X[1] <= 16.229
gini = 0.465
samples = 19
value = [7, 12]
gini = 0.469
samples = 8
value = [3, 5]
gini = 0.133
samples = 14
value = [1, 13]
gini = 0.0
samples = 216
value = [216, 0]
gini = 0.084
samples = 91
value = [87, 4]
gini = 0.375
samples = 4
value = [1, 3]
gini = 0.165
samples = 22
value = [20, 2]
gini = 0.183
samples = 49
value = [5, 44]
gini = 0.014
samples = 292
value = [2, 290]
gini = 0.498
samples = 15
value = [7, 8]
gini = 0.0
samples = 15
value = [0, 15]
gini = 0.444
samples = 6
value = [4, 2]
gini = 0.083
samples = 46
value = [44, 2]
gini = 0.492
samples = 16
value = [7, 9]
gini = 0.0
samples = 3
value = [0, 3]


Feature Space - Arbol-4 - Training: Acc = 0.9550
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0


Feature Space - Arbol-4 - Testing: Acc = 0.9250
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0


Feature Space - Arbol-5 - Training: Acc = 0.9675
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0


Feature Space - Arbol-5 - Testing: Acc = 0.8900
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0


X[1] <= 4.508
gini = 0.5
samples = 800
value = [400, 400]
X[0] <= 2.119
gini = 0.139
samples = 358
value = [332, 27]
X[0] <= 7.395
gini = 0.263
samples = 442
value = [69, 373]
X[1] <= -1.256
gini = 0.463
samples = 25
value = [7, 18]
X[1] <= 5.681
gini = 0.053
samples = 333
value = [324, 9]
X[0] <= 5.734
gini = 0.073
samples = 371
value = [14, 357]
X[1] <= 9.518
gini = 0.349
samples = 71
value = [55, 16]
gini = 0.0
samples = 3
value = [3, 0]
X[1] <= 2.171
gini = 0.298
samples = 22
value = [4, 18]
X[1] <= 1.471
gini = 0.026
samples = 307
value = [303, 4]
X[0] <= 5.408
gini = 0.311
samples = 26
value = [21, 5]
X[1] <= 8.245
gini = 0.04
samples = 341
value = [7, 334]
X[1] <= 8.406
gini = 0.355
samples = 30
value = [7, 23]
X[0] <= 8.264
gini = 0.142
samples = 52
value = [48, 4]
X[1] <= 16.229
gini = 0.465
samples = 19
value = [7, 12]
X[0] <= 4.382
gini = 0.469
samples = 8
value = [3, 5]
X[1] <= 4.067
gini = 0.133
samples = 14
value = [1, 13]
gini = 0.0
samples = 216
value = [216, 0]
X[1] <= 1.509
gini = 0.054
samples = 91
value = [87, 4]
X[0] <= 3.953
gini = 0.375
samples = 4
value = [1, 3]
X[1] <= 3.69
gini = 0.165
samples = 22
value = [20, 2]
X[0] <= 2.717
gini = 0.153
samples = 49
value = [5, 44]
X[1] <= 9.302
gini = 0.014
samples = 292
value = [2, 290]
X[0] <= 6.049
gini = 0.496
samples = 15
value = [7, 8]
gini = 0.0
samples = 15
value = [0, 15]
X[0] <= 7.986
gini = 0.444
samples = 6
value = [4, 2]
X[1] <= 4.947
gini = 0.053
samples = 46
value = [44, 2]
X[1] <= 11.447
gini = 0.492
samples = 16
value = [7, 9]
gini = 0.0
samples = 3
value = [0, 3]
gini = 0.0
samples = 3
value = [3, 0]
gini = 0.48
samples = 5
value = [3, 2]
gini = 0.0
samples = 12
value = [0, 12]
gini = 0.5
samples = 2
value = [1, 1]
gini = 0.0
samples = 1
value = [0, 1]
gini = 0.064
samples = 90
value = [87, 3]
gini = 0.0
samples = 2
value = [0, 2]
gini = 0.5
samples = 2
value = [1, 1]
gini = 0.0
samples = 1
value = [0, 1]
gini = 0.091
samples = 21
value = [20, 1]
gini = 0.0
samples = 37
value = [0, 37]
gini = 0.486
samples = 12
value = [5, 7]
gini = 0.045
samples = 81
value = [2, 79]
gini = 0.0
samples = 211
value = [0, 211]
gini = 0.0
samples = 2
value = [2, 0]
gini = 0.473
samples = 13
value = [5, 8]
gini = 0.0
samples = 4
value = [4, 0]
gini = 0.0
samples = 2
value = [0, 2]
gini = 0.32
samples = 5
value = [4, 1]
gini = 0.048
samples = 41
value = [40, 1]
gini = 0.346
samples = 9
value = [2, 7]
gini = 0.408
samples = 7
value = [5, 2]

Random Forest

Random Forest

TRAINING:

for i = 1 to n

Escoger aleatoriamente un subconjunto de training

Entrenar un árbol de decisión A_i

Random Forest

TRAINING:

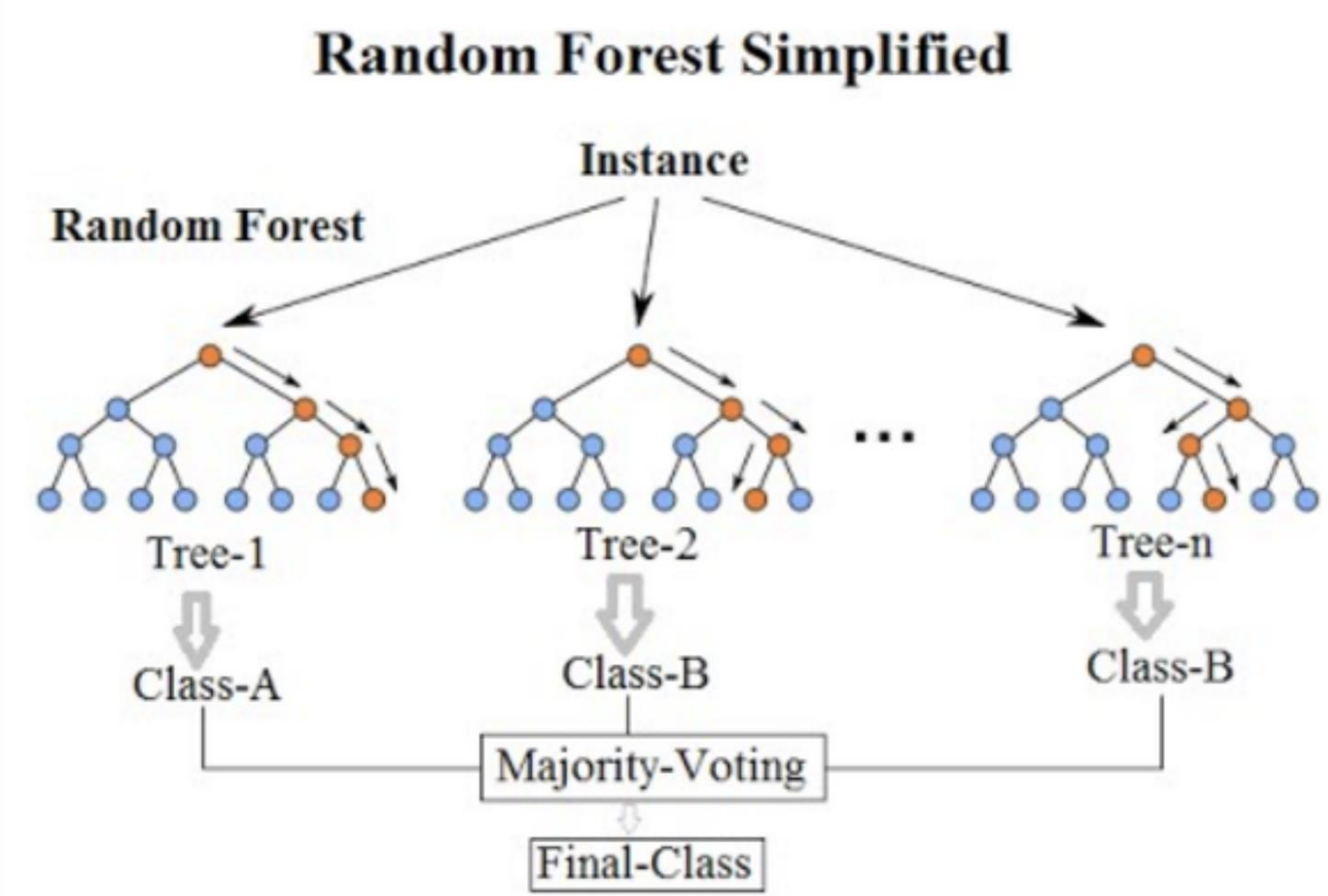
for i = 1 to n
    Escoger aleatoriamente un subconjunto de training
    Entrenar un árbol de decisión A_i

TESTING:

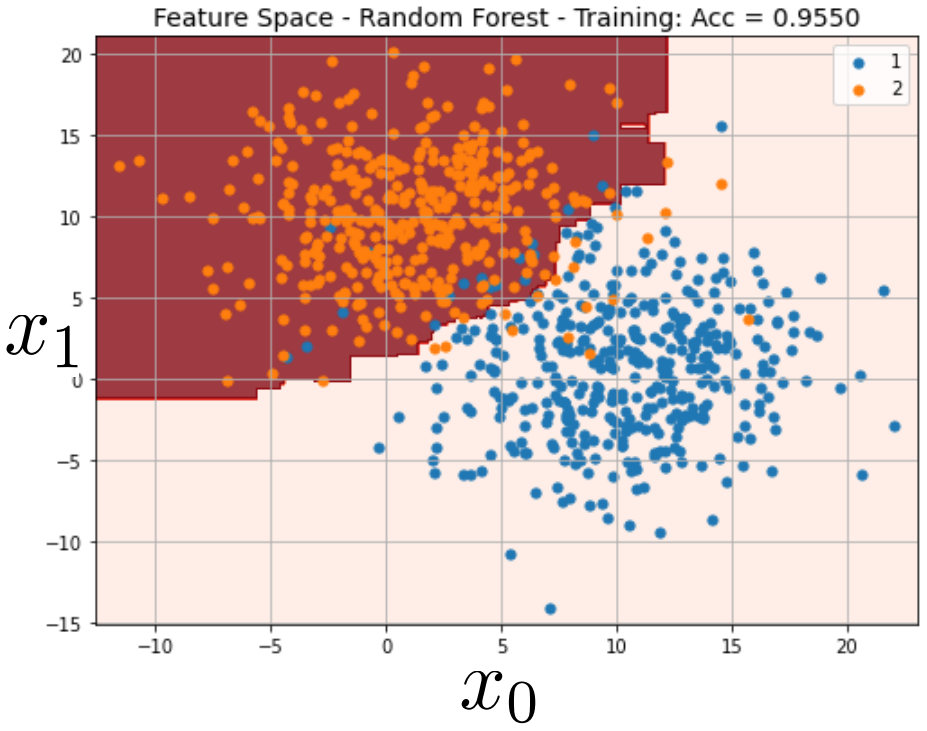
for i = 1 to n
    Clasificar la muestra de testing usando A_i

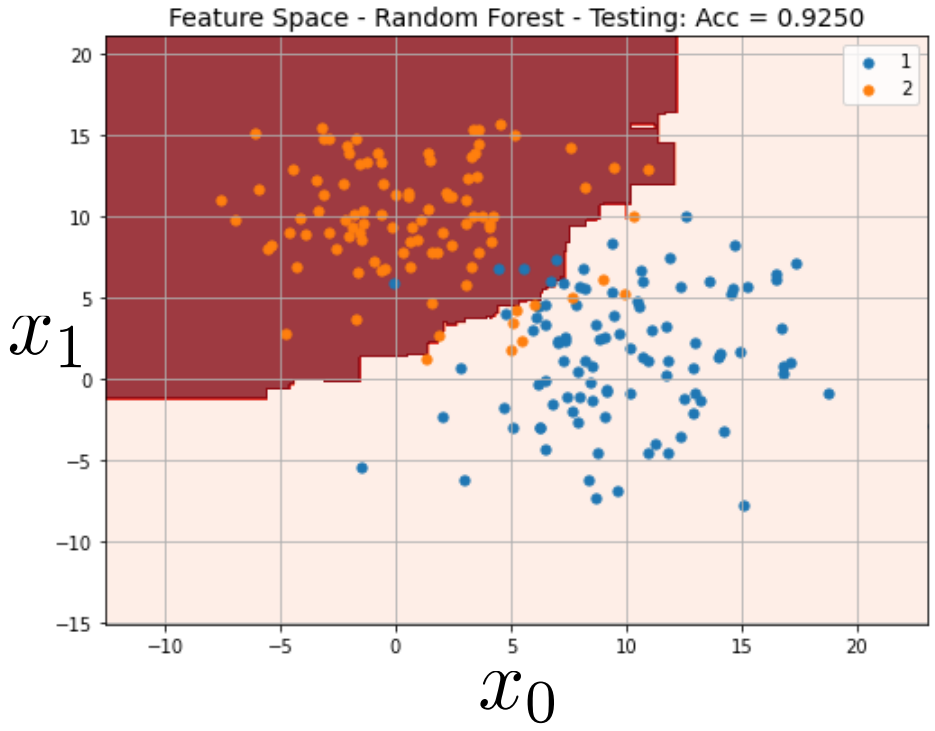
Clasificar la muestra según la mayoría de los n votos

Random Forest



https://en.wikipedia.org/wiki/Random_forest


Feature Space - Random Forest - Training: Acc = 0.9550
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0


Feature Space - Random Forest - Testing: Acc = 0.9250
1
2
20
15
10
5
0
−5
−10
−15
−10
−5
0
5
10
15
20
x_1
x_0